

**PERANCANGAN DAN IMPLEMENTASI
WAREHOUSE MANAGEMENT CONTROLLER &
SISTEM LOKALISASI UNTUK PROTOTIPE
SMART WAREHOUSE**

Laporan Tugas Akhir

Oleh

**Raizan iqbal Resi
18222068**



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Juli 2026

LEMBAR PENGESAHAN

Perancangan dan Implementasi Warehouse Management Controller & Sistem Lokalisasi untuk Prototipe Smart Warehouse

Laporan Tugas Akhir

Oleh

**Raizan Iqbal Resi
18222068**

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Laporan Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 13 Juli 2026

Pembimbing 1

Pembimbing 2

Ary Setijadi Prihatmanto, M.T.

I Gusti Bagus Baskara Nugraha, S.T.,
M.T., Ph.D.

NIP. 123456789

NIP. 9197601242010121001

LEMBAR PERNYATAAN

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tugas Akhir ini adalah hasil karya saya sendiri yang dibuat dengan sebenar-benarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan laporan tugas akhir ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan laporan tugas akhir.
3. Isi laporan ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa laporan ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 13 Juli 2026

John Doe

NIM: 18299000

ABSTRAK

Implementasi Sistem Smart Scale Berbasis IoT dengan Arsitektur Desentralisasi untuk Optimalisasi Rantai Pasok Jeruk di Tingkat Lapak

oleh

John Doe

123456789

Proses manual dalam rantai pasok jeruk di tingkat lapak, yang mencakup sortir, timbang, dan catat, terbukti tidak efisien, memakan waktu, dan rentan terhadap kesalahan. Meskipun solusi berbasis Internet of Things (IoT) dapat mengatasi masalah ini, arsitektur terpusat konvensional berbasis cloud justru menimbulkan masalah baru berupa latensi tinggi yang menghambat alur kerja real-time. Penelitian ini bertujuan untuk merancang dan mengimplementasikan sebuah purwarupa sistem smart scale berbasis IoT dengan pendekatan desentralisasi (edge computing) untuk meminimalkan latensi dan meningkatkan efisiensi operasional. Metode yang digunakan adalah Model Purwarupa, yang diwujudkan dalam bentuk perangkat keras dengan arsitektur prosesor ganda (Raspberry Pi 5 dan ESP32) yang mampu mengintegrasikan fungsi timbang dan analisis kualitas citra secara simultan. Hasil evaluasi menunjukkan bahwa purwarupa dengan arsitektur edge computing yang diimplementasikan mampu memberikan respons 43,5% lebih cepat (latensi rata-rata 18,05 detik) dibandingkan dengan simulasi arsitektur terpusat. Selain itu, sistem ini berhasil meningkatkan efisiensi waktu kerja secara keseluruhan sebesar 58,32% jika dibandingkan dengan metode manual konvensional. Pengujian Penerimaan Pengguna (UAT) yang melibatkan 30 partisipan juga menunjukkan penerimaan yang sangat positif, dengan skor rata-rata untuk kemudahan penggunaan (5,0/5,0), kejelasan antarmuka (4,87/5,0) dan persepsi kinerja (4,95/5,0). Kesimpulannya, implementasi sistem smart scale dengan arsitektur desentralisasi terbukti merupakan solusi yang valid dan efektif untuk menjawab permasalahan latensi dan inefisiensi di lapak. Sistem ini tidak hanya unggul secara teknis dalam hal performa, tetapi juga fungsional dan dapat diterima dengan baik oleh pengguna akhir, serta berpotensi besar untuk modernisasi rantai pasok agribisnis.

Kata kunci: Smart Scale, Internet of Things, Edge Computing, Rantai Pasok Jeruk.

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan penulisan Laporan Tugas Akhir yang berjudul "Implementasi Algoritma XYZ untuk Optimasi Jaringan Komputer". Laporan ini disusun sebagai salah satu syarat untuk menyelesaikan program Sarjana di Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Bandung, 20 Mei 2026

Penulis

John Doe

DAFTAR ISI

LEMBAR PENGESAHAN	iii
LEMBAR PERNYATAAN	v
ABSTRAK	vii
KATA PENGANTAR	ix
DAFTAR ISI	xi
DAFTAR GAMBAR	xvi
DAFTAR TABEL	xix
DAFTAR ALGORITMA	xxi
DAFTAR KODE PROGRAM	xxiii
DAFTAR SINGKATAN	xxv
DAFTAR SIMBOL	xxvii
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Tujuan	4
I.4 Batasan Masalah	5
I.5 Metodologi	5
I.6 Sistematika Penulisan	6
II STUDI LITERATUR	7
II.1 Pengetahuan tentang Kasus yang Dikaji: Manajemen Informasi Gudang FMCG	7
II.1.1 Karakteristik Operasional Inventori Gudang FMCG Berbasis Pull System	7
II.1.2 Identifikasi Masalah Sinkronisasi Data Stok	9
II.1.3 Konsep Digitalisasi Smart Warehouse	11
II.2 Landasan Teori: Metode Perancangan dan Evaluasi Perangkat Lunak	12
II.2.1 Metode Analytical Hierarchy Process (AHP)	12
II.2.2 Pemodelan Traceability (Keterlacakan) dan Latensi Data	12
II.3 Landasan Teori: Arsitektur Warehouse Management Controller	13
II.3.1 Model Manajemen Inventori Berbasis Event (Event-Driven Inventory Management)	13
II.3.2 Arsitektur Microservices	13
II.3.3 Infrastruktur Komunikasi Berbasis Message Broker	14
II.3.3.1 Message Broker dan RabbitMQ	15
II.3.3.2 Protokol MQTT untuk Komunikasi Robot-Server	15
II.3.3.3 WebSocket untuk Pembaruan Status Real-Time	16
II.3.4 Manajemen Basis Data Relasional untuk Inventori Real-Time	17
II.4 Tinjauan Penelitian Terdahulu	17
II.4.1 Re-desain Proses Pemenuhan Pesanan (Framinan et al., 2025)	17
II.4.2 Integrasi Cloud dan IoT untuk Visibilitas Gudang (van Gest et al., 2021)	18

II.4.3	Implementasi Event-Driven Architecture pada Sistem Manajemen Inventori Real-Time	19
II.5	Landasan Teori: Subsistem Lokalisasi	20
II.5.1	AprilTag sebagai Fiducial Marker	20
II.5.2	Koreksi Distorsi Lensa Fisheye (Model Kannala–Brandt)	20
II.5.3	Perspective Homography dan Pemetaan Koordinat Grid	20
III	ANALISIS MASALAH	23
III.1	Analisis Kondisi Saat Ini	23
III.1.1	Profil Operasional Gudang Berdasarkan Hasil Wawancara	23
III.1.2	Proses Bisnis Pergudangan Eksisting	24
III.1.3	Kondisi Sistem Saat Ini (As-Is)	25
III.1.4	Kondisi Sistem Smart Warehouse (To-Be)	28
III.1.5	Analisis Gap	29
III.2	Analisis Kebutuhan	30
III.2.1	Karakteristik Produk	30
III.2.2	Kebutuhan Fungsional	31
III.2.3	Kebutuhan Nonfungsional	33
III.3	Analisis Pemilihan Solusi	37
III.3.1	Pemilihan Solusi Level WMC	37
III.3.1.1	Alternatif Solusi	37
III.3.1.2	Analisis Penentuan Solusi	40
III.3.2	Pemilihan Solusi Subsistem Lokalisasi	44
III.3.2.1	Alternatif Solusi Lokalisasi	44
III.3.2.2	Kriteria Evaluasi	46
III.3.2.3	Pairwise Comparison dan Perhitungan Bobot Kriteria	47
III.3.2.4	Penilaian Alternatif dan Penentuan Solusi Terpilih	48
III.3.2.5	Justifikasi Solusi Terpilih	51
IV	PERANCANGAN	53
IV.1	Konsep Sistem	53
IV.2	Gambaran Umum Sistem	54
IV.3	Diagram Arsitektur	54
IV.3.1	Arsitektur Level 0	54
IV.3.2	Subsistem <i>Warehouse Management Controller</i> (WMC)	55
IV.3.2.1	Arsitektur Level 2 — WMC	55
IV.3.2.2	Arsitektur Level 3 — Komponen WMC	56
IV.3.2.2.1	Database	56
IV.3.2.2.2	Goods Detail Service	58
IV.3.2.2.3	Fleet Order Service	61
IV.3.2.2.4	Authentication Service	61
IV.3.3	Subsistem <i>Fleet Controller</i>	62
IV.3.3.1	Arsitektur Level 2 — Fleet Controller	62
IV.3.3.2	Arsitektur Level 3 — Komponen Fleet Controller	63
IV.3.3.2.1	Localisation System	65
IV.3.3.3	Desain Detail Subsistem Lokalisasi (<i>Localisation System</i>)	66

IV.3.3.3.1	Perangkat Lokalisasi	66
IV.3.3.3.2	Desain Pipeline Deteksi	66
IV.4	Diagram <i>Use Case</i>	66
IV.4.1	UC-01: <i>Login System</i>	68
IV.4.2	UC-02: <i>Logout System</i>	70
IV.4.3	UC-03: <i>View Inventory List</i>	71
IV.4.4	UC-04: <i>View Goods Detail</i>	72
IV.4.5	UC-05: <i>View Order Status</i>	73
IV.4.6	UC-06: <i>View Robot & Operation Status</i>	74
IV.4.7	UC-07: <i>Operate Digital Twin</i>	75
IV.4.8	UC-08: <i>Create Order Request</i>	76
IV.4.9	UC-09: <i>View Item History</i>	78
IV.5	<i>Class Diagram</i>	78
IV.5.1	Kelas <i>Controller</i>	81
IV.5.2	Kelas <i>Service</i>	81
IV.5.3	Kelas <i>Repository</i>	81
IV.5.4	Kelas <i>Entity</i>	81
IV.6	<i>Sequence Diagram</i>	81
V	IMPLEMENTASI	97
V.1	Lingkungan Implementasi	97
V.2	Implementasi Sistem WMC	97
V.2.1	Teknologi Yang Digunakan	98
V.2.2	Implementasi Modul	101
V.2.3	Implementasi Endpoint	101
V.3	Implementasi Subsistem Lokalisasi	101
V.3.1	Lingkungan dan Perangkat Keras	102
V.3.2	Teknologi yang Digunakan	102
V.3.3	Implementasi Pipeline Deteksi	102
V.3.4	Implementasi Publikasi MQTT	103
VI	EVALUASI	109
VI.1	Metode Evaluasi	109
VI.1.0.0.1	a. Pengujian Usability — Kuesioner System Usability Scale (SUS)	109
VI.1.0.0.2	b. Pengujian Traceability — Audit Log dan Pengukuran Latensi	110
VI.1.0.0.3	c. Pengujian Productivity — Pengukuran Throughput Tugas	110
VI.1.0.0.4	d. Pengujian Task Fulfilment Accuracy — Verifikasi Konsistensi Data	111
VI.2	Hasil Evaluasi	112
VI.2.1	Hasil Pengujian Fungsionalitas	112
VI.2.2	Hasil Pengujian Usability	113
VI.2.3	Hasil Pengujian Traceability	113
VI.2.4	Hasil Pengujian Produktivitas	114
VI.2.5	Hasil Pengujian Task Fulfilment Accuracy	114

VI.2.6 Hasil Pengujian Lokalisasi	114
VII PENUTUP	121
VII.1 Kesimpulan	121
VII.2 Saran	122
Lampiran A <i>SOURCE CODE</i>	129
A.1 Perangkat Lunak untuk Akuisisi Data dari Sensor Ultrasonik . . .	129
A.2 Perangkat Lunak untuk Pengolahan Data Akuisisi dan Visualisasi Hasil Pengukuran Jarak	130
Lampiran B HASIL SURVEI LAPANGAN	131
B.1 Foto Survei Lokasi 1	131
B.2 Foto Survei Lokasi 2	131
B.3 Transkrip Wawancara dengan Narasumber	131

DAFTAR GAMBAR

II.1	Ilustrasi arsitektur <i>microservices</i> pada <i>Warehouse Management Controller</i>	14
II.2	Ilustrasi alur <i>Event-Driven Architecture</i> pada sistem manajemen inventori	19
III.1	Diagram BPMN proses bisnis <i>warehouse</i> kondisi saat ini (<i>As-Is</i>)	26
III.2	Diagram BPMN proses bisnis <i>Smart Warehouse</i> kondisi <i>To-Be</i>	28
IV.1	Konsep Sistem <i>Smart Warehouse</i>	54
IV.2	Arsitektur Sistem <i>Smart Warehouse</i> Level 0	55
IV.3	Arsitektur <i>Warehouse Management Controller</i> (Level 2)	56
IV.4	Arsitektur Komponen <i>Database WMC</i> (Level 3)	57
IV.5	ERD Sistem <i>Warehouse Management Controller</i>	59
IV.6	Arsitektur Komponen <i>Goods Detail Service WMC</i> (Level 3)	61
IV.7	Arsitektur Komponen <i>Fleet Order Service WMC</i> (Level 3)	61
IV.8	Arsitektur Komponen <i>Authentication Service WMC</i> (Level 3)	62
IV.9	Arsitektur <i>Fleet Controller</i> (Level 2)	64
IV.10	Arsitektur Komponen <i>Localisation System Fleet Controller</i> (Level 3)	65
IV.11	<i>Use Case Diagram</i> Sistem <i>Warehouse Management Controller</i>	67
IV.12	<i>Sequence Diagram</i> UC-01: <i>Login System</i>	68
IV.13	<i>Sequence Diagram</i> UC-02: <i>Logout System</i>	70
IV.14	<i>Sequence Diagram</i> UC-03: <i>View Inventory List</i>	71
IV.15	<i>Sequence Diagram</i> UC-04: <i>View Goods Detail</i>	72
IV.16	<i>Sequence Diagram</i> UC-05: <i>View Order Status</i>	73
IV.17	<i>Sequence Diagram</i> UC-06: <i>View Robot & Operation Status</i>	74
IV.18	<i>Sequence Diagram</i> UC-07: <i>Operate Digital Twin</i>	75
IV.19	<i>Sequence Diagram</i> UC-08: <i>Create Order Request</i>	76
IV.20	<i>Sequence Diagram</i> UC-09: <i>View Item History</i>	78
IV.21	<i>Class Diagram</i> Sistem <i>Warehouse Management Controller</i>	79
IV.22	<i>Sequence Diagram</i> UC-01: <i>Login System</i>	92
IV.23	<i>Sequence Diagram</i> UC-02: <i>Logout System</i>	92

IV.24 <i>Sequence Diagram UC-03: View Inventory List</i>	93
IV.25 <i>Sequence Diagram UC-04: View Goods Detail</i>	93
IV.26 <i>Sequence Diagram UC-05: View Order Status</i>	93
IV.27 <i>Sequence Diagram UC-06: View Robot & Operation Status</i>	94
IV.28 <i>Sequence Diagram UC-07: Operate Digital Twin</i>	94
IV.29 <i>Sequence Diagram UC-08: Create Order Request</i>	94
IV.30 <i>Sequence Diagram UC-09: View Item History</i>	95

DAFTAR TABEL

III.1	Aktivitas Primer pada <i>Smart Warehouse</i>	25
III.2	<i>Gap Analysis</i> Kondisi Eksisting vs <i>Smart Warehouse</i> (B100 Tabel 2.1.3.2)	29
III.3	Karakteristik Produk Sistem <i>Smart Warehouse</i>	30
III.4	Kebutuhan Fungsional WMC	31
III.5	Kebutuhan Fungsional Subsistem Lokalisasi	33
III.6	Kebutuhan Nonfungsional WMC	34
III.7	Kebutuhan Nonfungsional Subsistem Lokalisasi	36
III.8	Perbandingan Karakteristik Alternatif Solusi Manajemen Data Inventori	39
III.9	Bobot Kriteria AHP Pemilihan Solusi Manajemen Data Inventori . .	40
III.10	Penjelasan Penilaian Alternatif 1: Event-Driven Inventory Management	41
III.11	Penjelasan Penilaian Alternatif 2: State-Driven Inventory Management	42
III.12	Rekapitulasi Penilaian AHP Solusi Manajemen Data Inventori (B100 Tabel 2.2.3.8)	43
III.13	Perbandingan Karakteristik Alternatif Solusi Lokalisasi	45
III.14	Kriteria Evaluasi Pemilihan Solusi Lokalisasi	46
III.15	Nilai Kepentingan Kriteria	47
III.16	Nilai Bobot Kriteria	48
III.17	Penjelasan Penilaian Alternatif 1: QR Code + Line Following . . .	48
III.18	Penjelasan Penilaian Alternatif 2: AprilTag + Overhead Fisheye Camera	49
III.19	Rekapitulasi Penilaian AHP Solusi Lokalisasi	50
IV.1	Fungsi, masukan, dan keluaran sistem <i>Smart Warehouse</i> (Level 0) .	55
IV.2	Fungsi, masukan, dan keluaran <i>Warehouse Management Controller</i> (Level 2)	57
IV.3	Fungsi, masukan, dan keluaran <i>Database WMC</i>	57
IV.4	Penjelasan entitas basis data <i>Warehouse Management Controller</i> . .	60

IV.5 Fungsi, masukan, dan keluaran <i>Goods Detail Service WMC</i>	61
IV.6 Fungsi, masukan, dan keluaran <i>Fleet Order Service WMC</i>	62
IV.7 Fungsi, masukan, dan keluaran <i>Authentication Service WMC</i>	63
IV.8 Fungsi, masukan, dan keluaran <i>Fleet Controller (Level 2)</i>	63
IV.9 Fungsi, masukan, dan keluaran <i>Localisation System</i>	65
IV.10 <i>Pipeline</i> pemrosesan Subsistem Lokalisasi	68
IV.11 Deskripsi <i>Use Case UC-01: Login System</i>	69
IV.12 Skenario <i>Use Case UC-01: Login System</i>	69
IV.13 Deskripsi <i>Use Case UC-02: Logout System</i>	70
IV.14 Skenario <i>Use Case UC-02: Logout System</i>	70
IV.15 Deskripsi <i>Use Case UC-03: View Inventory List</i>	71
IV.16 Skenario <i>Use Case UC-03: View Inventory List</i>	71
IV.17 Deskripsi <i>Use Case UC-04: View Goods Detail</i>	72
IV.18 Skenario <i>Use Case UC-04: View Goods Detail</i>	72
IV.19 Deskripsi <i>Use Case UC-05: View Order Status</i>	73
IV.20 Skenario <i>Use Case UC-05: View Order Status</i>	73
IV.21 Deskripsi <i>Use Case UC-06: View Robot & Operation Status</i>	74
IV.22 Skenario <i>Use Case UC-06: View Robot & Operation Status</i>	74
IV.23 Deskripsi <i>Use Case UC-07: Operate Digital Twin</i>	75
IV.24 Skenario <i>Use Case UC-07: Operate Digital Twin</i>	75
IV.25 Deskripsi <i>Use Case UC-08: Create Order Request</i>	76
IV.26 Skenario <i>Use Case UC-08: Create Order Request</i>	77
IV.27 Deskripsi <i>Use Case UC-09: View Item History</i>	77
IV.28 Skenario <i>Use Case UC-09: View Item History</i>	78
IV.29 Daftar kelas pada arsitektur <i>Warehouse Management Controller</i> . . .	80
IV.30 Detail Kelas – <i>AuthController</i>	81
IV.31 Detail Kelas – <i>ItemController</i>	81
IV.32 Detail Kelas – <i>OrderController</i>	82
IV.33 Detail Kelas – <i>RobotController</i>	82
IV.34 Detail Kelas – <i>DigitalTwinController</i>	82
IV.35 Detail Kelas – <i>AuthService</i>	83
IV.36 Detail Kelas – <i>ItemService</i>	83
IV.37 Detail Kelas – <i>OrderService</i>	84
IV.38 Detail Kelas – <i>RobotService</i>	85
IV.39 Detail Kelas – <i>TaskService</i>	85
IV.40 Detail Kelas – <i>DigitalTwinService</i>	85
IV.41 Detail Kelas – <i>LocationService</i>	86

IV.42 Detail Kelas – NotificationService	86
IV.43 Detail Kelas – UserRepository	86
IV.44 Detail Kelas – SessionRepository	86
IV.45 Detail Kelas – ItemRepository	87
IV.46 Detail Kelas – OrderRepository	87
IV.47 Detail Kelas – OrderItemRepository	87
IV.48 Detail Kelas – RobotRepository	87
IV.49 Detail Kelas – TaskRepository	88
IV.50 Detail Kelas – AprilTagMappingRepository	88
IV.51 Detail Kelas – User	88
IV.52 Detail Kelas – Session	88
IV.53 Detail Kelas – Item	89
IV.54 Detail Kelas – Order	89
IV.55 Detail Kelas – OrderItem	89
IV.56 Detail Kelas – Robot	89
IV.57 Detail Kelas – Task	90
IV.58 Detail Kelas – AprilTagMapping	90
V.1 Spesifikasi <i>hardware</i>	97
V.2 Spesifikasi <i>software</i>	98
V.3 Teknologi yang digunakan dalam pengembangan WMC	98
V.4 Modul <i>auth-service</i>	101
V.5 Modul <i>goods-service</i>	104
V.6 Modul <i>database</i>	105
V.7 Daftar <i>endpoint auth-service</i>	105
V.8 Daftar <i>endpoint goods-service</i>	106
V.9 Teknologi yang digunakan dalam implementasi Subsistem Lokalisasi	107
VI.1 Rangkuman kriteria dan metode evaluasi WMC	112
VI.2 Skenario dan hasil pengujian fungsionalitas WMC	116
VI.3 Hasil pengujian usability (SUS)	117
VI.4 Rekapitulasi skor SUS	117
VI.5 Hasil pengukuran latensi respons API	117
VI.6 Hasil audit log skenario Inbound–Storage–Outbound	118
VI.7 Hasil pengujian produktivitas sistem WMC	118
VI.8 Hasil pengujian Task Fulfilment Accuracy	118
VI.9 Hasil verifikasi posisi Subsistem Lokalisasi	119
VI.10 Hasil verifikasi orientasi Subsistem Lokalisasi	119

DAFTAR ALGORITMA

DAFTAR *LISTING*

A.1	Binary search code	129
-----	------------------------------	-----

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
AI	<i>Artificial Intelligence</i>	1
CNN	<i>Convolutional Neural Network</i>	2
GPU	<i>Graphics Processing Unit</i>	14

DAFTAR SIMBOL

Notasi	Deskripsi	Pemakaian pertama kali
α	Koefisien bobot untuk <i>loss function</i> , tingkat signifikansi statistik	75
β	Koefisien bobot untuk <i>Binary Cross-Entropy loss</i>	75
λ	Koefisien bobot untuk CTC <i>loss</i> , parameter regularisasi	75
θ	Parameter model <i>neural network</i>	75
σ	Deviasi standar	75
μ	Rata-rata (<i>mean</i>)	75
ϵ	<i>Token blank</i> /kosong dalam CTC, konstanta kecil untuk stabilitas numerik	75
Δ	Delta (selisih/perubahan nilai)	75
π	<i>Alignment path</i> dalam CTC	75
π_t	<i>Token</i> pada <i>timestep</i> t dalam <i>alignment path</i>	76
∇ atau ∂	Operator gradien atau turunan parsial	75
$\sum$	Operator penjumlahan	75
$\prod$	Operator perkalian	75
$\int$	Operator integral	75
argmax	Argumen yang memaksimalkan fungsi	76
$\sim$	Terdistribusi menurut	75
$\rightarrow$	Menuju atau konvergen ke	75
$\mathcal{L}$	Fungsi <i>loss</i> (kerugian)	75
$\mathcal{L}_{total}$	Total <i>loss function</i>	75
$\mathcal{L}_{adversarial}$ atau $\mathcal{L}_{adv}$	<i>Adversarial loss</i>	75
$\mathcal{L}_{reconstruction}$	<i>Reconstruction loss</i>	75
$\mathcal{L}_{L1}$	L1 <i>loss</i> (<i>Mean Absolute Error</i>)	75
$\mathcal{L}_{L2}$	L2 <i>loss</i> (<i>Mean Squared Error</i>)	75
$\mathcal{L}_{CTC}$	CTC (<i>Connectionist Temporal Classification</i>) <i>loss</i>	75
$\mathcal{L}_{perceptual}$	<i>Perceptual loss</i>	75

$\mathcal{L}_{pixel}$	<i>Pixel-wise loss</i>	75
$\mathcal{L}_{BCE}$	<i>Binary Cross-Entropy loss</i>	75
$\mathcal{L}_{GAN}$	<i>GAN loss</i>	75
$\mathcal{L}_{cycle}$	<i>Cycle consistency loss</i>	75
G	<i>Generator dalam GAN</i>	75
D	<i>Discriminator dalam GAN</i>	75
$G(z)$	<i>Output dari generator dengan input z</i>	75
$D(x)$	<i>Output dari discriminator dengan input x</i>	75
$G_{A \rightarrow B}$	<i>Generator dari domain A ke B</i>	75
$G_{B \rightarrow A}$	<i>Generator dari domain B ke A</i>	75
$V(D, G)$	<i>Fungsi nilai (value function) dalam GAN</i>	75
x	<i>Sampel data asli, input citra</i>	75
y	<i>Label atau ground truth, kondisi dalam conditional GAN</i>	75
z	<i>Noise vector atau representasi laten</i>	
$\mathbb{E}$	<i>Ekspektasi atau nilai harapan</i>	75
p_{data}	<i>Distribusi data asli</i>	75
p_z	<i>Distribusi prior (noise)</i>	75
$p(y x)$	<i>Probabilitas bersyarat output y given input x</i>	75
$p_t(\pi_t x)$	<i>Probabilitas token pada timestep t</i>	
$\mathcal{B}$	<i>Fungsi penciutan (collapse function) dalam CTC</i>	75
$\mathcal{B}^{-1}$	<i>Fungsi invers penciutan dalam CTC</i>	75
T	<i>Jumlah timestep atau panjang sequence</i>	
$ x - G(y) _1$	<i>L1 distance antara ground truth dan generated image</i>	75
$ x - G(y) _2$	<i>L2 distance antara ground truth dan generated image</i>	75
$N \times N$	<i>Ukuran patch dalam PatchGAN</i>	
d	<i>Cohen's d (effect size)</i>	75
n	<i>Jumlah sampel</i>	75
p	<i>p-value (nilai probabilitas statistik)</i>	75

r	Koefisien korelasi	75
R^2	Koefisien determinasi	
$O(n)$	Notasi Big-O untuk kompleksitas linear	75
$O(n^2)$	Notasi Big-O untuk kompleksitas kuadratik	75

BAB I

PENDAHULUAN

I.1 Latar Belakang

Industri *Fast Moving Consumer Goods* (FMCG) merupakan sektor dengan pertumbuhan paling pesat secara global. Pasar FMCG dunia bernilai USD 14,1 triliun pada tahun 2024 dengan CAGR 5,4% hingga tahun 2034 (Research and Markets 2025), sementara kawasan Asia-Pasifik menyumbang lebih dari 40% total penjualan global. Di Indonesia, nilai pasar FMCG melampaui USD 100 miliar dengan CAGR 7,6% (2021–2025), khususnya pada segmen kecantikan dan perawatan diri yang mencatat penjualan senilai Rp36 triliun sepanjang Januari hingga Juli 2025 (KADIN Indonesia 2025). Dinamika ini mendorong kebutuhan sistem pergudangan yang semakin efisien, seiring pasar *Smart Warehouse* global yang tumbuh dari USD 28 miliar (2024) menuju USD 90,21 miliar (2033) pada CAGR 14,2%, dengan Asia-Pasifik memimpin laju adopsi di CAGR 18,4% (Precedence Research 2025). Namun, sistem pergudangan konvensional pada industri FMCG masih mengandung inefisiensi signifikan: 62% keterlambatan pengiriman bersumber dari aktivitas *order picking* yang tidak terkoordinasi (Framinan, Ruiz-Usano, dan Leisten 2025), dengan kekurangan SDM dalam proses *picking* mencatat nilai keparahan (*mean severity*) 3,79 dari skala 5 (Habazin, Glasnovic, dan Bajor 2017). Kondisi ini mempertegas urgensi transformasi digital dalam manajemen pergudangan modern.

PT Paragon Technology and Innovation (Paragon Corp) merupakan salah satu perusahaan FMCG terkemuka di Indonesia yang bergerak di bidang manufaktur dan distribusi produk kecantikan serta perawatan diri. Perusahaan ini mengoperasikan lebih dari 40 *Distribution Center* (DC) yang tersebar di Indonesia dan Malaysia (Universitas Diponegoro 2019), dengan tiga jenis fasilitas gudang utama, yaitu gudang bahan baku, gudang *packaging*, dan gudang barang jadi (*Finished Goods*). Gudang barang jadi menjadi titik kritis dalam rantai distribusi karena menjadi tahap akhir sebelum produk dikirimkan kepada mitra ritel dan konsumen. Guna merespons kebutuhan operasional yang terus meningkat sekaligus selaras dengan agenda transformasi digital dalam konsep *Society 5.0*, Paragon Corp menginisiasi proyek

perancangan sistem *Smart Warehouse* yang mencakup integrasi perangkat keras robotika dan layanan perangkat lunak secara menyeluruh pada gudang barang jadinya (Tim TA252601001 2025a).

Berdasarkan analisis proses bisnis pada gudang barang jadi Paragon Corp, ditemukan permasalahan kritis pada tahap *stocking* dan *picking* yang masih bergantung sepenuhnya pada operator manusia. Kondisi ini menimbulkan *bottleneck* operasional dengan tingkat *overcapacity* di atas 80% disertai *waiting time* yang tinggi. Metode *picking* manual mencatat tingkat kesalahan 1–3%, angka yang 8–10 kali lebih tinggi dibanding lingkungan terotomasi (NetSuite 2024), dengan biaya per insiden berkisar USD 85–220 (SCM Champs 2024). Dampaknya meluas: 65% pelanggan berhenti berbelanja setelah 2–3 keterlambatan pengiriman (Conexiom 2024), sementara distorsi inventori global akibat ketidakakuratan data diestimasi merugikan USD 1,77 triliun pada tahun 2023 (IHL Group 2023). Geest, Tekinerdogan, dan Catal (2021) menegaskan bahwa transisi ke *Smart Warehouse*—melalui pilar *Automation and Robotics*, *IoT-based Visibility*, *AI-driven Optimization*, dan *Cyber-Physical Integration*—mampu meningkatkan efisiensi operasional gudang hingga 35% dibanding sistem konvensional.

Dalam operasionalnya saat ini, Paragon Corp memanfaatkan dua platform teknologi utama untuk pencatatan dan pelacakan barang di gudang. Platform pertama adalah sistem *Enterprise Resource Planning* (ERP) Odoo yang berfungsi sebagai pusat pengelolaan data transaksi bisnis dan inventori secara korporat. Platform kedua adalah *Warehouse Management System* (WMS) yang digunakan untuk mendukung manajemen aktivitas operasional di tingkat gudang secara lebih spesifik. Selain kedua platform tersebut, Paragon Corp juga telah mengadopsi robot *Automated Guided Vehicle* (AGV) untuk sebagian proses mobilisasi barang jadi dari area produksi menuju *Loading Dock*. Sistem kerja keseluruhan menerapkan pendekatan *Pull System*, yakni pergerakan barang berdasarkan permintaan aktual guna meminimalkan penumpukan stok dan mengoptimalkan responsivitas rantai pasok.

Meskipun kedua platform tersebut telah berperan dalam mendukung operasional, integrasi antara ERP Odoo dan WMS masih bersifat semi-manual, sehingga setiap proses sinkronisasi data memerlukan intervensi langsung dari operator gudang. Ketidadaan integrasi *real-time* antara kedua sistem ini mengakibatkan seringnya terjadi *mismatch* data stok antara kondisi fisik di lapangan dengan catatan digital pada sistem pusat. Hal ini diperparah oleh absennya mekanisme pencatatan barang yang bersifat atomik dan otomatis, sehingga area penempatan barang sementara kerap tidak

tercatat secara sistematis dalam WMS. Akibatnya, muncul permasalahan *mistracking* barang yang membuat staf gudang tidak dapat mengetahui posisi aktual suatu barang secara akurat. Selain itu, implementasi robot AGV yang masih terbatas pada area produksi menyebabkan proses *inbound* dan perpindahan barang di area gudang barang jadi belum dapat dieksekusi secara otomatis, sehingga ketergantungan pada tenaga manual di area kritis tersebut masih sangat tinggi.

Guna memperoleh gambaran kondisi operasional yang akurat dan kontekstual, tim TA252601001 melaksanakan wawancara daring pada tanggal 16 Oktober 2025 bersama Bapak Falih Muhtadi selaku *Staff Logistics and Business Integration Systems* Paragon Corp. Dari hasil wawancara tersebut, dikonfirmasi bahwa permasalahan utama pada gudang barang jadi meliputi tiga hal: pertama, data stok tidak bersifat *real-time* karena WMS belum terhubung langsung dengan sistem pusat; kedua, proses *inbound* barang yang lambat dan perhitungan yang tidak presisi akibat ketergantungan pada proses manual; serta ketiga, proses *picking* yang masih bergantung pada operator dan rawan *human error*, khususnya dalam konteks jumlah barang. Bapak Falih juga mengonfirmasi adanya permasalahan *mistracking* barang akibat area penempatan sementara yang tidak tercatat di sistem, serta potensi besar untuk mengotomasi proses *inbound*, khususnya pemindahan barang dari truk ke *pallet changer*. Berdasarkan seluruh temuan tersebut, tugas akhir ini mengusulkan pengembangan *Warehouse Management Controller* (WMC) sebagai layanan *backend* berbasis arsitektur *microservices* yang mencakup *Authentication Service*, *Goods Detail Service*, *Fleet Order Service*, dan antarmuka pengguna (*User Interface*), guna menjamin konsistensi data stok secara otomatis serta mengoordinasikan armada *mobile robot* dalam satu platform terpusat di PT Paragon Technology and Innovation.

I.2 Rumusan Masalah

Saat ini, operasional pergudangan barang jadi (*Finished Goods*) di PT Paragon Technology and Innovation masih bersifat semi-manual dan sangat bergantung pada operator manusia dalam pelaksanaan proses *stocking* serta *picking*. Pencatatan identitas dan aktivitas barang dilakukan menggunakan sistem ERP Odoo dan *Warehouse Management System* (WMS) yang belum terintegrasi secara *real-time*, sehingga sinkronisasi data masih memerlukan intervensi manual dari petugas gudang. Ketergantungan pada proses manual dan belum adanya integrasi sistem yang matang menyebabkan inefisiensi berupa terjadinya *mismatch* data antara kondisi fisik di lapangan dengan catatan digital pada sistem pusat, serta permasalahan pelacakan barang (*mistracking*) yang disebabkan oleh area penempatan sementara yang tidak tercatat secara sistematis.

Jika masalah tersebut tidak segera diatasi, maka titik kritis (*bottleneck*) operasional pada tahap *stocking* dan *picking* akan terus terjadi, yang memicu kondisi *overcapacity* di atas 80% serta *waiting time* yang panjang. Hal ini berdampak langsung pada menurunnya kecepatan rantai pasok secara keseluruhan, meningkatnya *lead time* distribusi, serta tingginya risiko *human error* terutama dalam konteks ketidaktepatan jumlah barang. Oleh karena itu, diperlukan transformasi menuju sistem *Smart Warehouse* yang mampu melakukan integrasi data secara *real-time*, melacak posisi serta pergerakan barang secara otomatis, dan mengeksekusi perpindahan barang melalui teknologi otomasi guna meningkatkan akurasi serta kecepatan distribusi.

Berdasarkan permasalahan tersebut, maka rumusan masalah dalam tugas akhir ini adalah:

1. Bagaimana menyinkronkan dan menjaga konsistensi data stok gudang secara otomatis antara kondisi fisik dan catatan digital, serta menyajikan informasi inventori tersebut secara akurat dan *real-time* kepada staf gudang, sehingga ketidaksesuaian (*mismatch*) data, *mistracking* barang, dan ketergantungan pada proses manual dapat diatasi?
2. Bagaimana bentuk rancangan dan prototipe mekanisme penentuan posisi robot dan barang di area gudang yang dapat menunjang pelaksanaan tugas pemindahan barang secara akurat pada prototipe *Smart Warehouse*?

I.3 Tujuan

Tujuan dari pelaksanaan tugas akhir ini adalah:

1. Mengembangkan sistem informasi manajemen gudang berbasis *microservices* yang menyinkronkan dan menjaga konsistensi data stok secara otomatis serta menyajikan informasi inventori secara akurat dan *real-time* bagi staf gudang.
2. Merancang dan membangun prototipe mekanisme penentuan posisi robot dan barang di area gudang untuk menunjang pelaksanaan tugas pemindahan barang secara akurat pada prototipe *Smart Warehouse*.

Kriteria keberhasilan tugas akhir ini meliputi:

1. Sistem dapat melakukan sinkronisasi data inventori secara otomatis untuk meniadakan selisih (*mismatch*) antara kondisi lapangan dan sistem pusat.
2. Sistem manajemen informasi memiliki antarmuka yang mudah digunakan oleh staf gudang untuk mengawasi status armada robot dan aktivitas inventori secara *real-time*.

I.4 Batasan Masalah

Untuk menjaga ruang lingkup pembahasan dan memastikan tercapainya tujuan penelitian dalam batasan waktu serta sumber daya yang tersedia, maka ditetapkan batasan-batasan proyek sebagai berikut:

1. **Dimensi Fisik Lingkungan:** Pengujian sistem tidak dilakukan pada lokasi gudang asli, melainkan disimulasikan pada *layout* berskala $2,4 \times 1,2$ meter yang dipetakan ke dalam bentuk matriks berukuran 12×6 grid.
2. **Asumsi Tata Letak Statis:** Konfigurasi dan posisi rak di dalam area operasional simulasi dianggap tetap (statis) dan tidak mengalami perubahan selama pengujian berlangsung.
3. **Kapasitas Armada Robot:** Sistem dibatasi untuk mengelola dua unit robot mikro (dimensi $\pm 15 \times 15$ cm) dengan kemampuan manuver antar-sel matriks serta kemampuan rotasi 360 derajat.
4. **Lingkup Aktivitas Operasional:** Fungsionalitas operasional dibatasi pada proses *picking* (pengambilan) dan *storing* (penyimpanan) barang antara rak tujuan dan area *docking* menggunakan mekanisme penugasan antrean (*pull system*).
5. **Fokus Pengembangan Individu:** Penulis secara spesifik bertanggung jawab atas pengembangan layanan perangkat lunak *backend* yang mencakup logika sinkronisasi data, manajemen antrean tugas, serta sistem autentikasi untuk kategori barang jadi dalam skala *Proof of Concept* (PoC).

I.5 Metodologi

Metodologi pengerjaan tugas akhir ini mengadopsi pendekatan *Design Thinking* yang dilakukan dalam dua siklus iterasi. *Design thinking* merupakan metode inovasi yang berpusat pada manusia (*human-centered approach*) melalui kolaborasi antara pengembang dan pengguna guna menghasilkan solusi yang relevan dengan cara pikir dan kebutuhan mereka.

Tahapan utama dalam metodologi ini mencakup lima fase sebagai berikut:

1. **Empathize:** Tahap awal untuk mendalami pengalaman dan kendala pengguna melalui observasi, wawancara, atau survei guna membangun pemahaman mendalam tentang konteks desain sistem.
2. **Define:** Proses mengolah data temuan dari tahap *empathize* untuk merumuskan masalah utama dalam bentuk *problem statement* yang spesifik dan terukur sebagai panduan pengembangan solusi.
3. **Ideate:** Fase eksplorasi solusi secara kreatif melalui berbagai teknik seperti *brainstorming*, sketsa, atau *mind mapping* untuk mengidentifikasi berbagai

potensi pemecahan masalah.

4. **Prototype:** Mewujudkan ide terpilih ke dalam bentuk nyata, seperti model fisik atau simulasi, yang bertujuan untuk menguji konsep dan mendapatkan pemahaman lebih lanjut tanpa harus membuat produk akhir terlebih dahulu.
5. **Test:** Tahap pengujian purwarupa untuk mendapatkan umpan balik langsung dari pengguna, yang berfungsi sebagai dasar untuk menyempurnakan solusi, rumusan masalah, maupun pemahaman terhadap kebutuhan pengguna.

I.6 Sistematika Penulisan

Berikut adalah gambaran umum isi dari setiap bab yang ada dalam laporan tugas akhir ini:

1. Bab I Pendahuluan: Berisi latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi, dan sistematika penulisan laporan tugas akhir.
2. Bab II Studi Literatur: Berisi kajian pustaka yang relevan dengan topik tugas akhir, termasuk teori-teori, konsep-konsep, dan hasil-hasil penelitian sebelumnya yang mendukung pelaksanaan tugas akhir.
3. Bab III Analisis Masalah: Berisi penjelasan tentang analisis kondisi saat ini, analisis kebutuhan, dan analisis pemilihan solusi WMC.
4. Bab IV Analisis Proses Bisnis & Perancangan: Berisi penjelasan tentang perbandingan sistem *as-is* dan *to-be*, arsitektur sistem, desain basis data, kebutuhan fungsional, dan skenario *use case*.
5. Bab V Implementasi: Berisi penjelasan tentang proses implementasi solusi yang diusulkan, termasuk lingkungan, teknologi, modul, dan *endpoint* yang dikembangkan.
6. Bab VI Evaluasi: Berisi analisis dan evaluasi terhadap solusi yang diimplementasikan, mencakup metode pengujian dan hasil pengujian fungsionalitas, *usability*, *traceability*, produktivitas, dan akurasi.
7. Bab VII Penutup: Berisi kesimpulan dari hasil pelaksanaan tugas akhir dan saran untuk pengembangan lebih lanjut.
8. Daftar Pustaka: Berisi daftar referensi yang digunakan dalam penyusunan laporan tugas akhir.

BAB II

STUDI LITERATUR

II.1 Pengetahuan tentang Kasus yang Dikaji: Manajemen Informasi Gudang FMCG

II.1.1 Karakteristik Operasional Inventori Gudang FMCG Berbasis Pull System

Pada industri *Fast Moving Consumer Goods* (FMCG), gudang penyimpanan berperan sebagai pusat kendali aliran barang dalam rantai pasok yang menuntut kecepatan, akurasi, dan integrasi sistem yang tinggi. Operasional gudang yang efektif harus mampu menangani volume barang yang besar, variasi *Stock Keeping Unit* (SKU) yang luas, serta kecepatan distribusi yang tinggi guna memastikan ketersediaan produk di pasar. Dalam konteks ini, gudang bukan lagi sekadar tempat penyimpanan pasif, melainkan komponen strategis yang secara langsung menentukan kelancaran seluruh rantai distribusi. Framinan, Ruiz-Usano, dan Leisten (2025) menegaskan bahwa sistem pergudangan konvensional pada industri FMCG mengandung inefisiensi signifikan akibat keterbatasan otomatisasi dan sistem informasi yang belum terintegrasi sepenuhnya, yang pada akhirnya berdampak pada keterlambatan pengiriman dan ketidakakuratan data inventori.

Secara operasional, terdapat empat sumber daya utama yang menggerakkan sebuah gudang, yaitu *storage system*, *lift trucks*, *personnel*, dan *Warehouse Management System* (WMS). Setiap komponen tersebut harus memenuhi *Flow Impact Test* (IFIT), di mana kapasitas setiap komponen berpengaruh langsung terhadap kapasitas atau laju keseluruhan operasional gudang (Brazhkin 2018). Keempat sumber daya ini bersifat saling bergantung dan harus beroperasi secara terkoordinasi untuk menjamin kelancaran alur barang dari tahap penerimaan hingga pengiriman akhir. Kegagalan pada salah satu komponen sumber daya akan menimbulkan efek domino yang mengganggu keseluruhan rantai operasional di dalam gudang.

Proses operasional inventori di dalam gudang secara umum mencakup sembilan tahapan utama, yaitu penerimaan (*receiving*), penempatan (*put-away*), pengisian

ulang internal (*internal replenishment*), pengambilan pesanan (*order picking*), pengumpulan dan penyortiran (*accumulating and sorting*), pengepakan (*packing*), lintas-dermaga (*cross docking*), pengiriman barang (*dispatch*), dan pengapalan (*shipping*) (Habazin, Glasnovic, dan Bajor 2017). Di antara seluruh tahapan tersebut, proses *order picking* menjadi titik paling kritis karena memiliki tingkat keterlibatan manual tertinggi dan paling rentan terhadap kesalahan serta keterlambatan. Studi empiris pada industri manufaktur di Kroasia menemukan bahwa kekurangan sumber daya manusia merupakan kendala dengan nilai keparahan rata-rata (*mean severity*) kedua tertinggi dalam proses *picking* di gudang dengan nilai 3,79 dari skala 5 (Habazin, Glasnovic, dan Bajor 2017), yang mengindikasikan tingginya ketergantungan terhadap faktor manusia dalam aktivitas inti pergudangan konvensional.

Geest, Tekinerdogan, dan Catal (2021) menegaskan bahwa kompleksitas rantai pasok modern pada industri FMCG telah mendorong evolusi konsep gudang konvensional menuju *Smart Warehouse*, yang dicirikan oleh empat pilar utama, yaitu *Automation and Robotics*, *IoT-based Visibility*, *AI-driven Optimization*, dan *Cyber-Physical Integration*. Dalam konteks operasional berbasis *pull system*, pilar *Cyber-Physical Integration* menjadi sangat krusial karena memungkinkan sinkronisasi antara permintaan digital dan eksekusi fisik berlangsung secara *real-time*. Dengan demikian, setiap aktivitas pergerakan barang mulai dari penerimaan, penyimpanan, hingga pengiriman dapat tercatat dan direspons secara otomatis oleh sistem tanpa keterlambatan akibat keterbatasan proses manual.

Profil Sistem Operasional Gudang Paragon Corp Berdasarkan Hasil Wawancara

Guna memperoleh gambaran kondisi operasional yang akurat dan kontekstual, kelompok TA252601001 melaksanakan wawancara daring dengan Bapak Falih Muhtadi selaku *Staff Logistics and Business Integration Systems* PT Paragon Technology and Innovation (Paragon Corp) pada tanggal 16 Oktober 2025. Dari wawancara tersebut diketahui bahwa Paragon Corp memiliki tiga jenis gudang, yaitu gudang bahan baku, gudang kemasan (*packaging warehouse*), serta gudang barang jadi (*Finished Goods*) yang terletak pada area terpisah. Fokus pengembangan tugas akhir ini adalah pada gudang barang jadi, yang secara langsung bersinggungan dengan proses distribusi produk ke konsumen akhir.

Berdasarkan hasil wawancara, sistem kerja operasional inventori Paragon Corp menerapkan pendekatan *Pull System*, yaitu suatu mekanisme di mana pergerakan dan pemrosesan barang didasarkan pada permintaan aktual yang masuk ke sistem, buk-

an pada prakiraan (*forecasting*) internal. Alur kerja utama gudang mencakup empat tahapan, yaitu *Incoming*, *Stocking*, *Picking & Checking*, serta *Dispatching* menuju *Distribution Center*. Dalam skema *pull system*, perintah *stocking* dan *picking* baru dieksekusi ketika terdapat permintaan nyata, sehingga meminimalkan akumulasi stok berlebih (*overstock*) namun di sisi lain menuntut kecepatan respons sistem yang sangat tinggi agar tidak menimbulkan *bottleneck* pada tahap distribusi.

Dari hasil wawancara juga terungkap bahwa teknologi yang saat ini digunakan di gudang Paragon mencakup sistem ERP Odoo dan *Warehouse Management System* (WMS) untuk pencatatan dan pelacakan barang. Namun, kedua sistem tersebut masih bersifat semi-manual sehingga membutuhkan input operator secara berkala dan belum memiliki integrasi *real-time* antar platform. Kondisi ini menyebabkan *mismatch* data yang sering terjadi antara kondisi fisik di lapangan dengan catatan digital pada sistem pusat. Paragon Corp sendiri saat ini tengah dalam proses transformasi menuju sistem SAP untuk meningkatkan kualitas integrasi antar sistem. Kompleksitas operasional semakin bertambah mengingat Paragon Corp mengelola lebih dari 40 *Distribution Centers* (DC) di Indonesia dan Malaysia (Universitas Diponegoro 2019), dengan volume produk dan variasi SKU kosmetik yang sangat beragam, sehingga ketersediaan sistem inventori yang akurat dan terintegrasi menjadi kebutuhan operasional yang mendesak.

II.1.2 Identifikasi Masalah Sinkronisasi Data Stok

Permasalahan inti yang menjadi landasan pengembangan *Warehouse Management Controller* (WMC) dalam tugas akhir ini adalah ketidaksinkronan data stok antara kondisi fisik di lapangan dengan catatan digital pada sistem pusat, yang dikodekan sebagai Masalah Aktual pertama (MA01) dalam dokumen B100. Dalam kondisi eksisting, *Warehouse Management System* (WMS) gudang barang jadi Paragon Corp beroperasi secara independen tanpa koneksi langsung ke sistem pusat perusahaan, sehingga pembaruan data stok masih bergantung pada intervensi manual operator. Akibatnya, terdapat jeda waktu yang tidak dapat diprediksi antara perubahan kondisi fisik barang di lapangan dengan refleksinya pada sistem informasi, sehingga data yang tersedia di sistem pusat tidak selalu mencerminkan kondisi aktual yang terjadi di gudang.

Ketidaksinkronan ini menghasilkan dua manifestasi utama yang secara langsung mengganggu efisiensi operasional. Pertama, terjadi *mismatch* data stok, yaitu selisih antara jumlah barang yang tercatat di sistem dengan jumlah fisik yang tersedia di rak gudang. Kondisi ini memicu *human error* pada proses *picking*, terutama dalam

konteks ketidaktepatan jumlah barang yang diambil, karena operator berpatokan pada data yang tidak akurat. Kedua, terjadi *mistracking* barang, di mana posisi fisik barang tidak terdokumentasikan secara akurat karena terdapat area penempatan sementara yang tidak tercatat di dalam sistem manajemen gudang yang ada. Kedua masalah ini secara kumulatif menyebabkan *bottleneck* pada tahap *stocking* dan *picking*, yang berujung pada kondisi *overcapacity* serta *waiting time* yang panjang di area gudang barang jadi.

Temuan ini sejalan dengan penelitian Framinan, Ruiz-Usano, dan Leisten (2025) yang mengkaji ulang proses pemenuhan pesanan pada perusahaan FMCG menggunakan metode *Business Process Redesign* (BPR). Penelitian tersebut menemukan bahwa 62% keterlambatan pengiriman bersumber dari aktivitas *order picking* dan *staging* yang tidak terkoordinasi secara digital, di mana sistem pencatatan yang tidak *real-time* menjadi akar utama inefisiensi. Relevansi temuan ini terhadap kondisi Paragon Corp sangat tinggi, mengingat kedua sistem beroperasi dengan pola semi-manual yang serupa dalam konteks industri FMCG dengan volume tinggi dan variasi SKU yang luas.

Geest, Tekinerdogan, dan Catal (2021) mengidentifikasi interoperabilitas sistem sebagai salah satu tantangan teknologi paling kritis dalam implementasi *Smart Warehouse*. Tantangan ini mencakup ketidakmampuan platform yang berbeda untuk saling bertukar data secara langsung, latensi sistem berbasis *cloud*, dan potensi kehilangan data akibat ketidaksesuaian protokol komunikasi antar sistem. Dalam konteks gudang Paragon, kondisi tersebut tercermin dari ketidakterhubungan antara WMS lokal dengan sistem ERP Odoo, yang masing-masing beroperasi sebagai silo informasi yang berdiri sendiri tanpa mekanisme sinkronisasi otomatis.

Berdasarkan analisis *gap* yang dilakukan pada dokumen B100, kondisi eksisting (MA01) ditargetkan untuk ditransformasi menjadi sistem yang sepenuhnya terintegrasi dan tersinkronisasi secara *real-time* antara gudang barang jadi dan sistem pusat. Kebutuhan ini diformulasikan dalam dokumen B200 sebagai karakteristik fungsi dasar pertama sistem, yaitu CH01: sistem dapat menyinkronkan data stok secara otomatis antara kondisi fisik dan catatan digital untuk menjaga konsistensi informasi inventori. Dari sisi parameter kinerja, spesifikasi *traceability* menetapkan bahwa sinkronisasi waktu antara robot dan server serta antara server dan WMS harus berada di bawah 200 milidetik, dengan pembaruan status robot ke server dilakukan dengan latensi tidak lebih dari 1 detik pada kondisi jaringan yang stabil (Tim TA252601001 2025b).

Oleh karena itu, perancangan *Warehouse Management Controller* (WMC) sebagai layanan *backend* dalam tugas akhir ini secara langsung merespons *gap* sinkronisasi ini. WMC dirancang untuk menjadi titik kendali tunggal yang mencatat setiap aktivitas pergerakan barang sebagai sebuah *event* digital, memastikan bahwa setiap tindakan fisik yang dilakukan oleh robot di lapangan secara otomatis memperbarui catatan stok pada basis data terpusat dan dapat diakses oleh sistem pusat tanpa memerlukan intervensi operator. Dengan demikian, pendekatan ini diharapkan dapat menutup *gap* MA01 secara fundamental dan menjamin konsistensi data inventori secara menyeluruh di seluruh lapisan sistem.

II.1.3 Konsep Digitalisasi Smart Warehouse

Konsep digitalisasi *Smart Warehouse* merupakan transformasi menyeluruh dari sistem pergudangan konvensional menuju sistem yang cerdas dan terhubung secara digital, dengan menempatkan data sebagai aset operasional utama. Geest, Tekinerdogan, dan Catal (2021) mendefinisikan *Smart Warehouse* melalui empat pilar teknologi penggerak (*technology enabler*), yaitu *Automation and Robotics* untuk otomatisasi proses fisik, *IoT-based Visibility* untuk deteksi dan pelacakan barang secara *real-time*, *AI-driven Optimization* untuk penjadwalan dan pengambilan keputusan otomatis, serta *Cyber-Physical Integration* sebagai sinergi antara operasi fisik dan visibilitas data berbasis *cloud*. Dalam perspektif sistem cerdas, Romero dkk. (2020) menegaskan bahwa sebuah *smart system* dicirikan oleh adanya elemen sistem, relasi antar entitas, objektif yang terukur, batasan lingkungan, dan antarmuka yang mengintegrasikan sistem dengan dunia luar. *Smart Warehouse* memenuhi seluruh karakteristik tersebut dengan objektif tunggal: meningkatkan produktivitas dan akurasi operasional gudang secara berkelanjutan.

Dalam konteks tugas akhir ini, pilar *Cyber-Physical Integration* menjadi landasan konseptual yang paling relevan bagi pengembangan *Warehouse Management Controller* (WMC). Integrasi siber-fisik mengacu pada sinergi antara lapisan fisik—yaitu pergerakan robot dan barang di dalam gudang—dengan lapisan digital berupa pencatatan data inventori secara *real-time* pada basis data terpusat berbasis *cloud*. Melalui pendekatan ini, setiap aktivitas fisik di lapangan, mulai dari penerimaan barang (PA01), penyimpanan cerdas (PA02), hingga proses *picking* dan manajemen gudang (PA03), secara otomatis tercermin dalam sistem informasi digital tanpa memerlukan intervensi manual. Digitalisasi berbasis *Cyber-Physical Integration* inilah yang menjadi dasar arsitektur WMC, di mana setiap perintah robot dan setiap perubahan status barang diperlakukan sebagai *event* yang langsung direkam dan disinkronkan ke seluruh lapisan sistem.

II.2 Landasan Teori: Metode Perancangan dan Evaluasi Perangkat Lunak

II.2.1 Metode Analytical Hierarchy Process (AHP)

Analytical Hierarchy Process (AHP) adalah metode pengambilan keputusan multi-kriteria yang dikembangkan oleh Thomas L. Saaty pada tahun 1970-an. Metode ini bekerja dengan menyusun permasalahan keputusan ke dalam struktur hierarki yang terdiri dari tiga tingkat utama, yaitu tujuan (*goal*), kriteria, dan alternatif solusi. Pada setiap tingkatan, AHP menggunakan teknik perbandingan berpasangan (*pairwise comparison*) untuk mengukur tingkat kepentingan relatif antara satu elemen dengan elemen lainnya menggunakan skala numerik 1 hingga 9 yang dikembangkan oleh Saaty. Hasil perbandingan tersebut diolah melalui perhitungan bobot prioritas (*eigenvector*) dan divalidasi melalui uji konsistensi dengan indikator *Consistency Ratio* (CR), di mana nilai CR di bawah 0,1 mengindikasikan bahwa penilaian yang dilakukan bersifat konsisten dan dapat diterima (Saaty 1977). Dalam konteks proyek *Smart Warehouse* ini, metode AHP telah diimplementasikan pada dokumen B100 untuk mengevaluasi dan memilih alternatif solusi terbaik dari setiap subsistem, mencakup solusi pemindahan barang, pengangkatan barang, dan pengendalian armada, berdasarkan kriteria akurasi, presisi, kestabilan, kemudahan penggunaan, kompleksitas produksi, dan biaya implementasi (Tim TA252601001 2025a).

II.2.2 Pemodelan Traceability (Keterlacakan) dan Latensi Data

Traceability atau keterlacakan adalah kemampuan sistem untuk mendokumentasikan dan merekonstruksi secara akurat seluruh rangkaian aktivitas operasional yang berkaitan dengan pergerakan barang dan status perangkat di dalam gudang, sebagaimana ditetapkan dalam standar ISO 9001:2015 sebagai bagian dari persyaratan pengendalian informasi terdokumentasi (International Organization for Standardization 2015). Dalam konteks sistem *real-time* seperti WMC, keterlacakan tidak hanya menyangkut kelengkapan data, tetapi juga kecepatan propagasi data tersebut antar-lapis sistem—dari lapisan fisik robot hingga lapisan informasi digital—yang diukur melalui parameter latensi. Berdasarkan spesifikasi yang ditetapkan dalam dokumen B200, sistem dinyatakan memenuhi persyaratan *traceability* apabila seluruh aktivitas robot tercatat tanpa kehilangan data, dengan sinkronisasi waktu antara robot dan server serta antara server dan WMS tidak melebihi 200 milidetik, serta pembaruan informasi status robot—meliputi posisi, kondisi baterai, dan status tugas—dikirimkan ke server dengan latensi tidak lebih dari 1 detik pada kondisi jaringan yang stabil (Tim TA252601001 2025b). Kedua parameter terukur ini menjadi acuan kuantitatif dalam perancangan arsitektur WMC, khususnya dalam menentukan mekanisme pencatatan *event* dan frekuensi *polling* data antara *Fleet Controller* dan

layanan *backend*.

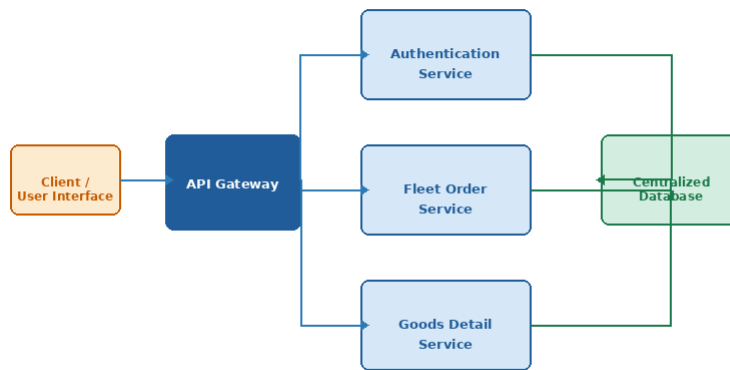
II.3 Landasan Teori: Arsitektur Warehouse Management Controller

II.3.1 Model Manajemen Inventori Berbasis Event (Event-Driven Inventory Management)

Event-Driven Inventory Management adalah pendekatan arsitektur sistem di mana setiap perubahan kondisi inventori—mulai dari penerimaan *order*, reservasi stok, eksekusi perpindahan barang oleh robot, hingga konfirmasi penyelesaian tugas—diperlakukan dan dicatat sebagai sebuah *event* diskrit yang bersifat *immutable* dan berurutan secara kronologis (Fowler 2002). Berbeda dengan model *polling* konvensional yang memeriksa kondisi sistem secara periodik, pendekatan *event-driven* hanya memproses perubahan *state* pada saat perubahan tersebut benar-benar terjadi, sehingga konsumsi sumber daya sistem lebih efisien dan latensi respons jauh lebih rendah. Keunggulan krusial dari pendekatan ini dalam konteks gudang otomatis adalah kemampuannya menjamin konsistensi data stok meskipun terjadi kegagalan pada lapisan fisik robot, karena setiap *event* yang telah tercatat di sistem tetap dapat direkonstruksi dan diverifikasi secara independen dari kondisi perangkat keras di lapangan (Geest, Tekinerdogan, dan Catal 2021). Berdasarkan evaluasi menggunakan metode AHP pada dokumen B100, arsitektur *event-driven inventory management* dipilih sebagai solusi manajemen informasi yang paling optimal untuk sistem *Smart Warehouse* ini, dengan implementasinya pada WMC mencakup pencatatan *Order Status Event* dari *Fleet Order Service*, pemrosesan *Robot Task Events* secara internal, serta pembaruan *Robot Status Event* ke seluruh lapisan sistem secara *real-time* (Tim TA252601001 2025c).

II.3.2 Arsitektur Microservices

Arsitektur *microservices* adalah gaya perancangan perangkat lunak di mana sebuah aplikasi dibangun sebagai kumpulan layanan kecil yang berdiri sendiri (*loosely coupled*), masing-masing menjalankan satu tanggung jawab bisnis yang spesifik dan dapat dikembangkan, di-*deploy*, serta diskalakan secara independen satu sama lain (Newman 2015). Berbeda dengan arsitektur monolitik di mana seluruh komponen terkompilasi dan berjalan dalam satu unit, pada arsitektur *microservices* setiap layanan berkomunikasi melalui antarmuka yang terdefinisi dengan baik—umumnya berupa *RESTful API* atau *message broker*—sehingga kegagalan pada satu layanan tidak secara langsung melumpuhkan keseluruhan sistem (Newman 2015). Pola ini memungkinkan tim pengembang untuk memilih teknologi yang paling sesuai untuk setiap layanan secara independen, meningkatkan ketahanan sistem secara keseluruhan, serta mempercepat siklus pengembangan dan pembaruan fitur tanpa harus



Gambar 2.3.2.1 Ilustrasi Arsitektur Microservices pada Warehouse Management Controller

Gambar II.1 Ilustrasi arsitektur *microservices* pada *Warehouse Management Controller*

melakukan *redployment* pada keseluruhan aplikasi (Richardson 2018).

Dalam implementasi *Warehouse Management Controller* (WMC) pada tugas akhir ini, pendekatan *microservices* diwujudkan melalui tiga layanan utama yang beroperasi secara independen namun saling terhubung melalui sebuah basis data terpusat. Pertama, *Authentication Service* bertugas memverifikasi identitas pengguna dan menghasilkan *Bearer Token* sebagai kredensial akses yang dikirimkan ke seluruh layanan lainnya. Kedua, *Goods Detail Service* mengelola data barang di dalam gudang, menerima permintaan informasi dari *User Interface*, dan memperbarui catatan inventori di basis data terpusat secara *real-time*. Ketiga, *Fleet Order Service* bertanggung jawab atas pembuatan dan pengelolaan *order stocking* maupun *picking*, menerima perintah dari *User Interface*, serta mengordinasikan eksekusi tugas dengan *Fleet Controller* yang mengelola armada robot di lapangan (Tim TA252601001 2025c). Pembagian tanggung jawab ini memastikan bahwa setiap aspek operasional WMC dapat dikembangkan dan diuji secara terpisah, sesuai dengan prinsip *single responsibility* dalam desain perangkat lunak modern.

II.3.3 Infrastruktur Komunikasi Berbasis Message Broker

Komunikasi antar komponen dalam sistem terdistribusi dapat dilakukan melalui dua pendekatan utama: komunikasi sinkron (*request-reply* langsung seperti REST API) dan komunikasi asinkron berbasis pesan (*message-passing*). Pada arsitektur WMC dan *Fleet Controller* yang terdiri dari beberapa layanan independen, penggunaan mekanisme komunikasi asinkron dan *real-time* menjadi kebutuhan yang tidak dapat dipenuhi oleh REST API semata. Oleh karena itu, sistem ini mengadopsi tiga infras-

struktur komunikasi pelengkap, yaitu RabbitMQ sebagai *message broker* antar layanan, MQTT sebagai protokol untuk komunikasi robot-server, serta WebSocket untuk pembaruan status *real-time* ke *User Interface*. Ketiga mekanisme ini dipilih berdasarkan evaluasi AHP yang dilakukan pada tahap analisis kebutuhan non-fungsional, khususnya untuk memenuhi KNF-02 (latensi pembaruan status ≤ 200 ms) dan KNF-06 (*deployability* independen antar layanan) (Tim TA252601001 2025a).

II.3.3.1 Message Broker dan RabbitMQ

Message broker adalah komponen perangkat lunak yang berfungsi sebagai perantara dalam pengiriman pesan antar produsen (*producer*) dan konsumen (*consumer*), memungkinkan kedua pihak untuk beroperasi secara asinkron dan *loosely coupled* tanpa harus saling mengetahui keberadaan satu sama lain secara langsung (Hohpe dan Woolf 2003). Pola ini pertama kali diformalkan oleh Hohpe dan Woolf (2003) dalam *Enterprise Integration Patterns* sebagai solusi integrasi sistem heterogen di mana komponen-komponen perangkat lunak berkomunikasi melalui pengiriman pesan diskrit melalui sebuah *event bus* atau kanal terpusat. RabbitMQ adalah implementasi *message broker open-source* yang mengimplementasikan protokol *Advanced Message Queuing Protocol* (AMQP), di mana pesan yang dipublikasikan oleh *producer* dikirimkan ke *exchange*, lalu diteruskan ke antrian (*queue*) yang sesuai berdasarkan *routing key*, dan akhirnya dikonsumsi oleh *consumer* yang berlangganan pada antrian tersebut (RabbitMQ 2023).

Dalam arsitektur WMC pada tugas akhir ini, RabbitMQ berperan sebagai *broker* terpusat yang memfasilitasi dua jalur komunikasi utama. Pertama, komunikasi antar layanan WMC (*Fleet Order Service* ke *Goods Detail Service*) untuk menyampaikan *event* pembaruan stok secara asinkron setelah eksekusi *order*. Kedua, RabbitMQ juga bertindak sebagai MQTT *broker* yang meneruskan pesan dari Subsistem Lokalisasi (*Fleet Controller*) ke *Fleet Order Service* melalui *topic location*. Pemilihan RabbitMQ dibandingkan alternatif lain didasarkan pada kemampuannya mendukung protokol AMQP dan MQTT secara bersamaan dalam satu *instance*, sehingga mengurangi jumlah komponen infrastruktur yang perlu dioperasikan secara independen, sesuai dengan prinsip KNF-06 (Tim TA252601001 2025a).

II.3.3.2 Protokol MQTT untuk Komunikasi Robot-Server

Message Queuing Telemetry Transport (MQTT) adalah protokol komunikasi *publish-subscribe* yang dirancang khusus untuk perangkat dengan keterbatasan sumber daya dan koneksi jaringan yang tidak andal, seperti perangkat IoT dan sensor terdistribusi (OASIS 2014). Protokol ini bekerja dengan model tiga komponen: *publisher* yang

mempublikasikan pesan ke sebuah topik (*topic*), *broker* yang menerima dan mendistribusikan pesan, serta *subscriber* yang berlangganan pada topik tertentu untuk menerima pesan secara *real-time*. Keunggulan utama MQTT dibandingkan protokol komunikasi berbasis HTTP adalah *overhead* yang sangat kecil—*header* paket MQTT hanya berukuran minimal 2 *byte*—sehingga sangat efisien untuk pengiriman data telemetri berfrekuensi tinggi seperti posisi robot yang diperbarui setiap 500 ms.

Pada sistem *Smart Warehouse* ini, MQTT digunakan untuk dua jalur komunikasi yang berbeda. Pertama, Subsistem Lokalisasi mempublikasikan *grid state* seluruh objek di area gudang ke *topic location* setiap 500 ms. Kedua, *Fleet Controller* mempublikasikan instruksi tugas (*task*) ke robot melalui *topic task*. *Fleet Order Service* pada WMC berlangganan pada *topic location* untuk meneruskan data posisi robot ke *User Interface* secara *real-time* melalui WebSocket, sehingga latensi *end-to-end* dari pembaruan posisi fisik robot hingga tampil di layar pengguna terikat pada interval 500 ms, jauh di bawah target KNF-02 sebesar 1 detik (Tim TA252601001 2025a).

II.3.3.3 WebSocket untuk Pembaruan Status Real-Time

WebSocket adalah protokol komunikasi berbasis TCP yang memungkinkan koneksi persisten dua arah (*full-duplex*) antara server dan klien, berbeda dengan paradigma *request-response* pada HTTP di mana setiap pertukaran data memerlukan inisiasi koneksi baru dari sisi klien (Fette dan Melnikov 2011). Setelah koneksi WebSocket terbentuk melalui mekanisme HTTP *Upgrade handshake*, server dapat mengirimkan data ke klien kapan saja tanpa harus menunggu permintaan dari klien (*server push*), sehingga latensi pengiriman data hanya dibatasi oleh latensi jaringan, bukan oleh interval *polling*. Keunggulan ini menjadikan WebSocket sebagai pilihan yang jauh lebih efisien dibandingkan teknik *long-polling* atau *Server-Sent Events* (SSE) untuk kasus penggunaan di mana data berubah secara kontinu dengan frekuensi tinggi, seperti pemantauan posisi robot secara *real-time*.

Dalam implementasi WMC pada tugas akhir ini, WebSocket server diintegrasikan langsung ke dalam *Fleet Order Service*. Koneksi WebSocket yang terbentuk antara server dan *User Interface* menyediakan dua *stream* data secara bersamaan: *order stream* yang mendorong (*push*) transisi status *order* (PENDING→IN_PROGRESS→COMPLETED) secara langsung tanpa *polling*, serta *robot stream* yang meneruskan *payload* koordinat grid dan orientasi robot yang diterima dari *topic* MQTT *location*. Pendekatan ini memenuhi KNF-02 dengan menjamin bahwa setiap pembaruan status *order* dan posisi robot dapat ditampilkan di *User Interface* dalam kurun waktu yang terikat langsung pada interval publikasi Subsistem Lokalisasi (500 ms), tanpa menambah

latensi dari mekanisme *polling* (Tim TA252601001 2025a).

II.3.4 Manajemen Basis Data Relasional untuk Inventori Real-Time

Basis data relasional adalah sistem penyimpanan data yang mengorganisasikan informasi ke dalam tabel-tabel (relasi) yang saling terhubung melalui kunci asing (*foreign key*), berdasarkan model relasional yang diperkenalkan oleh Codd (1970). Sistem ini menjamin integritas data melalui properti ACID—*Atomicity*, *Consistency*, *Isolation*, dan *Durability*—yang memastikan bahwa setiap transaksi *database* dieksekusi secara penuh atau tidak sama sekali, sehingga tidak ada kondisi data yang setengah diperbarui meskipun terjadi kegagalan sistem di tengah proses (Ramakrishnan dan Gehrke 2003). Dalam konteks manajemen inventori *real-time*, properti ACID menjadi sangat kritis karena setiap perubahan jumlah stok, perubahan lokasi barang, maupun pembaruan status *order* harus langsung tercermin secara konsisten di seluruh layanan yang mengakses basis data secara bersamaan, tanpa risiko pembacaan data yang tidak konsisten (*dirty read*) antarproses yang berjalan paralel.

Dalam arsitektur WMC pada tugas akhir ini, dipilih pendekatan basis data terpusat (*centralized database*) yang menjadi satu-satunya sumber kebenaran (*single source of truth*) bagi seluruh layanan *microservices*. Basis data ini menyimpan empat entitas data utama, yaitu data barang (*goods data*) yang mencakup identitas, jumlah, lokasi, dan kondisi stok; data pesanan (*order data*) yang merekam instruksi *stocking* dan *picking* beserta statusnya; data sesi pengguna (*session data*) untuk keperluan autentikasi; serta data sinkronisasi (*synced data*) yang merepresentasikan kondisi terkini gudang untuk dikirimkan ke sistem eksternal (Tim TA252601001 2025c). Pemilihan arsitektur terpusat dibandingkan basis data terdistribusi per layanan didasarkan pada kebutuhan konsistensi yang tinggi antara *Goods Detail Service*, *Fleet Order Service*, dan *Authentication Service*—di mana setiap *query* lokasi barang oleh *Fleet Order Service* harus selalu mencerminkan pembaruan terkini dari *Goods Detail Service* secara langsung, tanpa mekanisme sinkronisasi tambahan yang dapat menambah latensi maupun risiko inkonsistensi data.

II.4 Tinjauan Penelitian Terdahulu

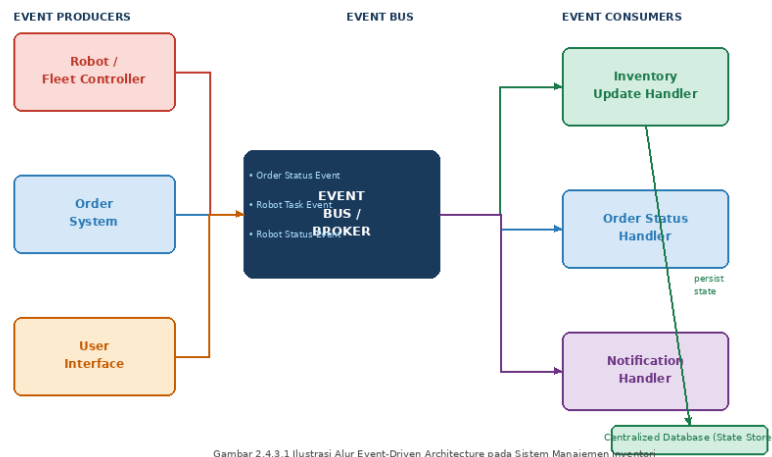
II.4.1 Re-desain Proses Pemenuhan Pesanan (Framinan et al., 2025)

Framinan, Ruiz-Usano, dan Leisten (2025) dalam penelitian berjudul *Redesigning the Customer Order Fulfilment Process in a Company in the FMCG Sector?* yang dipublikasikan di *IFAC-PapersOnLine* mengkaji proses pemenuhan pesanan pada sebuah perusahaan FMCG yang mengalami *bottleneck* akibat sistem pergudangan yang masih bergantung pada proses manual. Menggunakan metode *Business Pro-*

cess Redesign (BPR), penelitian ini menemukan bahwa 62% keterlambatan pengiriman bersumber dari aktivitas *order picking* dan *staging* yang tidak terkoordinasi secara digital, dan menegaskan bahwa sistem *warehouse* konvensional tidak mampu memenuhi tuntutan permintaan tinggi dan variasi SKU yang luas tanpa dukungan teknologi otomasi dan integrasi sistem informasi yang menyeluruh. Temuan ini memiliki relevansi langsung terhadap kondisi operasional gudang barang jadi Paragon Corp, di mana ketergantungan pada operator dalam proses *stocking* dan *picking*—serta ketiadaan integrasi *real-time* antara WMS lokal dengan sistem pusat—menghasilkan pola inefisiensi yang paralel: *lead time* yang panjang, *mismatch* data inventori, dan tingginya risiko *human error*. Penelitian ini memperkuat urgensi pengembangan WMC sebagai solusi digitalisasi yang secara langsung merespons akar permasalahan yang diidentifikasi oleh Framinan, Ruiz-Usano, dan Leisten (2025).

II.4.2 Integrasi Cloud dan IoT untuk Visibilitas Gudang (van Geest et al., 2021)

Geest, Tekinerdogan, dan Catal (2021) dalam artikel ilmiah berjudul “Smart Warehouses: Rationale, Challenges, and Solution Directions” yang diterbitkan di jurnal *Computers in Industry* menyajikan *systematic review* mengenai evolusi *Smart Warehouse* sebagai respons terhadap meningkatnya kompleksitas rantai pasok modern. Penelitian ini mengidentifikasi empat pilar *technology enabler* utama *Smart Warehouse*, yaitu *Automation and Robotics* (penggunaan AGV/AMR untuk transportasi internal), *IoT-based Visibility* (sensor dan RFID untuk deteksi posisi barang secara *real-time*), *AI-driven Optimization* (penjadwalan dan penentuan rute otomatis), serta *Cyber-Physical Integration* sebagai sinergi antara operasi fisik dan sistem digital berbasis *cloud*. Selain mengidentifikasi pilar-pilar tersebut, penulis juga mengklasifikasikan tantangan implementasi ke dalam tiga kategori, yaitu tantangan operasional (integrasi multi-robot dan koordinasi sistem), tantangan teknologi (interoperabilitas antar platform, latensi *cloud*, dan keamanan siber), serta tantangan organisasi (resistensi sumber daya manusia dan biaya transformasi). Relevansi penelitian ini terhadap tugas akhir sangat substansial: kondisi Paragon Corp yang masih berada pada tahap awal integrasi digital secara langsung mencerminkan tantangan teknologi yang diidentifikasi Geest, Tekinerdogan, dan Catal (2021), sementara arsitektur WMC yang dikembangkan—dengan basis data terpusat dan pencatatan *event real-time*—merupakan implementasi konkret dari pilar *Cyber-Physical Integration* yang direkomendasikan penelitian tersebut untuk meningkatkan konsistensi distribusi dan visibilitas stok secara menyeluruh.



Gambar II.2 Ilustrasi alur *Event-Driven Architecture* pada sistem manajemen inventori

II.4.3 Implementasi Event-Driven Architecture pada Sistem Manajemen Inventori Real-Time

Hohpe dan Woolf (2003) dalam *Enterprise Integration Patterns* meletakkan fondasi teoritis arsitektur berbasis *event* sebagai paradigma integrasi sistem di mana komponen-komponen perangkat lunak berkomunikasi melalui pengiriman dan penerimaan pesan (*event*) secara asinkron melalui sebuah *event bus* atau *broker* terpusat, sehingga setiap komponen tetap *loosely coupled* dan dapat beroperasi secara independen tanpa harus mengetahui keberadaan komponen lain secara langsung. Dalam konteks manajemen inventori *real-time*, pendekatan ini diwujudkan melalui tiga kategori komponen utama: *event producers* yang menghasilkan *event* (seperti *Fleet Controller*, *Order System*, dan *User Interface*), *event bus* sebagai kanal distribusi pesan, serta *event consumers* yang memproses *event* sesuai tanggung jawabnya masing-masing (seperti *Inventory Update Handler*, *Order Status Handler*, dan *Notification Handler*), sebagaimana diilustrasikan pada Gambar II.2. Setiap perubahan kondisi inventori—mulai dari eksekusi tugas robot, pembaruan status pesanan, hingga konfirmasi pengiriman barang—dipublikasikan sebagai *event* diskrit yang kemudian dikonsumsi oleh *handler* terkait untuk memperbarui *state* pada basis data terpusat secara atomik (Tim TA252601001 2025c). Relevansi tinjauan ini terhadap tugas akhir terletak pada adopsi pola yang sama dalam arsitektur WMC: *Order Status Event*, *Robot Task Event*, dan *Robot Status Event* masing-masing diproses oleh *handler* yang berbeda di dalam layanan *Fleet Order Service* dan *Goods Detail Service*, menjamin bahwa data inventori di basis data terpusat selalu mencerminkan kondisi fisik terkini di lapangan tanpa ketergantungan langsung antar layanan.

II.5 Landasan Teori: Subsistem Lokalisasi

II.5.1 AprilTag sebagai Fiducial Marker

AprilTag adalah sistem *fiducial marker* yang dikembangkan untuk keperluan *computer vision* pada aplikasi robotika. Setiap *tag* mengodekan ID unik dalam format pola biner berdasarkan keluarga kode tertentu, sehingga detektor dapat membedakan ratusan *tag* secara bersamaan dalam satu *frame* kamera. Keunggulan utama AprilTag dibandingkan alternatif seperti QR Code atau ArUco adalah kemampuannya memberikan estimasi *pose* 6-DOF (*degree of freedom*) secara lengkap—mencakup posisi (X, Y, Z) dan orientasi (*roll*, *pitch*, *yaw*)—dari satu gambar kamera *monocular* tanpa infrastruktur *sensing* aktif (Hartley dan Zisserman 2004). Dalam proyek ini, digunakan keluarga Tag36h11 berukuran fisik 8×8 cm yang ditempelkan pada robot dan barang di lantai gudang untuk keperluan identifikasi serta penentuan posisi dan orientasi secara *overhead*. Konfigurasi kamera *overhead* dipilih karena memungkinkan satu kamera yang dipasang di langit-langit untuk mengamati seluruh area lantai gudang tanpa oklusi dari rak atau robot (Bradski 2000).

II.5.2 Koreksi Distorsi Lensa Fisheye (Model Kannala–Brandt)

Lensa *fisheye* menyediakan sudut pandang yang sangat lebar sehingga memungkinkan satu kamera yang dipasang di langit-langit untuk mengamati seluruh area lantai gudang dari satu titik pemasangan. Namun, optik *fisheye* memperkenalkan distorsi radial yang parah: garis lurus tampak melengkung pada gambar dan jarak piksel tidak seragam di seluruh *frame*, sehingga pemetaan langsung dari koordinat piksel ke posisi fisik menjadi tidak valid (Bradski 2000). Distorsi ini harus dikoreksi sebelum operasi geometri apa pun dapat dilakukan pada gambar.

Model kalibrasi Kannala–Brandt merupakan model polinomial berbasis proyeksi ekuidistansi yang memodelkan distorsi lensa *fisheye* melalui serangkaian koefisien kalibrasi. Koefisien ini dihitung saat proses *setup* kalibrasi menggunakan papan catur (*checkerboard*) yang diletakkan di berbagai sudut dan jarak di depan kamera. Setelah koefisien diperoleh, proses *undistortion* diterapkan pada setiap *frame* video menggunakan OpenCV sebelum pemrosesan lebih lanjut, menghasilkan gambar yang secara geometri konsisten dan valid untuk operasi perspektif selanjutnya.

II.5.3 Perspective Homography dan Pemetaan Koordinat Grid

Setelah koreksi distorsi lensa, koordinat piksel dari titik tengah *tag* yang terdeteksi perlu diubah menjadi koordinat posisi di lantai gudang. Transformasi ini dilakukan menggunakan *perspective homography*, yaitu transformasi projektif yang memetakan titik pada satu bidang datar ke bidang datar lainnya. *Homography* merupakan

transformasi yang tepat untuk kasus ini karena lantai gudang bersifat planar dan kamera mengamatinya dari atas secara tegak lurus, sehingga setiap titik di lantai memiliki korespondensi satu-ke-satu dengan piksel di citra (Hartley dan Zisserman 2004).

Secara matematis, *perspective homography* direpresentasikan sebagai matriks H berukuran 3×3 . Representasi ini menggunakan koordinat homogen, yaitu skema di mana titik 2D (u, v) diangkat ke ruang tiga dimensi sebagai triplet $(u, v, 1)$. Dimensi ketiga ini memungkinkan transformasi perspektif—yang secara inheren non-linear akibat adanya pembagian—ditulis ulang dalam bentuk perkalian matriks linear (Bradski 2000). Perkalian matriks menghasilkan triplet antara (g'_x, g'_y, w') yang kemudian dikembalikan ke koordinat 2D melalui *perspective division*, yaitu pembagian dengan komponen ketiga w' :

$$(g'_x, g'_y, w') = H \cdot (u, v, 1) \quad (\text{II.1})$$

dengan

$$H = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & 1 \end{bmatrix} \quad (\text{II.2})$$

Elemen ke-9 matriks H dinormalisasi menjadi $h_{22} = 1$ sehingga H memiliki 8 derajat kebebasan. Koordinat lantai kontinu (g_x, g_y) diperoleh melalui *perspective division* terhadap hasil perkalian matriks tersebut:

$$g_x = \frac{h_{00} \cdot u + h_{01} \cdot v + h_{02}}{h_{20} \cdot u + h_{21} \cdot v + 1} \quad (\text{II.3})$$

$$g_y = \frac{h_{10} \cdot u + h_{11} \cdot v + h_{12}}{h_{20} \cdot u + h_{21} \cdot v + 1} \quad (\text{II.4})$$

Matriks H dihitung satu kali pada tahap kalibrasi dari empat titik korespondensi yang diketahui: empat sudut area lantai gudang yang koordinat pikselnya (u_i, v_i) diperoleh melalui klik manual pengguna pada citra kalibrasi, dan koordinat *grid* targetnya (g_{xi}, g_{yi}) ditetapkan sebagai sudut-sudut *rectangle grid*, yaitu $(0, 0)$, $(C, 0)$, (C, R) , dan $(0, R)$ dengan $C = 8$ kolom dan $R = 4$ baris. Setiap pasangan korespondensi menghasilkan dua persamaan linear setelah perkalian silang menghilangkan

penyebut (Hartley dan Zisserman 2004):

$$h_{00}u_i + h_{01}v_i + h_{02} - h_{20}u_i g_{xi} - h_{21}v_i g_{xi} = g_{xi} \quad (\text{II.5})$$

$$h_{10}u_i + h_{11}v_i + h_{12} - h_{20}u_i g_{yi} - h_{21}v_i g_{yi} = g_{yi} \quad (\text{II.6})$$

Dengan empat titik korespondensi, terbentuk sistem linear 8×8 dalam vektor parameter $h = (h_{00}, h_{01}, h_{02}, h_{10}, h_{11}, h_{12}, h_{20}, h_{21})$ yang diselesaikan secara eksak karena jumlah persamaan sama dengan jumlah variabel. Penyelesaian sistem linear ini dilakukan secara internal oleh fungsi `getPerspectiveTransform` pada pustaka OpenCV, sehingga pengguna hanya perlu menyediakan empat pasang titik korespondensi untuk memperoleh matriks H yang lengkap (Bradski 2000).

Koordinat rantai kontinu (g_x, g_y) yang diperoleh kemudian di-*snap* ke sel *grid* diskrit (c, r) menggunakan aturan *nearest-cell-centre*. Karena setiap sel berukuran 1×1 dalam satuan *grid* dan berpusat pada titik $(c + 0, 5, r + 0, 5)$, formula *snap* adalah:

$$c = \text{clip}(\text{round}(g_x - 0, 5), 0, 7) \quad (\text{II.7})$$

$$r = \text{clip}(\text{round}(g_y - 0, 5), 0, 3) \quad (\text{II.8})$$

Operasi *clip* memastikan indeks sel tidak keluar dari batas *grid* 8×4 , sehingga seluruh titik di area lantai gudang $(2, 4 \times 1, 2 \text{ m})$ selalu terpetakan ke dalam 32 sel yang valid. Pendekatan ini memungkinkan satu kamera *overhead* memetakan posisi seluruh robot, barang, dan *docking station* ke representasi *grid* diskrit secara *real-time* tanpa sensor tambahan.

BAB III

ANALISIS MASALAH

III.1 Analisis Kondisi Saat Ini

Sebelum mendefinisikan kebutuhan dan solusi pengembangan *Warehouse Management Controller* (WMC), perlu dipahami terlebih dahulu kondisi operasional pergudangan PT Paragon Technology and Innovation saat ini beserta permasalahan yang melekat di dalamnya. Analisis kondisi saat ini dilakukan dengan menggali informasi langsung melalui wawancara dengan pihak Paragon Corp, menelaah proses bisnis *warehouse* eksisting, serta mengidentifikasi kesenjangan (*gap*) antara kondisi tersebut dengan target *Smart Warehouse*. Seluruh analisis pada subbab ini bersumber dari hasil wawancara serta dokumen B100 subbab 2.1.3 Analisis Masalah (Tim TA252601001 2025a).

III.1.1 Profil Operasional Gudang Berdasarkan Hasil Wawancara

Guna memperoleh gambaran kondisi operasional yang akurat dan kontekstual, kelompok TA252601001 melaksanakan wawancara daring dengan Bapak Falih Muhtadi selaku *Staff Logistics and Business Integration Systems* PT Paragon Technology and Innovation (Paragon Corp) pada tanggal 16 Oktober 2025. Dari wawancara tersebut diketahui bahwa Paragon Corp memiliki tiga jenis gudang, yaitu gudang bahan baku, gudang kemasan (*packaging warehouse*), serta gudang barang jadi (*Finished Goods*) yang terletak pada area terpisah. Fokus pengembangan tugas akhir ini adalah pada gudang barang jadi, yang secara langsung bersinggungan dengan proses distribusi produk ke konsumen akhir.

Berdasarkan hasil wawancara, sistem kerja operasional inventori Paragon Corp menerapkan pendekatan *Pull System*, yaitu suatu mekanisme di mana pergerakan dan pemrosesan barang didasarkan pada permintaan aktual yang masuk ke sistem, bukan pada prakiraan (*forecasting*) internal. Alur kerja utama gudang mencakup empat tahapan, yaitu *Incoming*, *Stocking*, *Picking & Checking*, serta *Dispatching* menuju *Distribution Center*. Dalam skema *pull system*, perintah *stocking* dan *picking* baru dieksekusi ketika terdapat permintaan nyata, sehingga meminimalkan akumulasi

stok berlebih (*overstock*) namun di sisi lain menuntut kecepatan respons sistem yang sangat tinggi agar tidak menimbulkan *bottleneck* pada tahap distribusi.

Dari hasil wawancara juga terungkap bahwa teknologi yang saat ini digunakan di gudang Paragon mencakup sistem ERP Odoo dan *Warehouse Management System* (WMS) untuk pencatatan dan pelacakan barang. Namun, kedua sistem tersebut masih bersifat semi-manual sehingga membutuhkan input operator secara berkala dan belum memiliki integrasi *real-time* antar platform. Kondisi ini menyebabkan *mismatch* data yang sering terjadi antara kondisi fisik di lapangan dengan catatan digital pada sistem pusat. Paragon Corp sendiri saat ini tengah dalam proses transformasi menuju sistem SAP untuk meningkatkan kualitas integrasi antar sistem. Kompleksitas operasional semakin bertambah mengingat Paragon Corp mengelola lebih dari 40 *Distribution Centers* (DC) di Indonesia dan Malaysia, dengan volume produk dan variasi SKU kosmetik yang sangat beragam, sehingga ketersediaan sistem inventori yang akurat dan terintegrasi menjadi kebutuhan operasional yang mendesak.

Wawancara tersebut turut mengonfirmasi permasalahan utama pada gudang barang jadi Paragon Corp, yang meliputi tiga hal: pertama, data stok tidak bersifat *real-time* karena WMS belum terhubung langsung dengan sistem pusat; kedua, proses *inbound* barang yang lambat dan perhitungan yang tidak presisi akibat ketergantungan pada proses manual; serta ketiga, proses *picking* yang masih bergantung pada operator dan rawan *human error*, khususnya dalam konteks jumlah barang. Bapak Falih juga mengonfirmasi adanya permasalahan *mistracking* barang akibat area penempatan sementara yang tidak tercatat di sistem, serta potensi besar untuk mengotomasi proses *inbound*, khususnya pemindahan barang dari truk ke *pallet changer*.

III.1.2 Proses Bisnis Pergudangan Eksisting

Pada industri *Fast Moving Consumer Goods* (FMCG) seperti PT Paragon Technology and Innovation, kebutuhan akan operasional rantai pasok menjadi kunci utama penggerak bisnis. Terdapat empat sumber daya utama yang menggerakkan sebuah *warehouse*, yaitu *storage system*, *lift trucks*, *personnel*, dan *Warehouse Management System* (WMS). Kapasitas masing-masing komponen tersebut saling memengaruhi laju keseluruhan *warehouse* sesuai prinsip *flow impact test* (IFIT). Proses *warehouse* secara umum mencakup: penerimaan (*receiving*), penempatan (*put-away*), pengisian ulang internal (*internal replenishment*), pengambilan pesanan (*order picking*), pengumpulan dan penyortiran (*accumulating and sorting*), pengepakan (*packing*), lintas-dermaga (*cross docking*), pengiriman barang (*dispatch*), dan pengapalan (*shipping*).

Dari analisis proses operasional Paragon Corp yang termuat dalam dokumen B100, teridentifikasi sejumlah kelemahan mendasar pada kondisi eksisting. Pertama, sistem inventori disimpan secara independen dan tidak terintegrasi, sehingga berpotensi menimbulkan *mismatch* data stok antara kondisi fisik dan catatan digital. Kedua, proses pencatatan dan pengambilan barang (*case picking*) masih bergantung pada campur tangan operator manusia secara langsung. Hal ini menjadi hambatan operasional mengingat keterbatasan sumber daya manusia dalam hal kompetensi dan kuantitas merupakan salah satu kendala utama dalam proses *picking* di industri *warehousing*. Ketiga, walaupun Paragon Corp telah menggunakan robot *Automated Guided Vehicle* (AGV) sebagai penunjang mobilisasi barang jadi, manajemen sistem armada AGV tersebut belum memiliki kerangka informasi yang jelas dan terintegrasi. Ketiga permasalahan ini secara kumulatif menciptakan tantangan operasional dalam konteks volume penyimpanan stok yang terus bertambah dan *lead time* yang semakin ketat akibat tingginya permintaan industri FMCG.

Berdasarkan analisis proses bisnis *Smart Warehouse* pada dokumen B100, disusun sejumlah aktivitas primer yang menjadi fokus optimasi sistem, sebagaimana dirangkum pada Tabel III.1.

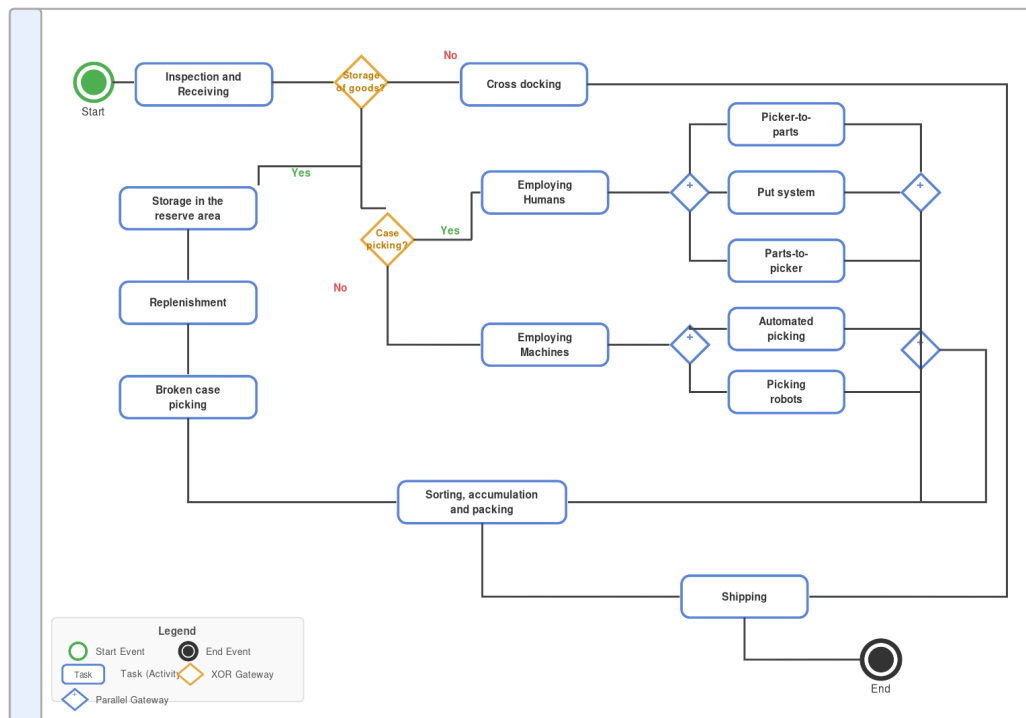
Tabel III.1 Aktivitas Primer pada *Smart Warehouse*

Kode	Activity
PA01	Otomatisasi Penerimaan Stok
PA02	Smart Storage & Inventory
PA03	Order Picking and Warehouse Management Process
PA04	Quality Control
PA05	Smart Vendor Management
PA06	E-Procurement & Smart Vendor Management

Fokus pengembangan *Warehouse Management Controller* (WMC) pada tugas akhir ini diarahkan pada aktivitas PA01, PA02, dan PA03, sebagai aktivitas primer yang paling berkaitan dengan permasalahan otomasi penerimaan, penyimpanan, dan pengelolaan *order* pada gudang Paragon Corp.

III.1.3 Kondisi Sistem Saat Ini (As-Is)

Pada kondisi saat ini, proses operasional gudang pada industri *Fast Moving Consumer Goods* (FMCG) seperti Paragon Corp berjalan secara konvensional dengan mengandalkan tenaga kerja manusia dan proses manual di hampir seluruh tahapan. Alur proses keseluruhan pada kondisi eksisting digambarkan pada Gambar III.1.



Gambar III.1 Diagram BPMN proses bisnis *warehouse* kondisi saat ini (*As-Is*)

Proses diawali dengan tahap inspeksi dan penerimaan barang (*inspection and receiving*), di mana staf gudang melakukan verifikasi fisik terhadap barang yang datang, mencakup pengecekan jumlah, kondisi kemasan, serta kesesuaiannya dengan dokumen pengiriman. Seluruh kegiatan pencatatan pada tahap ini masih dilakukan secara manual oleh staf gudang menggunakan dokumen fisik maupun lembar kerja yang tidak terintegrasi satu sama lain. Pendekatan ini rentan terhadap kesalahan pencatatan, ketidaksesuaian data antara dokumen fisik dan catatan digital, serta berpotensi memperlambat keseluruhan alur masuk barang apabila volume penerimaan terjadi dalam jumlah besar secara bersamaan. Tidak adanya mekanisme validasi otomatis pada tahap ini menjadikan keakuratan data penerimaan sepenuhnya bergantung pada ketelitian operator manusia.

Setelah barang berhasil diverifikasi, ditentukan apakah barang perlu disimpan terlebih dahulu ke area cadangan (*storage of goods*) atau dapat langsung diteruskan melalui jalur *cross docking*. Untuk barang yang memerlukan penyimpanan, staf secara manual menempatkan barang ke area penyimpanan cadangan (*reserve area*). Selanjutnya, proses pengisian kembali (*replenishment*) dilakukan untuk memastikan ketersediaan stok pada area aktif. Tidak adanya sistem pemantauan stok secara *real-time* pada tahap ini menyebabkan proses *replenishment* sering kali dilakukan berdasarkan estimasi visual operator di lapangan, sehingga berpotensi menimbulkan

kondisi *stockout* atau *overstock* yang tidak terdeteksi secara cepat. Pada jalur *cross docking*, barang yang tidak memerlukan penyimpanan langsung diteruskan ke tahap pengiriman, namun proses ini tetap bergantung pada koordinasi manual antara staf yang memperbesar risiko miskomunikasi dan keterlambatan.

Selanjutnya, sistem menentukan metode pengambilan barang (*case picking*) yang akan digunakan. Apabila proses *picking* dilakukan dengan melibatkan tenaga manusia (*employing humans*), terdapat tiga metode yang dapat diterapkan, yaitu *picker-to-parts* di mana operator bergerak menuju lokasi barang, *put system* di mana barang dibawa ke stasiun sortir, dan *parts-to-picker* di mana barang datang menuju operator. Keseluruhan metode ini masih bergantung sepenuhnya pada kinerja tenaga kerja manusia, sehingga kapasitas dan kecepatan operasional sangat dipengaruhi oleh kondisi fisik dan jumlah staf yang tersedia pada setiap waktu. Di sisi lain, apabila digunakan metode berbasis mesin (*employing machines*), pilihan yang tersedia mencakup *automated picking* dan *picking robots*, namun kedua sistem ini beroperasi secara terpisah tanpa integrasi terpadu dengan sistem manajemen gudang, sehingga koordinasi antar subsistem masih memerlukan intervensi manual dari staf. Kondisi ini menciptakan *bottleneck* operasional yang membatasi kapasitas pemenuhan pesanan, terutama pada periode permintaan tinggi.

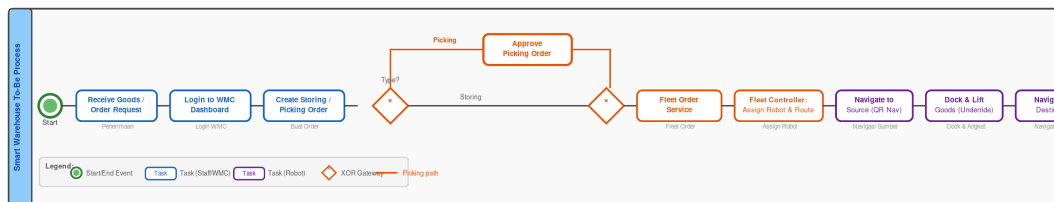
Setelah proses *picking* selesai, barang memasuki tahap sortir, akumulasi, dan pengemasan (*sorting, accumulation, and packing*). Pada tahap ini, staf gudang memilah barang sesuai tujuan pengiriman, mengumpulkan barang per pesanan, dan mengemasnya secara manual untuk selanjutnya dikirimkan. Proses ini sangat bergantung pada ketelitian staf karena tidak terdapat sistem validasi otomatis yang memastikan kesesuaian barang yang dikemas dengan detail pesanan. Kondisi ini menyebabkan tingginya risiko kesalahan pengiriman (*mispick*) yang baru terdeteksi pada tahap akhir distribusi, sehingga berpotensi menimbulkan pengembalian barang dan kerugian operasional. Setelah pengemasan selesai, barang dikirimkan (*shipping*) kepada pelanggan atau distributor tujuan sebagai tahap akhir dari alur operasional gudang.

Secara keseluruhan, kondisi eksisting sistem pergudangan ini menunjukkan beberapa kelemahan struktural yang signifikan. Pertama, seluruh tahap operasional—mulai dari penerimaan, penyimpanan, *picking*, hingga pengiriman—masih mengandalkan pencatatan dan koordinasi manual yang rentan terhadap kesalahan dan inkonsistensi data. Kedua, tidak adanya mekanisme pemantauan stok secara *real-time* menyebabkan ketidaksinkronan yang berulang antara kondisi fisik di lapangan dan catatan digital pada sistem informasi. Ketiga, ketergantungan pada tenaga kerja

manusia pada tahap *picking* menciptakan *bottleneck* operasional yang membatasi kapasitas dan kecepatan pemenuhan pesanan. Keempat, tidak adanya integrasi antar subsistem mengakibatkan visibilitas operasional yang terbatas bagi manajemen, sehingga memperlambat proses pengambilan keputusan serta meningkatkan potensi terjadinya *human error* di setiap tahap operasional.

III.1.4 Kondisi Sistem Smart Warehouse (To-Be)

Pada kondisi sistem *Smart Warehouse* yang diusulkan, proses operasional gudang mengalami transformasi signifikan melalui penerapan *Warehouse Management Controller* (WMC) yang mengintegrasikan antarmuka pengguna berbasis web, layanan manajemen pesanan, pengendalian armada robot, dan pembaruan inventori secara otomatis. Alur proses keseluruhan pada kondisi *to-be* digambarkan pada Gambar III.2.



Gambar III.2 Diagram BPMN proses bisnis *Smart Warehouse* kondisi *To-Be*

Proses dimulai ketika staf gudang melakukan login ke dalam *WMC Dashboard* melalui mekanisme autentikasi berbasis *Bearer Token* yang diterbitkan oleh *Authentication Service*. Setelah terautentikasi, staf membuat pesanan penyimpanan atau pengambilan barang (*storing* atau *picking*) melalui antarmuka WMC. Untuk pesanan *picking*, diperlukan persetujuan dari manajer gudang sebelum pesanan diproses lebih lanjut, sehingga seluruh aktivitas pengambilan barang memiliki rekam jejak otorisasi yang terstruktur.

Pesanan yang telah disetujui diterima oleh *Fleet Order Service* untuk divalidasi dan diteruskan ke *Fleet Controller*. *Fleet Controller* secara otomatis menentukan robot yang tersedia, mengalokasikan tugas, dan membangkitkan *waypoint* rute pergerakan tanpa memerlukan intervensi manual dari staf. Seluruh proses penugasan ini berlangsung secara terpusat melalui sistem kendali yang terintegrasi.

Robot *mobile* yang telah ditugaskan kemudian bergerak secara otonom menuju lokasi sumber barang menggunakan navigasi berbasis pemindaian *QR code* dan sensor IMU. Sesampainya di lokasi, robot melakukan *docking* dan mengangkat barang menggunakan mekanisme *underride lifting*, kemudian bernavigasi menuju lokasi tujuan dan menurunkan barang secara presisi. Seluruh tahap pemindahan barang ini

menggantikan proses *picking* dan *storing* manual yang sebelumnya dilakukan oleh tenaga kerja manusia.

Setelah robot menyelesaikan tugasnya, WMC secara otomatis memperbarui data inventori gudang melalui mekanisme *event-driven* yang mencatat setiap perubahan stok sebagai *event* diskrit. Status pesanan yang telah selesai langsung dikirimkan ke antarmuka WMC sehingga staf dapat memantau kondisi gudang secara *real-time* tanpa perlu melakukan pencatatan manual, sehingga konsistensi antara kondisi fisik di lapangan dan data digital pada sistem selalu terjaga.

III.1.5 Analisis Gap

Dengan memahami kondisi eksisting *warehouse* dan kondisi *Smart Warehouse* yang ditargetkan, dilakukan analisis *gap* untuk mengidentifikasi kesenjangan antara *current condition* dan *future condition* secara terstruktur. Analisis ini menjadi landasan utama dalam merumuskan kebutuhan sistem WMC. Tabel III.2 menyajikan hasil analisis *gap* berdasarkan dokumen B100 Tabel 2.1.3.2.

Tabel III.2 *Gap Analysis* Kondisi Eksisting vs *Smart Warehouse* (B100 Tabel 2.1.3.2)

Kode	Current Condition (Masalah Aktual Paragon)	Gap	Future Condition
MA01	Data stok tidak <i>real-time</i> ; WMS tidak terhubung dengan sistem pusat.	Tidak ada integrasi <i>real-time</i> antar sistem (manual <i>update</i> , rawan <i>error</i>).	Sistem terintegrasi dan sinkron antara <i>warehouse</i> barang jadi dan pusat.
MA02	Pendataan dan penaruhan <i>inbound</i> barang lambat; perhitungan barang tidak presisi.	Ketergantungan tinggi pada proses manual; <i>error rate</i> tinggi dan proses lambat.	Sistem deteksi dan pelacakan barang akurat yang dilakukan secara otomatis.
MA03	<i>Picking</i> manual yang lambat dan ketergantungan dengan operator manusia.	Belum ada sistem otomatisasi untuk memastikan keakuratan permintaan dan waktu pemenuhan.	Sistem perpindahan barang dari semua tahapan dilakukan secara otomatis dan memenuhi keakuratan permintaan serta waktu.

Dari hasil analisis *gap*, teridentifikasi tiga kelompok masalah aktual (MA01–MA03) yang secara langsung menjadi dasar perancangan WMC. Masalah MA01 berkaitan langsung dengan kebutuhan modul *Goods Detail Service* untuk menjamin sinkronisasi stok *real-time*; MA02 mendasari kebutuhan otomasi pencatatan dan pemantauan barang; sedangkan MA03 menjadi latar belakang pengembangan *Fleet Order Service* untuk mengordinasikan armada robot dalam proses *stocking* dan *picking* secara terprogram.

III.2 Analisis Kebutuhan

Analisis kebutuhan bertujuan untuk mengidentifikasi secara sistematis dan menjabarkan seluruh persyaratan yang harus dipenuhi oleh *Warehouse Management Controller* (WMC) guna menyelesaikan permasalahan operasional yang ada. Tahapan ini mencakup penjabaran karakteristik produk, penetapan kebutuhan fungsional dan nonfungsional, serta penjabaran batasan sistem sebagai landasan perancangan solusi yang tepat sasaran.

III.2.1 Karakteristik Produk

Berdasarkan tujuan proyek *Smart Warehouse* yang telah dirumuskan pada dokumen B100 subbab 2.2.1, sistem *Warehouse Management Controller* (WMC) dirancang untuk mengotomatisasi proses *stocking* dan *picking* pada pergudangan, mulai dari pencatatan, pelacakan, hingga perpindahan barang. Sistem ini memastikan integrasi data antara kondisi fisik dan digital secara *real-time*, memungkinkan deteksi dan pemantauan pergerakan barang yang akurat, serta mendukung proses perpindahan barang secara presisi dan tepat waktu. Karakteristik produk tersebut dirangkum dan diberi kodefikasi pada Tabel III.3.

Tabel III.3 Karakteristik Produk Sistem *Smart Warehouse*

Kode	Deskripsi
CH01	Sistem dapat menyinkronkan data stok secara otomatis antara kondisi fisik dan catatan digital untuk menjaga konsistensi informasi inventori.
CH02	Sistem mampu mendeteksi, mencatat, dan memantau pergerakan barang secara menyeluruh dari proses <i>stocking</i> hingga <i>picking</i> .
CH03	Sistem dapat mengatur perpindahan barang secara otomatis dengan presisi sesuai waktu dan kebutuhan <i>order</i> .
CH04	Sistem menyajikan laporan <i>real-time</i> kepada pengguna untuk mendukung pengambilan keputusan operasional.
CH05	Sistem dapat menyesuaikan kapasitas dan alur kerja secara otomatis sesuai perubahan volume dan variasi produk.

lanjutan Tabel III.3

Kode	Deskripsi
CH06	Sistem mampu beradaptasi dengan kebutuhan gudang yang berbeda melalui penyesuaian otomasi tambahan.
CH07	Sistem mampu melakukan pembentukan <i>order stocking</i> secara otomatis.

Karakteristik CH01–CH05 merupakan fungsi dasar yang menjadi inti kapabilitas sistem, sedangkan CH06–CH07 merupakan fungsi dan fitur tambahan. Ketujuh karakteristik produk ini menjadi acuan dalam penurunan kebutuhan fungsional dan nonfungsional WMC pada Subbab III.2.2 dan III.2.3 berikutnya.

III.2.2 Kebutuhan Fungsional

Kebutuhan fungsional mendefinisikan kapabilitas spesifik yang harus dimiliki oleh *Warehouse Management Controller* (WMC) untuk menyelesaikan setiap permasalahan yang telah diidentifikasi. Kebutuhan-kebutuhan berikut diturunkan dari karakteristik produk yang ditetapkan pada dokumen B100 (CH01–CH07) dan disesuaikan dengan lingkup pengembangan WMC sebagai sistem perangkat lunak yang mencakup *Authentication Service*, *Goods Detail Service*, *Fleet Order Service*, dan *User Interface*. Rincian kebutuhan fungsional disajikan pada Tabel III.4.

Tabel III.4 Kebutuhan Fungsional WMC

Kode	Deskripsi Kebutuhan	Acuan
KF-01	Login Pengguna — Pengguna dapat melakukan login menggunakan <i>username</i> dan <i>password</i> yang terdaftar untuk mengakses fitur-fitur sistem WMC.	P-05
KF-02	Logout Pengguna — Pengguna dapat melakukan <i>logout</i> untuk mengakhiri sesi aktif dan keluar dari sistem WMC secara aman.	P-05
KF-03	Melihat Daftar Barang — Pengguna dapat melihat daftar seluruh barang yang tersimpan di gudang beserta informasi ringkas seperti nama, SKU, kategori, dan jumlah stok.	CH01, P-01, P-02
KF-04	Melihat Detail Informasi Barang — Pengguna dapat melihat informasi lengkap suatu barang, mencakup atribut, dimensi, berat, dan kondisi stok terkini.	CH01, P-01

lanjutan Tabel III.4

Kode	Deskripsi Kebutuhan	Acuan
KF-05	Melihat Lokasi Fisik Barang — Pengguna dapat melihat lokasi penyimpanan fisik barang di dalam gudang, mencakup zona, rak, dan slot penyimpanan.	P-02
KF-06	Membuat Order Picking — Pengguna dapat membuat <i>order picking</i> untuk meminta robot mengambil barang dari lokasi penyimpanan ke titik keluaran gudang.	CH07, P-03
KF-07	Membuat Order Storing — Pengguna dapat membuat <i>order storing</i> untuk meminta robot menyimpan barang dari titik masuk ke lokasi penyimpanan yang sesuai.	CH03, P-03
KF-08	Melihat Daftar dan Status Order — Pengguna dapat melihat seluruh daftar <i>order</i> yang telah dibuat beserta status perkembangannya secara <i>real-time</i> .	CH03, CH07, P-03
KF-09	Memantau Status Task Robot — Pengguna dapat memantau status seluruh <i>task</i> yang sedang maupun telah dieksekusi oleh robot, termasuk informasi kemajuan dan log aktivitas.	CH04, P-06
KF-10	Memantau Status Robot — Pengguna dapat memantau kondisi operasional seluruh robot yang terdaftar dalam sistem, termasuk status, dan histori aktivitas robot.	CH04, P-06

Selain kebutuhan fungsional pada lapisan WMC, pengembangan Subsistem Lokalisasi sebagai komponen pendukung *Fleet Controller* juga memerlukan kebutuhan fungsional tersendiri, sebagaimana disajikan pada Tabel III.5.

Tabel III.5 Kebutuhan Fungsional Subsistem Lokalisasi

Kode	Deskripsi Kebutuhan	Acuan
KF-L01	Mendeteksi dan Mengidentifikasi Objek — Sistem dapat mendeteksi dan mengidentifikasi objek (robot, rak, <i>docking station</i>) di area gudang menggunakan <i>marker fiducial</i> unik.	B100/B300 – Lokalisasi
KF-L02	Pemetaan Posisi ke <i>Grid</i> — Sistem dapat memetakan posisi objek terdeteksi ke koordinat <i>grid</i> diskrit sesuai <i>layout</i> gudang.	B300 – Representasi Posisi
KF-L03	Penentuan Orientasi — Sistem dapat menentukan orientasi (<i>heading</i>) objek terdeteksi berbasis sudut pandang atas.	Paper Individu – Klasifikasi & Heading
KF-L04	Publikasi Status Posisi — Sistem dapat mempublikasikan status posisi terbaru secara berkala agar dikonsumsi <i>Fleet Controller</i> dan <i>Fleet Order Service</i> (WMC).	B200 – Traceability

III.2.3 Kebutuhan Nonfungsional

Kebutuhan nonfungsional mendefinisikan batasan kualitas dan performa yang harus dipenuhi oleh WMC sebagai atribut kualitas sistem di luar fungsi utamanya. Kebutuhan-kebutuhan ini diturunkan dari spesifikasi produk yang ditetapkan pada dokumen B200, khususnya pada aspek *Usability*, *Traceability*, *Availability*, dan *Safety* yang relevan dengan komponen perangkat lunak WMC. Rincian kebutuhan nonfungsional disajikan pada Tabel III.6.

Tabel III.6 Kebutuhan Nonfungsional WMC

Kode	Kategori	Deskripsi Kebutuhan	Parameter	Acuan
KNF-01	Usability	Sistem manajemen inventori dan <i>fleet management</i> pada WMC harus memiliki tingkat kemudahan penggunaan yang tinggi sehingga mendukung pengambilan keputusan operasional secara cepat dan minim kesalahan oleh operator gudang.	SUS > 80	B200 – Usability
KNF-02	Traceability & Latensi	Seluruh aktivitas pergerakan barang dan status robot harus tercatat secara lengkap dan berurutan tanpa kehilangan data, dengan sinkronisasi waktu antara robot–server dan server–WMS terjamin, serta pembaruan status robot dikirimkan ke server secara periodik.	Sinkronisasi ≤ 200 ms; status robot ≤ 1 detik	B200 – Traceability
KNF-03	Availability	Seluruh layanan WMC (<i>Authentication Service</i> , <i>Goods Detail Service</i> , <i>Fleet Order Service</i>) harus dapat beroperasi secara kontinu selama jam operasional gudang tanpa <i>downtime</i> yang tidak terencana, guna menjamin ketersediaan sistem bagi operator.	Availability $\geq 95\%$	B200 – Availability

lanjutan Tabel III.6

Kode	Kategori	Deskripsi Kebutuhan	Parameter	Acuan
KNF-04	Keamanan	Seluruh <i>endpoint</i> layanan WMC harus hanya dapat diakses oleh pengguna yang telah terautentikasi melalui mekanisme <i>Bearer Token</i> yang diterbitkan oleh <i>Authentication Service</i> , sehingga tidak ada akses tidak sah terhadap fungsi-fungsi kritis sistem.	0 akses tanpa token valid	P-05, B200 – Safety
KNF-05	Konsistensi Data	Seluruh transaksi pengelolaan data inventori pada basis data terpusat WMC harus memenuhi properti ACID (<i>Atomicity, Consistency, Isolation, Durability</i>) untuk menjamin tidak ada kondisi data yang setengah diperbarui atau inkonsisten antarlayanan meskipun terjadi kegagalan sistem.	0 <i>dirty read</i> ; 0 <i>partial update</i>	B200 – Task Fulfilment Accuracy

lanjutan Tabel III.6

Kode	Kategori	Deskripsi Kebutuhan	Parameter	Acuan
KNF-06	Skalabilitas	Arsitektur <i>microservices</i> WMC harus memungkinkan setiap layanan (<i>Authentication Service, Goods Detail Service, Fleet Order Service</i>) untuk dikembangkan, diuji, dan di-deploy secara independen tanpa memerlukan <i>redployment</i> keseluruhan sistem, guna mendukung adaptasi terhadap kebutuhan gudang yang berbeda.	Independent deploy per service	CH05, CH06

Selain kebutuhan nonfungsional WMC, Subsistem Lokalisasi juga memiliki batasan kualitas dan performa tersendiri yang diturunkan dari dokumen B200, sebagaimana disajikan pada Tabel III.7.

Tabel III.7 Kebutuhan Nonfungsional Subsistem Lokalisasi

Kode	Kategori	Deskripsi Kebutuhan	Parameter	Acuan
KNF-L01	Traceability & Latensi	Status posisi robot diperbarui ke server dengan latensi tidak lebih dari 1 detik pada kondisi jaringan stabil.	Latensi ≤ 1 detik	B200 – Traceability

lanjutan Tabel III.7

Kode	Kategori	Deskripsi Kebutuhan	Parameter	Acuan
KNF-L02	Reliability	Sistem tidak mengalami degradasi akurasi posisi meskipun robot beroperasi berulang kali.	Task Repeatability 100%	B200 – Reliability
KNF-L03	Availability & Safety	Sistem dapat mendeteksi kegagalan pelacakan (<i>mismatch</i> koordinat server vs posisi fisik) dan mengirimkan notifikasi yang memuat koordinat terakhir robot.	Notifikasi memuat <i>last-known coordinate</i>	B200 – Safety

III.3 Analisis Pemilihan Solusi

Berdasarkan identifikasi masalah dan kebutuhan yang telah dirumuskan pada subbab sebelumnya, tahap berikutnya adalah menganalisis dan menentukan solusi arsitektur yang paling sesuai untuk diimplementasikan sebagai *Warehouse Management Controller* (WMC). Analisis ini dilakukan dengan terlebih dahulu mengidentifikasi alternatif-alternatif solusi yang mungkin, kemudian mengevaluasinya secara sistematis menggunakan metode *Analytical Hierarchy Process* (AHP) sebagaimana yang telah ditetapkan pada dokumen B100, sehingga keputusan pemilihan solusi bersifat objektif dan dapat dipertanggungjawabkan secara teknis.

III.3.1 Pemilihan Solusi Level WMC

III.3.1.1 Alternatif Solusi

Dalam pengembangan WMC sebagai sistem perangkat lunak inti *Smart Warehouse*, permasalahan utama yang perlu diselesaikan pada lapisan manajemen data adalah bagaimana sistem mencatat, memperbarui, dan menjaga konsistensi informasi inventori secara *real-time* sejalan dengan aktivitas fisik robot di lapangan. Berdasarkan kajian pada dokumen B100 subbab 2.2.2.4, terdapat dua pendekatan arsitektur

yang dapat dipertimbangkan sebagai solusi manajemen data inventori WMC, yaitu *Event-Driven Inventory Management* dan *State-Driven Inventory Management*. Karakteristik masing-masing alternatif diuraikan sebagai berikut.

a. Alternatif 1: Event-Driven Inventory Management

Pada pendekatan *Event-Driven Inventory Management*, WMC mengelola *order*, stok, dan proses pemindahan barang secara terintegrasi melalui mekanisme pencatatan *event*. Ketika *order stocking* atau *picking* dibuat melalui *Fleet Order Service*, sistem langsung mencatat *order* dan melakukan reservasi stok secara internal untuk memastikan ketersediaan barang tanpa langsung mengurangi stok secara final. Setelah reservasi dilakukan, sistem mengirimkan instruksi pemindahan barang ke *Fleet Controller*. Selama proses berlangsung, seluruh aktivitas dan hasil pemindahan dicatat sebagai *event* operasional diskrit yang merepresentasikan kondisi fisik di lapangan. Berdasarkan hasil *event* tersebut, sistem menentukan apakah pemindahan berhasil atau gagal, lalu memperbarui stok secara final atau mengembalikan stok ke kondisi semula sesuai hasil aktual.

Keunggulan utama alternatif ini terletak pada keseimbangan antara latensi rendah dan keandalan tinggi karena pembaruan stok hanya dilakukan setelah konfirmasi *event* dari lapangan. Selain itu, riwayat seluruh pergerakan barang tetap tersimpan sebagai catatan *event* yang dapat ditelusuri untuk kebutuhan audit dan analisis operasional, sehingga memenuhi persyaratan *traceability* sesuai standar ISO 9001:2015. Adapun kekurangan dari pendekatan ini adalah kompleksitas implementasi yang lebih tinggi karena membutuhkan mekanisme pengelolaan *event* yang terstruktur.

b. Alternatif 2: State-Driven Inventory Management

Pada pendekatan *State-Driven Inventory Management*, WMC beroperasi dengan menjadikan satu sistem terpusat sebagai satu-satunya pengelola *state* stok dan pergerakan barang. Ketika *order picking* atau *stocking* dibuat, sistem langsung mencatat *order* dan sekaligus memperbarui stok secara langsung di basis data tanpa mekanisme reservasi bertingkat. Setelah *order* tercatat, sistem mengirimkan instruksi ke *Fleet Controller* untuk melakukan pemindahan barang. Apabila pemindahan gagal, sistem akan menyesuaikan kembali kondisi stok berdasarkan hasil aktual di lapangan. Setelah proses pemindahan selesai, data inventori terbaru disinkronisasi ke seluruh lapisan sistem sebagai referensi kondisi terkini.

Keunggulan utama alternatif ini adalah alur sistem yang lebih sederhana dengan kompleksitas implementasi yang lebih rendah dibandingkan pendekatan *event-driven*,

serta latensi yang rendah karena pembaruan stok dilakukan secara langsung. Proses pemantauan dan penelusuran masalah juga menjadi lebih mudah karena seluruh data *state* terpusat di satu lokasi. Namun, kekurangan pendekatan ini adalah tingkat keandalan yang lebih rendah terhadap kegagalan parsial, karena jika terjadi kegagalan komunikasi antara WMC dan *Fleet Controller* di tengah proses, konsistensi antara *state* stok digital dan kondisi fisik barang tidak dapat dijamin secara otomatis. Selain itu, tidak adanya pencatatan *event* membuat keterlacakan historis operasional menjadi terbatas.

Tabel III.8 Perbandingan Karakteristik Alternatif Solusi Manajemen Data Inventori

Aspek	Alt. 1: Event-Driven	Alt. 2: State-Driven
Mekanisme Pembaruan Stok	Bertahap: reservasi → eksekusi → konfirmasi <i>event</i> → finalisasi stok	Langsung: pencatatan <i>order</i> → pembaruan stok instan → eksekusi
Ketahanan terhadap Kegagalan Parsial	Tinggi — stok hanya difinalisasi setelah konfirmasi hasil <i>event</i> dari lapangan	Sedang — risiko inkonsistensi stok jika kegagalan terjadi di tengah eksekusi
Keterlacakan (Traceability)	Komprehensif — seluruh pergerakan barang tercatat sebagai <i>event</i> historis yang dapat diaudit	Terbatas — hanya mencatat <i>state</i> akhir tanpa riwayat <i>event</i> operasional
Kompleksitas Implementasi	Lebih tinggi — membutuhkan mekanisme <i>event handler</i> , reservasi, dan konfirmasi bertingkat	Lebih rendah — alur sistem lebih sederhana dengan pembaruan langsung

lanjutan Tabel III.8

Aspek	Alt. 1: Event-Driven	Alt. 2: State-Driven
Skalabilitas	Tinggi — arsitektur <i>loosely coupled</i> mendukung penambahan layanan tanpa perubahan besar	Sedang — terpusat di satu <i>state manager</i> sehingga <i>scaling</i> lebih terbatas
Kemudahan Pemantauan	Sangat baik — setiap <i>event</i> tercatat sehingga diagnosis masalah dapat dilakukan secara mendetail	Mudah — seluruh data terpusat di satu lokasi sehingga pemantauan sederhana
Skor AHP (B100)	3,74	3,31

III.3.1.2 Analisis Penentuan Solusi

Penentuan solusi dilakukan dengan mengevaluasi kedua alternatif yang telah diidentifikasi pada Subbab III.3.1.1 menggunakan metode *Analytical Hierarchy Process* (AHP). Metode ini menghasilkan keputusan objektif dengan membandingkan setiap alternatif berdasarkan enam kriteria yang telah ditetapkan pada dokumen B100, yaitu *Reliability*, *Scalability*, *Cost of Production*, *Complexity of Production*, *Task Completion Speed*, dan Kemudahan Penggunaan. Bobot setiap kriteria ditetapkan melalui proses perbandingan berpasangan dan telah divalidasi dengan nilai *Consistency Ratio* (CR) yang memenuhi syarat konsistensi.

Bobot setiap kriteria yang digunakan sebagai dasar penilaian ditetapkan berdasarkan hasil perhitungan AHP pada dokumen B100. Nilai bobot mencerminkan tingkat kepentingan relatif setiap kriteria terhadap tujuan pemilihan solusi manajemen data inventori WMC, sebagaimana disajikan pada Tabel III.9.

Tabel III.9 Bobot Kriteria AHP Pemilihan Solusi Manajemen Data Inventori

Kriteria	Penjelasan	Bobot
Reliability	Ketahanan sistem bekerja secara konsisten dan akurat tanpa gangguan.	0,36

lanjutan Tabel III.9

Kriteria	Penjelasan	Bobot
Scalability	Kemampuan sistem untuk dikembangkan sesuai peningkatan kebutuhan.	0,25
Cost of Production	Total biaya yang diperlukan untuk membuat dan mengoperasikan sistem.	0,17
Complexity of Production	Tingkat kesulitan dalam perancangan, pembuatan, dan integrasi sistem.	0,11
Task Completion Speed	Kecepatan sistem dalam menyelesaikan tugas permintaan dan penaruhan barang.	0,07
Kemudahan Penggunaan	Seberapa mudah sistem dioperasikan tanpa pelatihan khusus.	0,05

Berdasarkan bobot kriteria yang telah ditetapkan, setiap alternatif dievaluasi dengan memberikan nilai pada masing-masing kriteria sesuai karakteristik arsitekturnya. Tabel III.10 menyajikan penilaian dan justifikasi untuk Alternatif 1: *Event-Driven Inventory Management*.

Tabel III.10 Penjelasan Penilaian Alternatif 1: Event-Driven Inventory Management

Kriteria	Nilai	Penjelasan
Reliability	4	Sistem memperbarui stok berdasarkan urutan kejadian (<i>event</i>) sehingga data inventori tetap konsisten meskipun terjadi kegagalan pada robot di lapangan.
Scalability	4	Arsitektur berbasis <i>event</i> memungkinkan sistem menangani peningkatan volume <i>order</i> tanpa perubahan besar pada komponen inti layanan.
Cost of Production	3	Biaya pengembangan relatif moderat dan berskala linear sesuai kebutuhan tiap gudang.
Complexity of Production	3	Kompleksitas muncul pada perancangan mekanisme transisi status stok dan pengelolaan <i>event handler</i> yang terstruktur.

lanjutan Tabel III.10

Kriteria	Nilai	Penjelasan
Task Completion Speed	3	Proses penyelesaian tugas memerlukan waktu tambahan karena setiap perubahan stok harus melalui validasi kejadian (<i>event confirmation</i>) terlebih dahulu.
Kemudahan Penggunaan	5	Pengguna lebih mudah mengoperasikan sistem karena seluruh data dan kontrol terpusat pada satu antarmuka terintegrasi.

Tabel III.11 menyajikan penilaian dan justifikasi untuk Alternatif 2: *State-Driven Inventory Management*.

Tabel III.11 Penjelasan Penilaian Alternatif 2: State-Driven Inventory Management

Kriteria	Nilai	Penjelasan
Reliability	3	Keandalan sistem bergantung pada satu pusat pengelolaan <i>state</i> sehingga operasi tetap stabil selama sistem utama berjalan, namun berisiko inkonsistensi stok jika terjadi kegagalan di tengah eksekusi.
Scalability	3	Peningkatan kapasitas sistem memerlukan penambahan sumber daya pada sistem pusat, sehingga <i>scaling</i> lebih terbatas dibandingkan pendekatan berbasis <i>event</i> .
Cost of Production	3	Biaya produksi berada pada tingkat menengah karena sistem terpusat membutuhkan infrastruktur yang cukup andal.
Complexity of Production	4	Kompleksitas pengembangan lebih rendah karena seluruh logika inventori dan pemin-dahan barang berada dalam satu sistem yang terpusat.
Task Completion Speed	4	Penyelesaian tugas berlangsung cepat karena keputusan dan pembaruan stok dilakukan langsung tanpa proses sinkronisasi <i>event</i> bertingkat.

lanjutan Tabel III.11

Kriteria	Nilai	Penjelasan
Kemudahan Penggunaan	5	Pengguna lebih mudah mengoperasikan sistem karena seluruh data dan kontrol terpusat pada satu antarmuka.

Setelah seluruh kriteria dinilai untuk kedua alternatif, skor akhir diperoleh dengan mengalikan nilai setiap kriteria dengan bobot yang telah ditetapkan, lalu dijumlahkan. Rekapitulasi perhitungan AHP disajikan pada Tabel III.12.

Tabel III.12 Rekapitulasi Penilaian AHP Solusi Manajemen Data Inventori (B100 Tabel 2.2.3.8)

Kriteria	Bobot	Alt. 1: Event-Driven		Alt. 2: State-Driven	
		Nilai	$\text{Bobot} \times \text{Nilai}$	Nilai	$\text{Bobot} \times \text{Nilai}$
Reliability	0,36	4	1,44	3	1,08
Scalability	0,25	4	1,00	3	0,75
Cost of Production	0,17	3	0,51	3	0,51
Complexity of Prod.	0,11	3	0,33	4	0,44
Task Completion Speed	0,07	3	0,21	4	0,28
Kemudahan Penggunaan	0,05	5	0,25	5	0,25
Total	1	—	3,74	—	3,31

Berdasarkan hasil perhitungan AHP pada Tabel III.12, Alternatif 1: *Event-Driven Inventory Management* memperoleh skor total 3,74, lebih tinggi dibandingkan Alternatif 2: *State-Driven Inventory Management* dengan skor 3,31. Keunggulan utama *Event-Driven* terletak pada aspek *Reliability* (bobot tertinggi 0,36) dan *Scalability* (0,25) yang secara agregat memberikan kontribusi skor lebih besar, mencerminkan kemampuan arsitektur berbasis *event* dalam menjaga konsistensi data stok meskipun terjadi kegagalan pada komponen fisik robot, serta fleksibilitasnya dalam menangani peningkatan volume transaksi tanpa membebani satu titik pusat sistem.

Dengan demikian, berdasarkan analisis AHP tersebut, *Event-Driven Inventory Management* dipilih sebagai arsitektur solusi yang diimplementasikan pada *Warehouse Management Controller* (WMC) dalam tugas akhir ini. Keputusan ini selaras

dengan kesimpulan pada dokumen B100 subbab 2.2.4, di mana *Event-Driven Inventory Management* ditetapkan sebagai solusi manajemen inventori terpilih karena kemampuannya menjaga konsistensi data stok melalui pencatatan setiap perubahan sebagai *event*, serta memudahkan proses audit dan keterlacakan operasional gudang.

III.3.2 Pemilihan Solusi Subsistem Lokalisasi

Selain solusi arsitektur WMC pada Subbab III.3.1.1, subbab ini menjabarkan pemilihan solusi teknis untuk subsistem lokalisasi menggunakan metode *Analytical Hierarchy Process* (AHP) yang sama, guna menjustifikasi pemilihan AprilTag dan kamera *fisheye overhead* dibandingkan alternatif lain.

III.3.2.1 Alternatif Solusi Lokalisasi

Sama seperti pada subbab sebelumnya, pemilihan solusi teknis untuk subsistem lokalisasi diawali dengan mengidentifikasi alternatif pendekatan lokalisasi yang dapat diterapkan. Berdasarkan kajian pada dokumen B100/B300 dan implementasi yang dilakukan pada tugas akhir ini, terdapat dua pendekatan yang dapat dipertimbangkan, yaitu *QR Code + Line Following* dan *AprilTag + Overhead Fisheye Camera*. Karakteristik masing-masing alternatif diuraikan sebagai berikut.

a. Alternatif 1: QR Code + Line Following

Pendekatan ini menempatkan seluruh perangkat lokalisasi pada masing-masing robot (*on-board*, desentralisasi) sebagaimana dirancang pada dokumen B100/B300. Robot mengikuti garis lintasan di lantai secara kontinu menggunakan *Line Detector Sensor* BFD-1000 (6 kanal IR) untuk memperoleh posisi relatif terhadap jalur, kemudian memindai *QR code* menggunakan GM65 *reader* pada titik-titik tertentu untuk memverifikasi posisi absolut dalam matriks *grid* 12×6 . Data ini digabung dengan pembacaan IMU dan diolah oleh ESP32-S3 di tiap robot untuk menghasilkan status posisi yang dikirim ke *Fleet Controller*.

Keunggulan utama alternatif ini adalah setiap robot mandiri melakukan *self-localization* sehingga tidak bergantung pada satu titik kegagalan sentral, dan infrastruktur jalur serta *QR code* relatif murah untuk dipasang di lantai gudang. Adapun kekurangannya, akurasi posisi antar-*checkpoint* bergantung pada kestabilan pembacaan sensor garis yang sensitif terhadap kondisi lantai (debu, aus, pencahayaan), setiap robot memerlukan satu set sensor lengkap sehingga biaya bertambah linear seiring penambahan robot, dan verifikasi posisi absolut hanya terjadi secara diskrit di titik *QR*, bukan kontinu di seluruh area.

b. Alternatif 2: AprilTag + Overhead Fisheye Camera

Pendekatan ini memindahkan seluruh proses lokalisasi ke satu titik sentral, yaitu kamera IP *fisheye* 1080p yang dipasang setinggi 60 cm di atas area lantai gudang ($2,4 \times 1,2$ m), melakukan *streaming* video melalui protokol RTSP, kemudian diproses oleh Subsistem Lokalisasi di server melalui *pipeline* koreksi distorsi Kannala–Brandt → transformasi *homography* → deteksi *marker* AprilTag Tag36h11 → *snap-ping* ke sel *grid*, yang berjalan pada frekuensi 4 Hz dan dipublikasikan ke *topic* MQTT *location* setiap 500 ms. Pendekatan inilah yang diimplementasikan pada tugas akhir ini.

Keunggulan utama alternatif ini adalah satu perangkat mampu mencakup seluruh area tanpa oklusi, akurasi terverifikasi 100% pada pengujian posisi maupun orientasi (Tabel VI.9–VI.10), kalibrasi *homography* hanya dilakukan sekali di awal pemasangan (*one-time*), dan penambahan robot baru cukup dilakukan dengan menempelkan *marker* AprilTag baru tanpa tambahan sensor per unit. Adapun kekurangannya, pendekatan ini bersifat *single point of failure* karena jika kamera atau koneksi RTSP mengalami gangguan maka seluruh sistem lokalisasi ikut lumpuh, area cakupan dibatasi oleh *field-of-view* satu kamera sehingga perluasan ke gudang yang lebih besar memerlukan penambahan kamera dan proses *stitching*, serta memerlukan tahap kalibrasi optik yang lebih matematis dibandingkan sensor garis.

Tabel III.13 Perbandingan Karakteristik Alternatif Solusi Lokalisasi

Aspek	Alt. 1: QR Code + Line Following	Alt. 2: AprilTag + Overhead Fisheye
Arsitektur Pemrosesan	Desentralisasi — tiap robot (ESP32-S3) mengolah posisinya sendiri	Sentralisasi — satu server mengolah seluruh objek dari satu kamera
Sumber Posisi Absolut	<i>QR code</i> , dibaca diskrit di titik tertentu	AprilTag, terdeteksi kontinu di setiap <i>frame</i> (4 Hz)
Cakupan Area	Seluruh lantai (jalur + titik QR tersebar)	Dibatasi <i>field-of-view</i> satu kamera per titik pemasangan
Akurasi Terukur	Tidak diuji langsung pada TA ini (target <i>repeatability</i> 100% di B200)	100% pada 6 kasus posisi & 4 kasus orientasi (Tabel VI.9–VI.10)
Latensi Publikasi	Target ≤ 1 detik (KNF-L01)	Terukur 500 ms

lanjutan Tabel III.13

Aspek	Alt. 1: QR Code + Line Following	Alt. 2: AprilTag + Overhead Fisheye
Penambahan Robot Baru	Perlu sensor lengkap (BFD-1000, GM65, IMU, ESP32-S3) per unit	Cukup <i>marker</i> AprilTag cetak per unit
Titik Kegagalan	Terdistribusi per robot	Terpusat pada satu kamera/server
Sensitivitas Lingkungan	Aus/kotor pada garis & <i>marker</i> lantai	Pencahayaannya & oklusi terhadap kamera

III.3.2.2 Kriteria Evaluasi

Berbeda dengan pemilihan solusi level WMC yang menggunakan kembali kriteria umum dari dokumen B100, pemilihan solusi lokalisasi memerlukan kriteria yang lebih spesifik terhadap karakteristik teknis lokalisasi robot. Kriteria evaluasi ditetapkan berdasarkan keterkaitannya dengan kebutuhan nonfungsional subsistem lokalisasi (KNF-L01–KNF-L03) pada Tabel III.7, mencakup aspek akurasi, latensi, skalabilitas, ketahanan operasional, biaya, dan kompleksitas kalibrasi, sebagaimana dirangkum pada Tabel III.14.

Tabel III.14 Kriteria Evaluasi Pemilihan Solusi Lokalisasi

Kriteria	Penjelasan
Akurasi Posisi	Ketepatan sistem memetakan posisi fisik robot ke koordinat <i>grid</i> dan orientasi (<i>heading</i>) sebenarnya.
Latensi Publikasi	Kecepatan sistem memperbarui status posisi ke server/UI sejak perubahan posisi fisik terjadi.
Skalabilitas ke Banyak Robot	Kemudahan dan biaya marginal menambah robot baru ke dalam sistem lokalisasi.
Ketahanan Operasional (Reliability)	Konsistensi akurasi sistem terhadap kondisi lingkungan (pencahayaannya, kotoran, aus) dan operasi berulang, terkait KNF-L02.

lanjutan Tabel III.14

Kriteria	Penjelasan
Biaya Implementasi	Total biaya perangkat keras yang dibutuhkan untuk mencakup seluruh area dan armada robot.
Kompleksitas Kalibrasi	Tingkat kesulitan proses instalasi dan kalibrasi awal sebelum sistem dapat digunakan.

III.3.2.3 Pairwise Comparison dan Perhitungan Bobot Kriteria

Penentuan bobot kriteria dilakukan melalui metode *Analytical Hierarchy Process* (AHP) dengan melakukan perbandingan berpasangan (*pairwise comparison*) antar keenam kriteria pada Tabel III.14. Urutan kepentingan kriteria ditetapkan berdasarkan keterkaitannya dengan kebutuhan nonfungsional inti subsistem lokalisasi: Akurasi Posisi dan Latensi Publikasi diprioritaskan karena berkaitan langsung dengan pemenuhan KNF-L01 dan KNF-L02, diikuti oleh Skalabilitas, Ketahanan Operasional, Biaya Implementasi, dan Kompleksitas Kalibrasi. Hasil perbandingan berpasangan menggunakan skala Saaty disajikan pada Tabel III.15 (kriteria disingkat berturut-turut sebagai AP, LP, SK, KO, BI, dan KK sesuai urutan pada Tabel III.14).

Tabel III.15 Nilai Kepentingan Kriteria

Kriteria	AP	LP	SK	KO	BI	KK
Akurasi Posisi (AP)	1	2	3	4	4	4
Latensi Publikasi (LP)	0,5	1	2	3	4	4
Skalabilitas (SK)	0,33	0,5	1	2	3	4
Ketahanan Operasional (KO)	0,25	0,33	0,5	1	2	3
Biaya Implementasi (BI)	0,25	0,25	0,33	0,5	1	2
Kompleksitas Kalibrasi (KK)	0,25	0,25	0,25	0,33	0,5	1

Bobot masing-masing kriteria dihitung dengan menormalisasi setiap kolom matriks terhadap jumlah kolomnya, kemudian merata-ratakan nilai pada setiap baris (metode *eigenvector* aproksimasi), sebagaimana dilakukan pada dokumen B100. Hasil perhitungan bobot disajikan pada Tabel III.16.

Tabel III.16 Nilai Bobot Kriteria

Kriteria	Bobot
Akurasi Posisi	0,36
Latensi Publikasi	0,25
Skalabilitas ke Banyak Robot	0,17
Ketahanan Operasional (Reliability)	0,11
Biaya Implementasi	0,07
Kompleksitas Kalibrasi	0,05
Total	1,00

Uji konsistensi terhadap matriks perbandingan berpasangan pada Tabel III.15 menghasilkan nilai *eigenvalue* maksimum (λ_{maks}) sebesar 6,226. *Consistency Index* (CI) dihitung dengan rumus $CI = (\lambda_{maks} - n)/(n - 1) = (6,226 - 6)/(6 - 1) = 0,045$. Dengan nilai *Random Index* (RI) untuk matriks berukuran $n = 6$ sebesar 1,24 (Saaty 1977), diperoleh *Consistency Ratio* $CR = CI/RI = 0,045/1,24 = 0,036$. Karena $CR = 0,036 < 0,1$, matriks perbandingan berpasangan pada Tabel III.15 dinyatakan konsisten sehingga bobot kriteria pada Tabel III.16 dapat digunakan sebagai dasar penilaian alternatif.

III.3.2.4 Penilaian Alternatif dan Penentuan Solusi Terpilih

Berdasarkan bobot kriteria yang telah ditetapkan pada Tabel III.16, setiap alternatif dievaluasi dengan memberikan nilai pada skala 1–5 (1 = sangat buruk, 5 = sangat baik) untuk masing-masing kriteria sesuai karakteristik teknisnya, mengikuti rubrik penilaian yang sama dengan dokumen B100. Tabel III.17 menyajikan penilaian dan justifikasi untuk Alternatif 1: QR Code + Line Following.

Tabel III.17 Penjelasan Penilaian Alternatif 1: QR Code + Line Following

Kriteria	Nilai	Penjelasan
Akurasi Posisi	4	Posisi relatif terjaga kontinu oleh sensor garis, namun akurasi absolut hanya terverifikasi diskrit di titik QR sehingga berpotensi <i>drift</i> antar- <i>checkpoint</i> .
Latensi Publikasi	3	Memenuhi target ≤ 1 detik (KNF-L01) namun bergantung pada rantai komunikasi robot→server yang lebih panjang dibanding sistem tersentralisasi.

lanjutan Tabel III.17

Kriteria	Nilai	Penjelasan
Skalabilitas ke Banyak Robot	4	Bersifat desentralisasi — penambahan robot tidak membebani satu titik pemrosesan, cocok untuk perluasan area tanpa infrastruktur kamera tambahan.
Ketahanan Operasional	3	Rentan terhadap ausnya <i>marker</i> /garis lantai dan perubahan pencahayaan lokal di sepanjang jalur.
Biaya Implementasi	3	Biaya bertambah linear karena setiap robot membutuhkan satu set sensor lengkap (BFD-1000, GM65, IMU, ESP32-S3).
Kompleksitas Kalibrasi	4	Instalasi relatif sederhana — garis dan QR ditempel di lantai tanpa perhitungan optik yang kompleks.

Tabel III.18 menyajikan penilaian dan justifikasi untuk Alternatif 2: AprilTag + Overhead Fisheye Camera.

Tabel III.18 Penjelasan Penilaian Alternatif 2: AprilTag + Overhead Fisheye Camera

Kriteria	Nilai	Penjelasan
Akurasi Posisi	5	Terverifikasi akurasi 100% pada 6 kasus posisi dan 4 kasus orientasi (Tabel VI.9–VI.10), dengan deteksi kontinu 4 Hz di seluruh area.
Latensi Publikasi	5	Terukur konsisten 500 ms, jauh di bawah target 1 detik (KNF-L01).
Skalabilitas ke Banyak Robot	3	Penambahan robot cukup dengan <i>marker</i> cetak, namun cakupan dibatasi <i>field-of-view</i> satu kamera sehingga perluasan area memerlukan kamera tambahan.

lanjutan Tabel III.18

Kriteria	Nilai	Penjelasan
Ketahanan Operasional	4	Tidak mengalami degradasi fisik seperti aus lantai; pencahayaan LED tambahan menjaga kontras <i>marker</i> , meski tetap berisiko oklusi.
Biaya Implementasi	4	Biaya marjinal per robot sangat rendah (hanya <i>marker</i> cetak); biaya utama berupa satu kamera per zona cakupan.
Kompleksitas Kalibrasi	3	Memerlukan kalibrasi <i>homography</i> dan koreksi distorsi <i>fish-eye</i> di awal pemasangan, meski hanya dilakukan sekali (<i>one-time</i>).

Setelah seluruh kriteria dinilai untuk kedua alternatif, skor akhir diperoleh dengan mengalikan nilai setiap kriteria dengan bobot yang telah ditetapkan pada Tabel III.16, lalu dijumlahkan. Rekapitulasi perhitungan AHP disajikan pada Tabel III.19.

Tabel III.19 Rekapitulasi Penilaian AHP Solusi Lokalisasi

Kriteria	Bobot	Alt. 1: QR Code + Line		Alt. 2: AprilTag + Overhead	
		Nilai	$\text{Bobot} \times \text{Nilai}$	Nilai	$\text{Bobot} \times \text{Nilai}$
Akurasi Posisi	0,36	4	1,44	5	1,80
Latensi Publikasi	0,25	3	0,75	5	1,25
Skalabilitas ke Banyak Robot	0,17	4	0,68	3	0,51
Ketahanan Operasional	0,11	3	0,33	4	0,44
Biaya Implementasi	0,07	3	0,21	4	0,28
Kompleksitas Kalibrasi	0,05	4	0,20	3	0,15

lanjutan Tabel III.19

Kriteria	Bobot	Alt. 1: QR Code + Line		Alt. 2: AprilTag + Overhead	
		Nilai	Bobot $\times$ Nilai	Nilai	Bobot $\times$ Nilai
Total	1	—	3,61	—	4,43

Berdasarkan hasil perhitungan AHP pada Tabel III.19, Alternatif 2: AprilTag + Overhead Fisheye Camera memperoleh skor total 4,43, lebih tinggi dibandingkan Alternatif 1: QR Code + Line Following dengan skor 3,61. Keunggulan utama AprilTag + Overhead Fisheye Camera terletak pada aspek Akurasi Posisi (bobot tertinggi 0,36) dan Latensi Publikasi (0,25) yang bersama menyumbang 3,05 dari 4,43 total skor, selaras dengan hasil pengujian aktual pada Tabel VI.9–VI.10 (akurasi 100%, latensi 500 ms). Alternatif QR Code + Line Following unggul pada kriteria Skalabilitas ke Banyak Robot dan Kompleksitas Kalibrasi, namun kedua kriteria tersebut berbobot relatif rendah (0,17 dan 0,05) sehingga tidak mengubah hasil akhir penentuan solusi.

Dengan demikian, berdasarkan analisis AHP tersebut, AprilTag + Kamera *Fisheye Overhead* dipilih sebagai solusi teknis Subsistem Lokalisasi yang diimplementasikan pada tugas akhir ini, menggantikan pendekatan QR Code + *Line Following* yang dirancang pada dokumen B100/B300. Keputusan ini konsisten dengan implementasi Subsistem Lokalisasi pada Bab IV Subbab 1.4 dan Bab V Subbab 3, serta hasil pengujian pada Bab VI Subbab 2.6 yang membuktikan pemenuhan seluruh kebutuhan nonfungsional lokalisasi.

III.3.2.5 Justifikasi Solusi Terpilih

Kaitan hasil AHP dengan pemenuhan kebutuhan nonfungsional Subsistem Lokalisasi (Subbab III.2.3) diuraikan sebagai berikut. Pertama, KNF-L01 (latensi status posisi ≤ 1 detik) terpenuhi dengan latensi publikasi terukur 500 ms melalui *topic MQTT location*, jauh di bawah ambang batas yang disyaratkan, sejalan dengan skor tertinggi AprilTag + Overhead Fisheye Camera pada kriteria Latensi Publikasi (nilai 5). Kedua, KNF-L02 (tidak ada degradasi akurasi posisi/*Task Repeatability* 100%) terpenuhi dengan hasil verifikasi 100% pada 6 kasus posisi dan 4 kasus orientasi (Tabel VI.9–VI.10), sejalan dengan skor tertinggi pada kriteria Akurasi Posisi (nilai 5). Ketiga, KNF-L03 (deteksi kegagalan pelacakan dan notifikasi koordinat terakhir) didukung oleh arsitektur tersentralisasi yang memudahkan pemantauan status koneksi kamera dan validitas *payload* MQTT secara terpusat, sebagaimana dijelaskan pada mekanisme ketahanan operasional pada Tabel III.17–III.18.

Dengan terpenuhinya seluruh kebutuhan nonfungsional tersebut serta unggulnya skor AHP pada kriteria berbobot tertinggi, pemilihan AprilTag + Overhead Fisheye Camera sebagai solusi Subsistem Lokalisasi dapat dijustifikasi secara objektif dan dapat dipertanggungjawabkan secara teknis, sekaligus menunjukkan bahwa pengembangan pada tugas akhir ini memberikan penyempurnaan terhadap rancangan awal lokalisasi pada dokumen B100/B300.

BAB IV

PERANCANGAN

IV.1 Konsep Sistem

Sebelum mengembangkan sistem *Smart Warehouse*, dilakukan analisis kesenjangan (*gap analysis*) untuk mengidentifikasi permasalahan pada operasional gudang konvensional saat ini (kondisi *As-Is*) dan merumuskan target perbaikannya pada rancangan sistem usulan (kondisi *To-Be*).

Pada kondisi *As-Is*, operasional pergudangan untuk industri *Fast Moving Consumer Goods* (FMCG) seperti Paragon Corp masih bersifat semi-manual dan sangat bergantung pada operator manusia dalam proses *stocking*, *picking*, serta *checking*. Berdasarkan analisis proses bisnis, kondisi saat ini memunculkan celah operasional berupa data stok yang tidak *real-time* karena belum tersinkronisasinya WMS dengan sistem pusat, proses *inbound* yang lambat dan rawan *error*, serta kelambatan dalam *picking* barang yang berdampak pada meningkatnya *lead time* distribusi logistik.

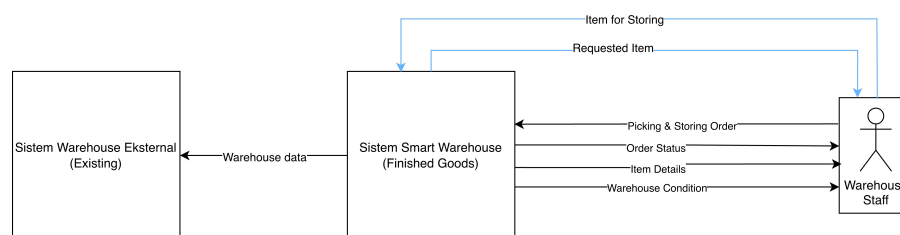
Untuk menyelesaikan berbagai celah operasional tersebut, diusulkan kondisi *To-Be* berupa sistem *Smart Warehouse* yang sepenuhnya terotomasi. Transformasi pada kondisi *To-Be* dirancang dengan cakupan sebagai berikut:

- **Integrasi Sistem *Real-Time*:** Sistem inventori terintegrasi secara langsung antara *warehouse* barang jadi dengan sistem pusat sehingga pencatatan data stok selalu tersinkronisasi dan aktual.
- **Deteksi dan Pelacakan Otomatis:** Proses pendataan, pelacakan lokasi, dan pemantauan pergerakan barang dilakukan secara otomatis dengan tingkat presisi yang tinggi untuk meminimalisasi *human error*.
- **Otomasi Perpindahan Barang:** Menggantikan proses manual dengan sistem perpindahan barang otomatis (menggunakan robot) untuk seluruh tahapan operasional—baik *storing* maupun *picking*—guna menjamin kecepatan dan keakuratan pemenuhan pesanan.

IV.2 Gambaran Umum Sistem

Sistem *Smart Warehouse* yang dikembangkan pada penelitian ini dirancang sebagai sebuah *Proof of Concept* (PoC) untuk memvalidasi fungsionalitas, kinerja, dan integrasi dari seluruh subsistem otomasi pergudangan sebelum diimplementasikan pada lingkungan industri yang sesungguhnya. Implementasi fisik dari sistem ini dilaksanakan pada *layout* yang diskalakan $\pm 2,4 \times 1,2$ m, robot dengan ukuran $\pm 15 \times 15$ cm, dan rak barang dengan ukuran $\pm 20 \times 20$ cm sebagai representasi proses gudang yang sama seperti skala besar.

Selain itu, perangkat lunak yang digunakan memiliki kesamaan fungsi dengan rancangan skala industri, sehingga yang diuji pada PoC mencerminkan perilaku sistem penuh. Melalui pendekatan pengujian berskala ini, seluruh proses operasional mulai dari manajemen pesanan, sinkronisasi inventori, pemetaan lokasi, hingga pengendalian pergerakan armada robot dapat dievaluasi secara komprehensif dan akurat selayaknya operasional gudang pada tahap *Industry Scale*.



Gambar IV.1 Konsep Sistem *Smart Warehouse*

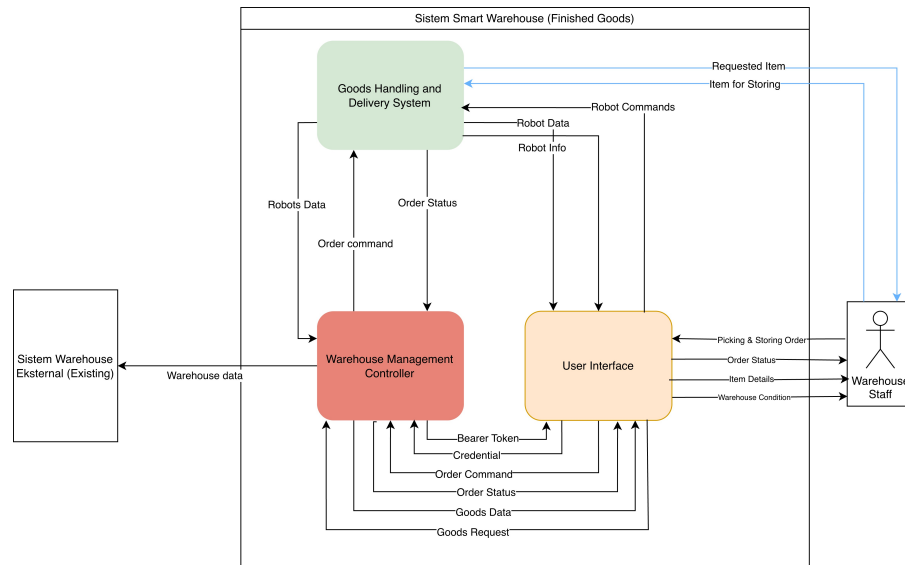
IV.3 Diagram Arsitektur

Subbab ini menjabarkan arsitektur sistem *Smart Warehouse* secara bertingkat, dari gambaran sistem keseluruhan pada Level 0 hingga dekomposisi komponen perangkat lunak yang menyusun *Warehouse Management Controller* (WMC) pada Level 3. Fokus pembahasan diarahkan pada WMC sebagai subsistem perangkat lunak yang dikembangkan, beserta komponen-komponen yang secara langsung berinteraksi dengannya, termasuk arsitektur *Fleet Controller* sebagai konteks integrasi dan komponen *Localisation System* yang menjadi kontribusi individu penulis di dalamnya.

IV.3.1 Arsitektur Level 0

Arsitektur Level 0 menggambarkan sistem *Smart Warehouse* secara menyeluruh sebagai satu kesatuan modul tunggal. Pada level ini, sistem dipandang sebagai kotak hitam (*black box*) yang menerima berbagai masukan dari lingkungan eksternal dan menghasilkan keluaran berupa eksekusi fisik pemindahan barang serta laporan status operasional. Sistem ini berinteraksi langsung dengan staf gudang (*Warehouse*

Staff) dan sistem logistik eksternal. Gambar IV.2 mengilustrasikan arsitektur sistem pada Level 0.



Gambar IV.2 Arsitektur Sistem *Smart Warehouse* Level 0

Tabel IV.1 Fungsi, masukan, dan keluaran sistem *Smart Warehouse* (Level 0)

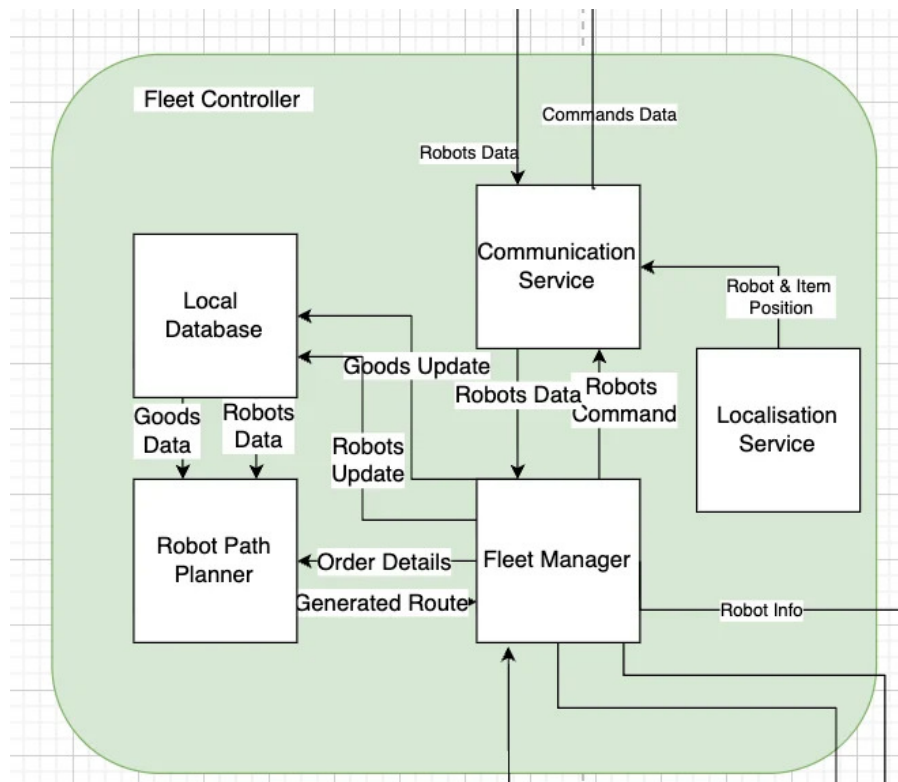
Elemen	Deskripsi
Modul	Sistem <i>Smart Warehouse (Finished Goods)</i>
Fungsi	Melakukan interaksi perintah <i>picking</i> dan <i>storing</i> serta data yang dikirimkan dari <i>Warehouse Staff</i> , serta melakukan integrasi data gudang dengan sistem yang sudah ada.
Masukan	1. <i>Warehouse data</i> — informasi stok, lokasi penyimpanan, dan status barang dari sistem eksternal. 2. <i>Requested item</i> — permintaan pengambilan barang dari <i>Warehouse Staff</i> . 3. <i>Item for storing</i> — barang fisik yang akan disimpan beserta data ID barang. 4. <i>Warehouse condition report</i> — kapasitas ruang, posisi rak, dan kondisi area penyimpanan.
Luaran	1. <i>Order status</i> — informasi keberhasilan proses ke operator dan sistem. 2. <i>Robots data</i> — status operasi robot, progres navigasi, dan hasil tugas. 3. <i>Goods delivery</i> — barang fisik yang dikirim ke sistem logistik. 4. <i>Logistic documentation</i> — laporan pelepasan barang dan dokumen <i>tracking</i> .

IV.3.2 Subsistem *Warehouse Management Controller* (WMC)

IV.3.2.1 Arsitektur Level 2 — WMC

Pada Level 2, sistem *Smart Warehouse* didekomposisi menjadi dua subsistem utama, yaitu *Goods Handling and Delivery System* (GHDS) yang menangani aktivitas fisik robot, dan *Warehouse Management Controller* (WMC) yang menjadi fokus pengembangan pada tugas akhir ini. WMC berfungsi sebagai pusat kendali digital

yang mengelola permintaan barang, mengoordinasikan pengendalian robot, serta memperbarui status pesanan secara terintegrasi. WMC berinteraksi langsung dengan antarmuka pengguna, basis data, dan *Fleet Controller* pada GHDS. Gambar IV.3 mengilustrasikan arsitektur WMC pada Level 2.



Gambar IV.3 Arsitektur *Warehouse Management Controller* (Level 2)

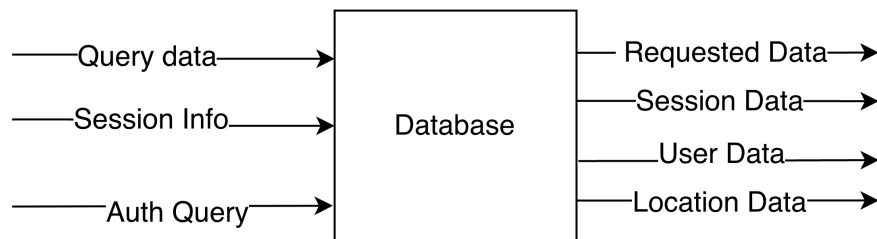
IV.3.2.2 Arsitektur Level 3 — Komponen WMC

Pada Level 3, WMC didekomposisi menjadi empat komponen perangkat lunak yang masing-masing memiliki tanggung jawab yang terdefinisi. Keempat komponen tersebut adalah *Database*, *Goods Detail Service*, *Fleet Order Service*, dan *Authentication Service*. Setiap komponen saling terhubung dan beroperasi dalam arsitektur *microservices* yang memungkinkan setiap layanan dikembangkan dan di-deploy secara independen (lihat Subbab II tentang arsitektur *microservices*, khususnya Gambar II.1).

IV.3.2.2.1 Database Komponen *Database* merupakan lapisan penyimpanan data terpusat pada WMC. Komponen ini menyimpan seluruh data operasional gudang, mencakup detail barang, *layout* penyimpanan, data pengguna, sesi autentikasi, dan status pengiriman robot. Seluruh layanan WMC lainnya berinteraksi dengan komponen ini melalui mekanisme *query* yang terstruktur. Gambar IV.4 mengilustrasikan arsitektur komponen *Database*.

Tabel IV.2 Fungsi, masukan, dan keluaran *Warehouse Management Controller* (Level 2)

Elemen	Deskripsi
Fungsi	Mengelola dan mengkoordinasikan proses permintaan barang, pengendalian robot, serta pembaruan status pesanan di gudang secara terintegrasi.
Masukan	1. Data robot — informasi kondisi, posisi, dan status operasional robot. 2. Status pesanan — informasi perkembangan pesanan yang sedang diproses. 3. Permintaan barang — permintaan pengambilan atau penyediaan barang dari gudang. 4. Perintah pesanan — instruksi untuk memproses atau mengeksekusi pesanan tertentu. 5. Kredensial — data autentikasi untuk memverifikasi akses pengguna.
Luaran	1. Barang yang diminta — informasi atau perintah terkait barang yang harus diambil atau disiapkan. 2. Informasi & data robot — kondisi dan data operasional robot yang telah diperbarui. 3. Status pesanan — status terbaru pesanan setelah diproses oleh sistem. 4. <i>Bearer Token</i> — token autentikasi untuk mengamankan komunikasi antar layanan.



Gambar IV.4 Arsitektur Komponen *Database* WMC (Level 3)

Tabel IV.3 Fungsi, masukan, dan keluaran *Database* WMC

Elemen	Deskripsi
Fungsi	Menyimpan dan menyediakan data keseluruhan operasional gudang, mencakup detail barang, <i>layout</i> penyimpanan, status pengiriman robot, data pengguna, dan sesi autentikasi.
Masukan	1. <i>Query Data Goods</i> — permintaan baca/ubah/hapus data barang dari <i>Goods Detail Service</i> . 2. <i>Auth Query / Session Data</i> — diterima dari <i>Authentication Service</i> untuk validasi pengguna dan penyimpanan sesi. 3. <i>Location Query & Order Data</i> — diterima dari <i>Fleet Order Service</i> untuk mendapatkan lokasi barang dan memperbarui detail barang.
Luaran	1. <i>Goods Data</i> — dikirim ke <i>Goods Detail Service</i> sebagai hasil <i>query</i> data barang. 2. <i>User Data / Session Info</i> — dikirim ke <i>Authentication Service</i> . 3. <i>Location Data / Updated Goods Detail</i> — dikirim ke <i>Fleet Order Service</i> .

Sebagai kelanjutan dari perancangan komponen *Database*, subbab ini turut menjabarkan skema basis data sistem yang dimodelkan menggunakan *Entity Relationship Diagram* (ERD) berikut.

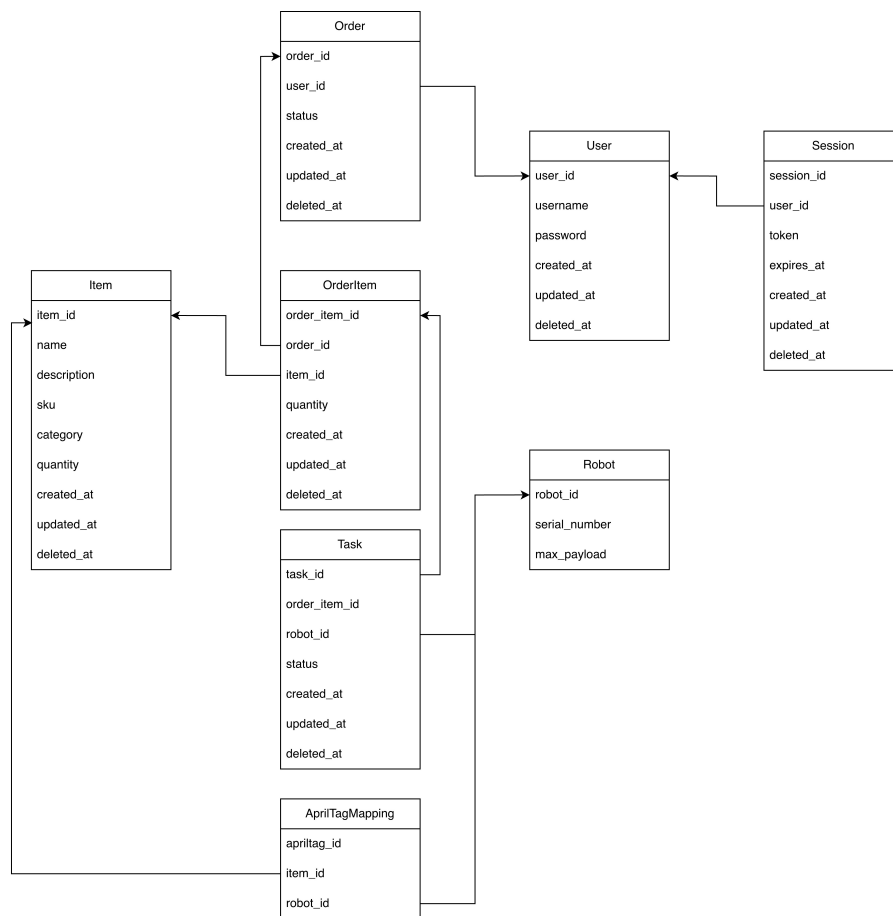
Entity Relationship Diagram (ERD) adalah diagram yang menggambarkan struktur data dalam suatu sistem informasi yang berguna untuk mengetahui hubungan antar entitas, atribut dari masing-masing entitas, dan apa saja entitas yang berperan di dalam penyimpanan data (Silberschatz, Korth, dan Sudarshan 2020). ERD merupakan bagian penting dalam proses perancangan sistem, khususnya dalam merancang skema basis data relasional yang terstruktur dan konsisten.

Pada tugas akhir ini, ERD digunakan untuk memodelkan entitas-entitas utama yang terlibat dalam sistem *Warehouse Management Controller*, yaitu *User*, *Session*, *Order*, *OrderItem*, *Item*, *ItemLocation*, *Task*, *Robot*, dan *RobotLog*, beserta seluruh relasi yang menghubungkan entitas-entitas tersebut. Entitas-entitas ini saling terhubung secara relasional sesuai dengan pendekatan basis data SQL yang diterapkan pada sistem.

Berikut ini merupakan *Entity Relationship Diagram* yang digunakan dalam pengembangan sistem *Warehouse Management Controller*.

Berdasarkan ERD pada Gambar IV.5, terdapat sembilan entitas yang dirancang dalam basis data sistem. Entitas *User* menyimpan data pengguna sistem beserta kredensial autentikasi, sedangkan entitas *Session* mencatat informasi sesi aktif pengguna termasuk token dan masa berlakunya. Entitas *Order* merepresentasikan setiap permintaan *picking* maupun *storing* yang dibuat oleh pengguna, dan terhubung dengan entitas *OrderItem* yang memuat rincian barang dalam setiap *order* beserta kuantitasnya. Entitas *Item* menjadi master data barang yang tersimpan di gudang, dilengkapi dengan entitas *ItemLocation* untuk menyimpan informasi posisi fisik barang berupa koordinat rak dalam area gudang. Entitas *Task* menghubungkan *OrderItem* dengan *Robot*, merepresentasikan instruksi operasional yang diberikan kepada robot untuk dieksekusi. Entitas *Robot* menyimpan data identitas dan kapasitas robot, sementara entitas *RobotLog* mencatat seluruh aktivitas dan kejadian pada setiap robot sebagai mekanisme *logging* dan pelacakan kesalahan sistem.

IV.3.2.2.2 Goods Detail Service Komponen *Goods Detail Service* bertanggung jawab mengelola data master seluruh barang yang tersimpan di gudang. Komponen ini menjadi antarmuka antara *Database* dan komponen lain yang memerlukan informasi barang, mencakup penyimpanan, pembaruan, dan pengambilan informasi

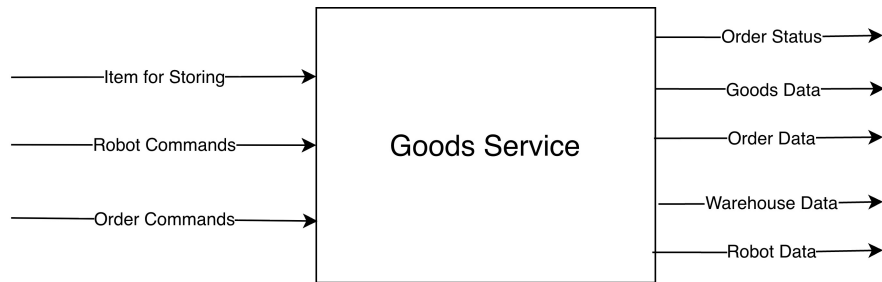


Gambar IV.5 ERD Sistem *Warehouse Management Controller*

Tabel IV.4 Penjelasan entitas basis data *Warehouse Management Controller*

Entitas	Deskripsi
<i>User</i>	Menyimpan data pengguna sistem yang merupakan staf gudang yang bertugas mengelola dan memantau seluruh operasional <i>Warehouse Management Controller</i> . Struktur data dari entitas <i>User</i> mencakup informasi identitas pengguna seperti nama, <i>username</i> , dan <i>password</i> yang digunakan untuk keperluan autentikasi, serta <i>timestamp</i> pembuatan, pembaruan, dan penghapusan data. Entitas <i>User</i> terhubung dengan dua entitas lain dalam sistem. Relasi dengan entitas <i>Order</i> memungkinkan sistem melacak seluruh permintaan <i>picking</i> dan <i>storing</i> yang dibuat oleh masing-masing pengguna, sedangkan relasi dengan entitas <i>Session</i> digunakan untuk mengelola sesi login aktif pengguna berdasarkan token autentikasi yang diterbitkan setelah proses login berhasil dilakukan.
<i>Order</i>	Menyimpan data permintaan operasional gudang yang diajukan oleh pengguna, baik berupa proses <i>picking</i> (pengambilan barang) maupun <i>storing</i> (penyimpanan barang). Struktur data dari entitas <i>Order</i> mencakup referensi pengguna yang mengajukan permintaan, status terkini dari permintaan tersebut, serta <i>timestamp</i> pembuatan, pembaruan, dan penghapusan data. Entitas <i>Order</i> terhubung dengan dua entitas lain dalam sistem. Relasi dengan entitas <i>User</i> mengikat setiap permintaan kepada pengguna yang membuatnya sehingga sistem dapat melakukan pelacakan dan audit aktivitas secara terstruktur, sedangkan relasi dengan entitas <i>OrderItem</i> memungkinkan satu permintaan mencakup beberapa barang sekaligus dengan rincian kuantitas masing-masing.
<i>OrderItem</i>	Menyimpan data rincian barang yang terdapat dalam suatu permintaan operasional gudang. Struktur data dari entitas <i>OrderItem</i> mencakup referensi ke permintaan induk, referensi ke barang yang diminta, jumlah kuantitas yang diperlukan, serta <i>timestamp</i> pembuatan, pembaruan, dan penghapusan data. Entitas <i>OrderItem</i> merupakan entitas penghubung yang memiliki relasi dengan tiga entitas sekaligus dalam sistem. Relasi dengan entitas <i>Order</i> mengaitkan setiap detail barang ke permintaan asalnya, relasi dengan entitas <i>Item</i> menentukan barang spesifik yang menjadi objek permintaan, sedangkan relasi dengan entitas <i>Task</i> menjadi dasar pembentukan instruksi operasional yang akan diberikan kepada robot untuk dieksekusi.
<i>Item</i>	Menyimpan master data seluruh barang yang dikelola di dalam gudang. Struktur data dari entitas <i>Item</i> mencakup informasi identitas barang seperti nama, deskripsi, <i>stock-keeping unit</i> (SKU), kategori, dan jumlah stok yang tersedia, serta <i>timestamp</i> pembuatan, pembaruan, dan penghapusan data. Entitas <i>Item</i> terhubung dengan dua entitas lain dalam sistem. Relasi dengan entitas <i>OrderItem</i> memungkinkan sistem merujuk data barang secara akurat ketika memproses permintaan operasional, sedangkan relasi dengan entitas <i>ItemLocation</i> digunakan untuk menyimpan informasi posisi fisik barang di dalam gudang berupa koordinat rak sehingga robot dapat menavigasi ke lokasi yang tepat.
<i>Task</i>	Menyimpan data instruksi operasional yang diberikan kepada robot sebagai tindak lanjut dari permintaan <i>picking</i> atau <i>storing</i> yang masuk ke sistem. Struktur data dari entitas <i>Task</i> mencakup

secara terperinci. Gambar IV.6 mengilustrasikan arsitektur komponen ini.

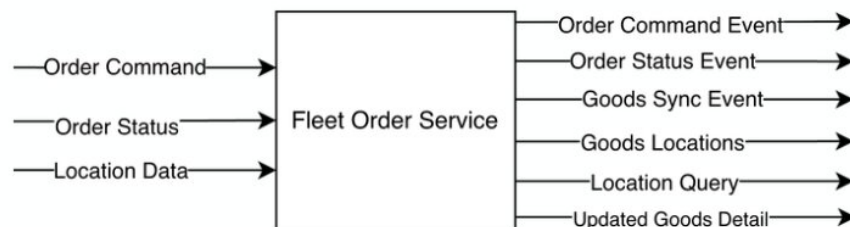


Gambar IV.6 Arsitektur Komponen *Goods Detail Service* WMC (Level 3)

Tabel IV.5 Fungsi, masukan, dan keluaran *Goods Detail Service* WMC

Elemen	Deskripsi
Fungsi	Mengelola data master seluruh barang di gudang, mencakup penyimpanan, pembaruan, dan pengambilan informasi barang secara terperinci.
Masukan	1. <i>Goods Data</i> — objek data berisi informasi lengkap atau pembaruan detail satu atau lebih item barang. 2. <i>Query Data</i> — permintaan spesifik untuk mengambil, menghapus, atau memverifikasi detail barang.
Luaran	1. <i>Goods Data</i> — data barang sebagai respons atas permintaan <i>Query Data</i> . 2. <i>Query Data</i> — respons konfirmasi bahwa permintaan telah dieksekusi.

IV.3.2.2.3 Fleet Order Service Komponen *Fleet Order Service* merupakan *gateway* utama antara sistem WMC dan komponen *Fleet Controller* pada robot. Komponen ini mengolah seluruh logika pembuatan dan pembatalan *order*, pengaturan tugas robot, serta pemantauan dan sinkronisasi status tugas secara *real-time*. Gambar IV.7 mengilustrasikan arsitektur komponen ini.



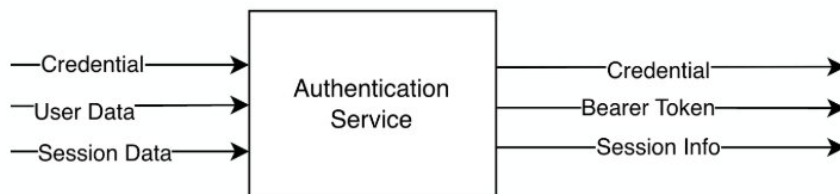
Gambar IV.7 Arsitektur Komponen *Fleet Order Service* WMC (Level 3)

IV.3.2.2.4 Authentication Service Komponen *Authentication Service* bertanggung jawab memvalidasi identitas pengguna yang mengakses sistem WMC. Kom-

Tabel IV.6 Fungsi, masukan, dan keluaran *Fleet Order Service* WMC

Elemen	Deskripsi
Fungsi	Mengolah seluruh data dan logika pembuatan <i>order</i> , pengaturan tugas robot, pemantauan status robot, serta sinkronisasi status tugas. Menjadi <i>gateway</i> antara sistem internal WMC dan <i>Fleet Controller</i> .
Masukan	1. <i>Order Command</i> — instruksi pembuatan atau pembatalan <i>order</i> dari antarmuka pengguna. 2. <i>Robot Status & Task Status</i> — informasi posisi, baterai, <i>error</i> , dan progres tugas dari <i>Fleet Controller</i> . 3. <i>Order Data / Location Data</i> — diterima dari <i>Database</i> untuk memproses dan memperbarui catatan <i>order</i> .
Luaran	1. <i>Order Status Event</i> — dikirim ke antarmuka pengguna sebagai respons status <i>order</i> terkini. 2. <i>Command</i> ke <i>Fleet Controller</i> — instruksi robot berupa <i>start task</i> , <i>cancel task</i> , atau <i>emergency stop</i> . 3. <i>Updated Goods Detail</i> — dikirim ke <i>Database</i> setelah <i>order</i> diproses.

ponen ini menerima kredensial dari antarmuka pengguna, memverifikasinya terhadap data pada *Database*, kemudian menerbitkan *Bearer Token* sebagai bukti otorisasi untuk seluruh permintaan berikutnya. Gambar IV.8 mengilustrasikan arsitektur komponen ini.



Gambar IV.8 Arsitektur Komponen *Authentication Service* WMC (Level 3)

IV.3.3 Subsistem *Fleet Controller*

IV.3.3.1 Arsitektur Level 2 — *Fleet Controller*

Selain *Warehouse Management Controller*, subsistem *Fleet Controller* turut menjadi bagian penting dari arsitektur *Smart Warehouse* karena berperan sebagai penghubung langsung antara WMC dan armada robot fisik pada *Goods Handling and Delivery System* (GHDS). Sebagaimana dijelaskan pada dokumen B300, *Fleet Controller* bertanggung jawab mengelola penugasan robot, manajemen status, dan perencanaan rute pergerakan armada, dengan seluruh data status robot diterima melalui subsistem *Communication* pada GHDS. Pengembangan *Fleet Controller* secara keseluruhan berada di luar cakupan tugas akhir individu ini, kecuali komponen *Localisation System* yang menjadi kontribusi khusus penulis (lihat Subbab I.4). Pemaparan arsitektur *Fleet Controller* pada subbab ini diperlukan untuk memberikan konteks

Tabel IV.7 Fungsi, masukan, dan keluaran *Authentication Service* WMC

Elemen	Deskripsi
Fungsi	Memvalidasi identitas pengguna berdasarkan kredensial yang diberikan, menghasilkan token autentikasi (<i>Bearer Token</i>), serta mengelola sesi pengguna.
Masukan	1. <i>Credential</i> Pengguna — data <i>username</i> dan <i>password</i> diterima dari antarmuka pengguna via API HTTP/HTTPS. 2. <i>Session Info / User Data</i> — hasil pencarian pengguna dari <i>Database</i> . 3. <i>Token / Session ID</i> (opsional) — diterima dari antarmuka pengguna untuk validasi sesi yang masih aktif.
Luaran	1. <i>Bearer Token</i> — dikirim ke antarmuka pengguna sebagai <i>string</i> token untuk otorisasi permintaan berikutnya. 2. <i>Session Data</i> — dikirim ke <i>Database</i> untuk disimpan atau diperbarui. 3. Informasi Penolakan Akses — pesan kesalahan dikirim ke antarmuka pengguna bila kredensial tidak valid atau sesi telah kedaluwarsa.

interaksi antara WMC dan *Fleet Controller* melalui *Fleet Order Service* (lihat Subbab IV.3.2.2.3).

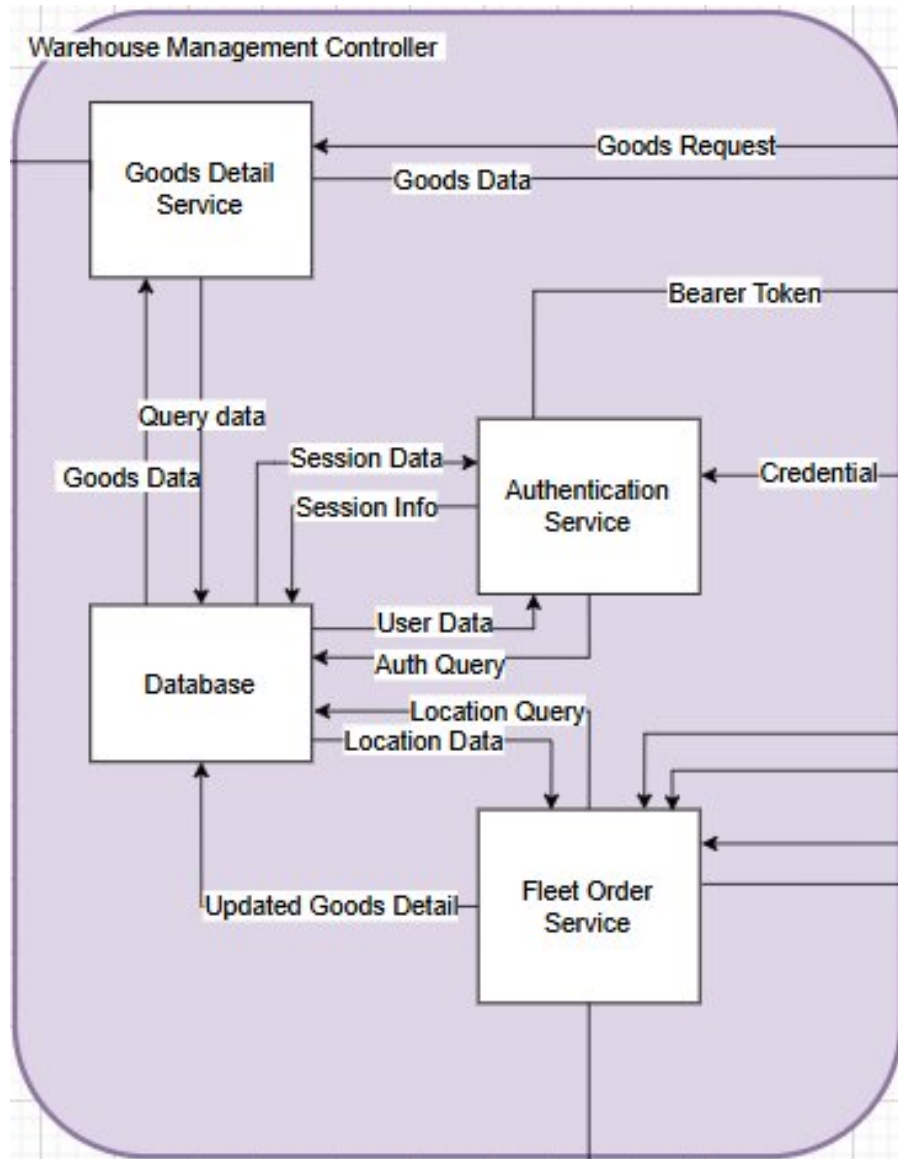
Pada Level 2, *Fleet Controller* dipandang sebagai satu modul tunggal yang menerima instruksi *order* dan data robot dari WMC, kemudian menghasilkan perintah eksekusi bagi robot serta status terkini yang dikembalikan ke WMC. Gambar IV.9 berikut mengilustrasikan arsitektur *Fleet Controller* pada Level 2.

Tabel IV.8 Fungsi, masukan, dan keluaran *Fleet Controller* (Level 2)

Elemen	Deskripsi
Fungsi	Mengelola penugasan robot, manajemen status, dan rute pergerakan seluruh armada robot berdasarkan instruksi <i>order</i> yang diterima dari WMC.
Masukan	1. <i>Robots Data</i> — informasi status, posisi, dan kondisi seluruh robot yang beroperasi di gudang. 2. <i>Order Commands</i> — instruksi pembuatan, pembaruan, atau pembatalan tugas robot, diterima dari <i>Fleet Order Service</i> WMC.
Luaran	1. <i>Order Status</i> — status pelaksanaan tugas dikirim ke <i>Fleet Order Service</i> WMC. 2. <i>Robot Commands</i> — perintah kendali dikirim ke robot melalui subsistem <i>Communication</i> pada GHDS. 3. <i>Robots Data</i> — data status robot terbaru diteruskan ke WMC untuk ditampilkan pada <i>User Interface</i> .

IV.3.3.2 Arsitektur Level 3 — Komponen *Fleet Controller*

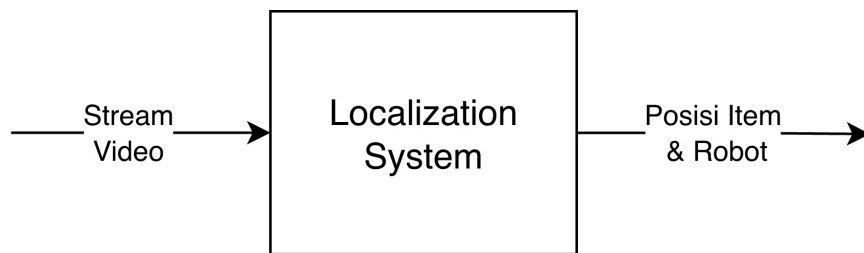
Pada Level 3, *Fleet Controller* didekomposisi menjadi lima komponen perangkat lunak, yaitu *Communication Service*, *Fleet Manager*, *Local Database*, *Robot Path Planner*, dan *Localisation System*. Keempat komponen pertama merupakan hasil



Gambar IV.9 Arsitektur *Fleet Controller* (Level 2)

rancangan kelompok TA252601001 sebagaimana didokumentasikan pada dokumen B300, sedangkan komponen *Localisation System* merupakan kontribusi individu penulis yang menggantikan pendekatan lokalisasi berbasis QR Code + *Line Following* pada rancangan awal dengan pendekatan AprilTag + *Overhead Fisheye Camera* (lihat Subbab I.4).

IV.3.3.2.1 Localisation System *Localisation System* merupakan komponen *Fleet Controller* yang menjadi fokus kontribusi individu penulis pada tugas akhir ini, berjalan secara paralel dengan subsistem *Navigation* pada robot. Berbeda dengan *Navigation* yang menentukan posisi robot dari perspektif internal robot melalui pembacaan kode QR di lantai dan sensor IMU, *Localisation System* menyediakan perspektif eksternal dari atas (*overhead*) yang mencakup seluruh area lantai gudang sekaligus, termasuk posisi robot, barang, dan *docking station*. Gambar IV.10 mengilustrasikan arsitektur komponen ini; rincian perancangan perangkat dan *pipeline* deteksinya dijabarkan lebih lanjut pada Subbab IV.3.3.3.



Gambar IV.10 Arsitektur Komponen *Localisation System Fleet Controller* (Level 3)

Tabel IV.9 Fungsi, masukan, dan keluaran *Localisation System*

Elemen	Deskripsi
Fungsi	Mendeteksi posisi <i>grid</i> dan orientasi seluruh robot, barang, serta <i>docking station</i> secara terpusat dari sudut pandang <i>overhead</i> menggunakan AprilTag <i>fiducial marker</i> , kemudian mempublikasikan hasil deteksi sebagai data <i>grid state</i> kepada komponen lain <i>Fleet Controller</i> .
Masukan	1. <i>Video Stream</i> — umpan video RTSP dari kamera IP <i>fisheye overhead</i> . 2. Parameter Kalibrasi — koefisien distorsi lensa (Kannala-Brandt) dan matriks <i>homography</i> hasil kalibrasi satu kali di awal pemasangan.
Luaran	1. <i>Grid State (Location Data)</i> — data posisi <i>grid</i> dan orientasi seluruh entitas terklasifikasi, dipublikasikan setiap 500 ms melalui <i>topic MQTT location</i> , dikonsumsi oleh <i>Fleet Manager</i> dan diteruskan ke <i>Fleet Order Service WMC</i> untuk ditampilkan <i>real-time</i> pada <i>User Interface</i> .

IV.3.3.3 Desain Detail Subsistem Lokalisasi (*Localisation System*)

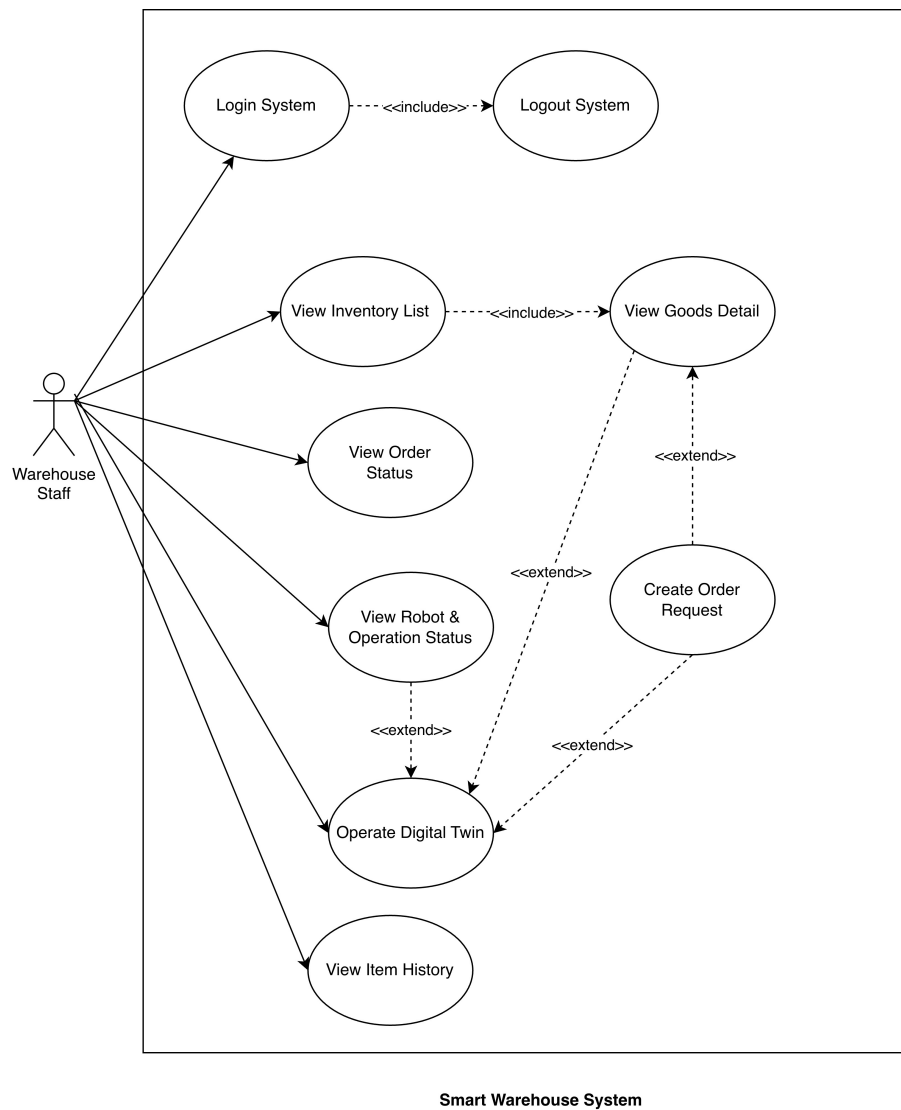
Subsistem Lokalisasi merupakan komponen perangkat lunak yang berada di dalam *Fleet Controller*, berjalan secara paralel dengan *Navigation subsystem* pada robot. Jika *Navigation subsystem* menentukan posisi robot dari perspektif internal robot melalui pembacaan kode QR di lantai dan sensor IMU, Subsistem Lokalisasi menyediakan perspektif eksternal dari atas (*overhead*) yang mencakup seluruh area lantai gudang sekaligus—termasuk posisi robot, barang, dan *docking station*. *Output* subsistem ini berupa data *grid state* yang dipublikasikan melalui MQTT ke *topic location* pada RabbitMQ broker, dikonsumsi oleh *Fleet Manager* untuk keperluan *digital twin* koordinasi robot, serta oleh *Fleet Order Service* di WMC untuk diteruskan ke *User Interface* secara *real-time*.

IV.3.3.3.1 Perangkat Lokalisasi Perangkat lokalisasi adalah rakitan kamera *overhead* yang dipasang setinggi 60 cm di atas lantai gudang berukuran $2,4 \times 1,2$ m. Rakitan ini terdiri dari rangka penyangga, sebuah kamera IP *fisheye* berkemampuan 1080p (3 MP) dengan *H.264 encoding* yang melakukan *streaming* melalui protokol RTSP over TCP, serta pencahayaan LED tambahan untuk memastikan iluminasi yang seragam di seluruh area lantai. Optik *fisheye* dipilih karena kemampuannya mencakup seluruh area lantai gudang dari satu titik pemasangan tunggal tanpa oklusi, meskipun memerlukan proses koreksi distorsi sebelum dapat digunakan untuk pemetaan posisi yang akurat (lihat Subbab 2.5.2).

IV.3.3.3.2 Desain Pipeline Deteksi Subsistem Lokalisasi didesain sebagai *pipeline* pemrosesan *frame* yang berjalan secara kontinu. Sebuah *background thread* mengambil *frame* terbaru dari *stream* RTSP secara terus-menerus dan membuang *frame* yang sudah basi, sementara *detection loop* utama berjalan pada frekuensi 4 Hz. Setiap *frame* diproses melalui enam tahap secara berurutan, sebagaimana dirangkum pada Tabel IV.10 berikut.

IV.4 Diagram Use Case

Kebutuhan fungsional sistem mendefinisikan kapabilitas spesifik yang harus tersedia bagi pengguna dalam berinteraksi dengan sistem *Warehouse Management Controller*. Kebutuhan ini diturunkan langsung dari Tabel III.4 dengan perincian yang lebih granular, sehingga setiap kebutuhan merepresentasikan satu fungsi yang dapat diuji secara mandiri. Berikut ini adalah kebutuhan fungsional yang diidentifikasi untuk mendukung pengembangan sistem *Warehouse Management Controller* sebagai perangkat lunak pengelola operasional gudang berbasis robot.



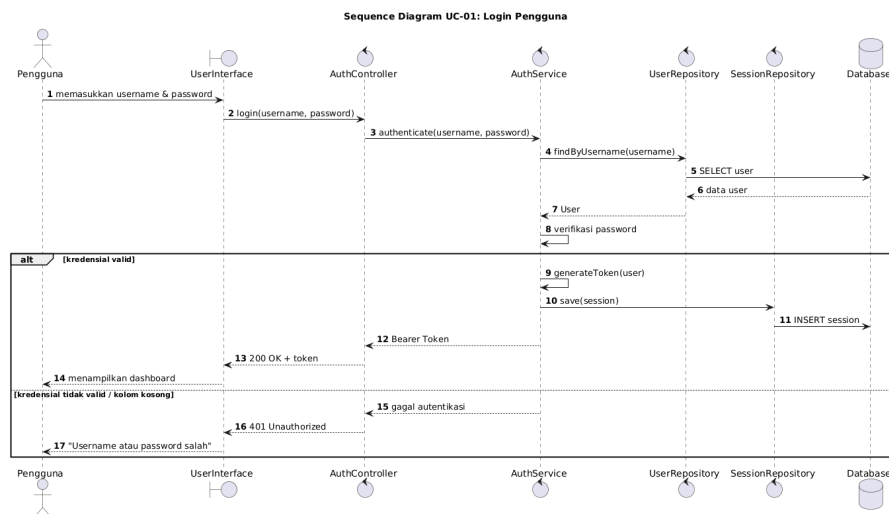
Gambar IV.11 *Use Case Diagram Sistem Warehouse Management Controller*

Tabel IV.10 *Pipeline* pemrosesan Subsistem Lokalisasi

No.	Tahap	Deskripsi
1	Koreksi Distorsi Lensa	Menghilangkan distorsi radial <i>fisheye</i> menggunakan koefisien kalibrasi Kannala–Brandt yang dihitung saat <i>setup</i> .
2	Pra-pemrosesan	Meningkatkan kontras <i>frame</i> yang telah terkoraksi dan mengonversinya ke <i>grayscale</i> untuk detektor.
3	Deteksi AprilTag	Menjalankan detektor Tag36h11 (<i>marker</i> fisik 8×8 cm) untuk mengekstrak ID, posisi piksel pusat, dan empat koordinat sudut setiap <i>tag</i> .
4	Pemetaan Grid	Mentransformasi koordinat piksel ke sel <i>grid</i> diskrit menggunakan <i>homography</i> H dan aturan <i>nearest-cell-centre</i> (lihat Subbab 2.5.3).
5	Klasifikasi dan Heading	Mengklasifikasikan setiap <i>tag</i> ID ke salah satu dari tiga kategori: <i>docking station</i> , robot, atau <i>warehouse item</i> . Orientasi (<i>heading</i> 0° – 359°) diperoleh dari sudut sisi depan <i>tag</i> .
6	Publikasi MQTT	Mengodekan seluruh <i>grid occupancy state</i> sebagai <i>payload</i> JSON dan mempublikasikannya setiap 500 ms ke <i>topic location</i> pada RabbitMQ <i>broker</i> .

Skenario *use case* mendeskripsikan interaksi antara pengguna dan sistem *Warehouse Management Controller* (WMC) untuk setiap kebutuhan fungsional. Setiap skenario mencakup kondisi awal, kondisi akhir, langkah-langkah interaksi normal, serta skenario alternatif apabila terjadi kondisi di luar alur utama.

IV.4.1 UC-01: Login System



Gambar IV.12 *Sequence Diagram* UC-01: Login System

Tabel IV.11 Deskripsi *Use Case* UC-01: *Login System*

Elemen	Keterangan
ID <i>Use Case</i>	UC-01
Aktor Utama	Staf Gudang (<i>Warehouse Staff</i>)
Referensi SR	FR01–FR05 (Bab III.4)
Kondisi Awal	(1) Pengguna telah memiliki akun terdaftar. (2) Pengguna memiliki akses ke aplikasi <i>cross-platform</i> .
Kondisi Akhir	Pengguna berhasil masuk ke sistem dan dapat mengakses seluruh fitur sesuai hak akses.
Deskripsi	<i>Use case</i> ini menjelaskan proses autentikasi staf gudang yang telah memiliki akun untuk mengakses fitur sistem <i>Smart Warehouse</i> .

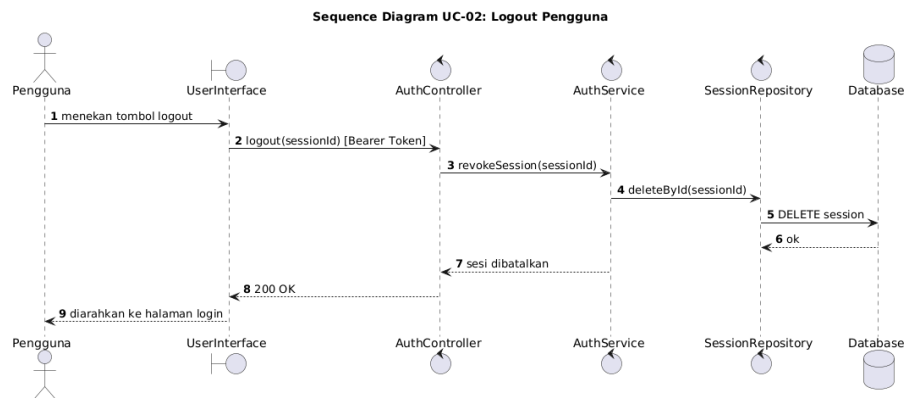
Tabel IV.12 Skenario *Use Case* UC-01: *Login System*

Aktor	Sistem
Skenario Utama	
1. Pengguna mengakses halaman masuk.	
	2. Sistem menampilkan formulir <i>username</i> dan kata sandi.
3. Pengguna memasukkan kredensial dan menekan tombol masuk.	
	4. Sistem memverifikasi kredensial ke basis data.
	5. Sistem membuat sesi pengguna aktif dan menghasilkan <i>bearer token</i> .
	6. Sistem menampilkan <i>dashboard</i> utama.
Skenario Alternatif	
4a. Kredensial yang dimasukkan tidak valid.	Sistem menampilkan pesan kesalahan dan meminta pengguna mencoba kembali.

Tabel IV.13 Deskripsi *Use Case UC-02: Logout System*

Elemen	Keterangan
ID <i>Use Case</i>	UC-02
Aktor Utama	Staf Gudang (<i>Warehouse Staff</i>)
Referensi SR	FR01–FR05 (Bab III.4)
Kondisi Awal	Pengguna sedang dalam keadaan masuk (<i>logged in</i>).
Kondisi Akhir	Sesi pengguna berakhir dan akses ke fitur sistem ditutup.
Deskripsi	<i>Use case</i> ini menjelaskan proses pengakhiran sesi pengguna yang sedang aktif.

IV.4.2 UC-02: *Logout System*



Gambar IV.13 *Sequence Diagram UC-02: Logout System*

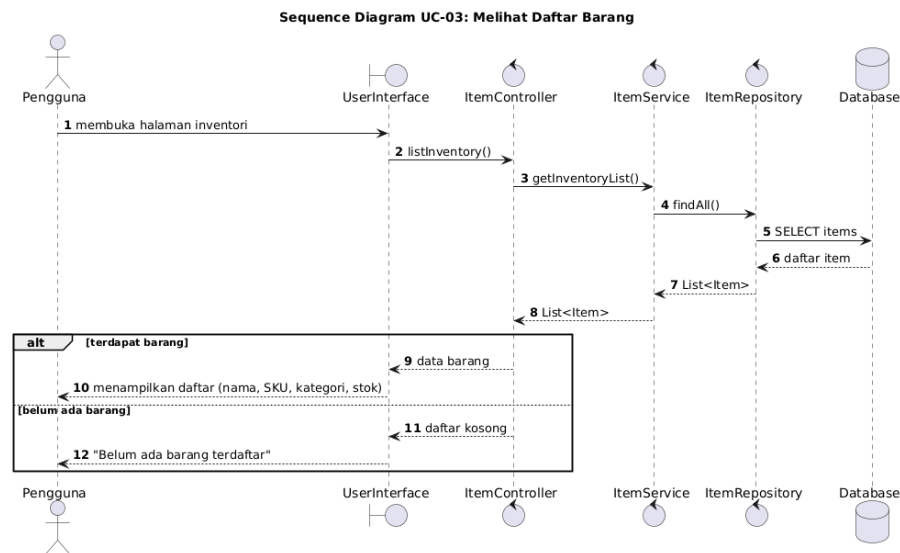
Tabel IV.14 Skenario *Use Case UC-02: Logout System*

Aktor	Sistem
Skenario Utama	
1. Pengguna memilih opsi keluar.	
	2. Sistem membatalkan sesi aktif berdasarkan <i>session ID</i> .
	3. Sistem menampilkan kembali halaman masuk.
Skenario Alternatif	
Tidak terdapat skenario alternatif yang dirumuskan untuk <i>use case</i> ini pada dokumen sumber.	

Tabel IV.15 Deskripsi *Use Case* UC-03: *View Inventory List*

Elemen	Keterangan
ID <i>Use Case</i>	UC-03
Aktor Utama	Staf Gudang (<i>Warehouse Staff</i>)
Referensi SR	FR01–FR05 (Bab III.4)
Kondisi Awal	Pengguna telah masuk ke sistem.
Kondisi Akhir	Pengguna memperoleh gambaran kondisi stok gudang secara terpusat.
Deskripsi	<i>Use case</i> ini memungkinkan staf gudang melihat daftar seluruh barang yang tercatat dalam inventori.

IV.4.3 UC-03: *View Inventory List*



Gambar IV.14 *Sequence Diagram* UC-03: *View Inventory List*

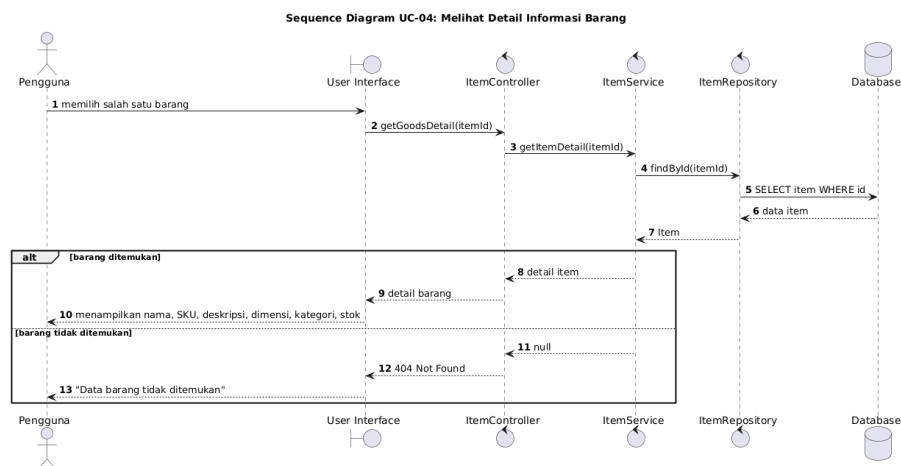
Tabel IV.16 Skenario *Use Case* UC-03: *View Inventory List*

Aktor	Sistem
Skenario Utama	
1. Pengguna membuka halaman inventori.	
	2. Sistem mengambil data barang dari <i>goods detail service</i> .
	3. Sistem menampilkan daftar barang beserta kuantitas dan kategori.
Skenario Alternatif	
Tidak terdapat skenario alternatif yang dirumuskan untuk <i>use case</i> ini pada dokumen sumber.	

IV.4.4 UC-04: *View Goods Detail*

Tabel IV.17 Deskripsi *Use Case* UC-04: *View Goods Detail*

Elemen	Keterangan
ID <i>Use Case</i>	UC-04
Aktor Utama	Staf Gudang (<i>Warehouse Staff</i>)
Referensi SR	FR01–FR05 (Bab III.4)
Kondisi Awal	Pengguna sedang melihat daftar inventori (UC-03).
Kondisi Akhir	Pengguna memperoleh informasi lengkap barang yang dipilih.
Deskripsi	<i>Use case</i> ini menampilkan informasi rinci suatu barang termasuk SKU, deskripsi, kuantitas, dan lokasi penyimpanan.



Gambar IV.15 *Sequence Diagram* UC-04: *View Goods Detail*

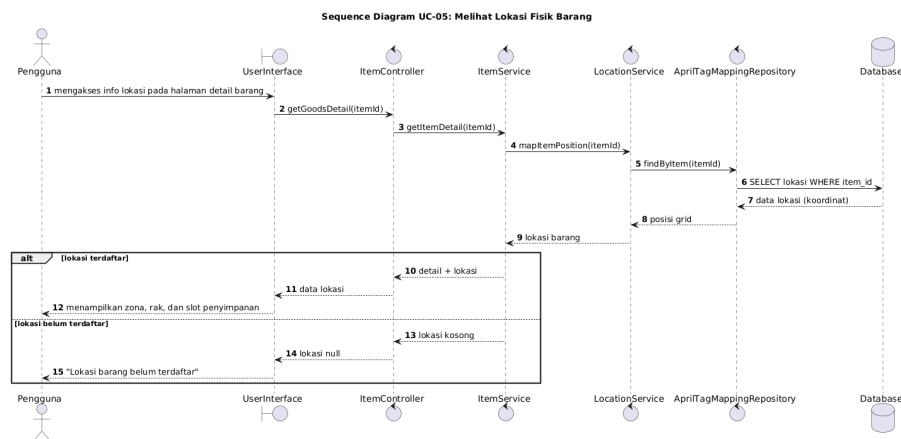
Tabel IV.18 Skenario *Use Case* UC-04: *View Goods Detail*

Aktor	Sistem
Skenario Utama	
1. Pengguna memilih barang tertentu.	
	2. Sistem mengirim permintaan detail barang ke <i>goods detail service</i> .
	3. Sistem menampilkan halaman detail barang.
Skenario Alternatif	
Tidak terdapat skenario alternatif yang dirumuskan untuk <i>use case</i> ini pada dokumen sumber.	

Tabel IV.19 Deskripsi *Use Case* UC-05: *View Order Status*

Elemen	Keterangan
ID <i>Use Case</i>	UC-05
Aktor Utama	Staf Gudang (<i>Warehouse Staff</i>)
Referensi SR	FR01–FR05 (Bab III.4)
Kondisi Awal	Pengguna telah masuk ke sistem.
Kondisi Akhir	Pengguna dapat memantau progres setiap permintaan penyimpanan atau pengambilan barang.
Deskripsi	<i>Use case</i> ini memungkinkan staf gudang memantau status <i>order inbound</i> maupun <i>outbound</i> .

IV.4.5 UC-05: *View Order Status*



Gambar IV.16 *Sequence Diagram* UC-05: *View Order Status*

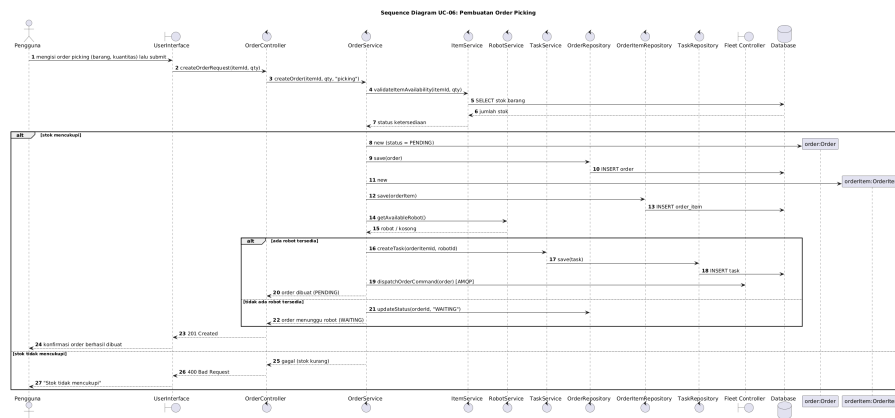
Tabel IV.20 Skenario *Use Case* UC-05: *View Order Status*

Aktor	Sistem
Skenario Utama	
1. Pengguna membuka halaman status <i>order</i> .	
	2. Sistem mengambil data <i>order</i> dari <i>fleet order service</i> .
	3. Sistem menampilkan daftar <i>order</i> beserta status terkininya (<i>Pending</i> , <i>InProgress</i> , <i>Completed</i> , atau <i>Failed</i>).
Skenario Alternatif	
Tidak terdapat skenario alternatif yang dirumuskan untuk <i>use case</i> ini pada dokumen sumber.	

Tabel IV.21 Deskripsi *Use Case* UC-06: *View Robot & Operation Status*

Elemen	Keterangan
ID <i>Use Case</i>	UC-06
Aktor Utama	Staf Gudang (<i>Warehouse Staff</i>)
Referensi SR	FR01–FR05 (Bab III.4)
Kondisi Awal	Pengguna telah masuk ke sistem.
Kondisi Akhir	Pengguna memperoleh visibilitas operasional armada robot.
Deskripsi	<i>Use case</i> ini menampilkan status armada AGV termasuk posisi, baterai, dan tugas aktif.

IV.4.6 UC-06: *View Robot & Operation Status*



Gambar IV.17 *Sequence Diagram* UC-06: *View Robot & Operation Status*

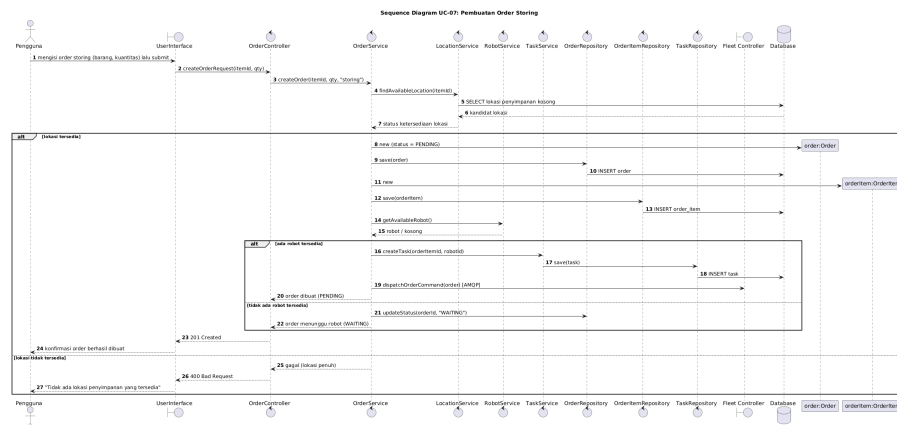
Tabel IV.22 Skenario *Use Case* UC-06: *View Robot & Operation Status*

Aktor	Sistem
Skenario Utama	
1. Pengguna membuka halaman status robot.	
	2. Sistem berlangganan <i>telemetry stream</i> melalui <i>WebSocket</i> .
	3. Sistem menampilkan panel status armada AGV secara <i>real-time</i> .
Skenario Alternatif	
Tidak terdapat skenario alternatif yang dirumuskan untuk <i>use case</i> ini pada dokumen sumber.	

Tabel IV.23 Deskripsi Use Case UC-07: Operate Digital Twin

Elemen	Keterangan
ID Use Case	UC-07
Aktor Utama	Staf Gudang (<i>Warehouse Staff</i>)
Referensi SR	FR01–FR05 (Bab III.4)
Kondisi Awal	Pengguna telah masuk ke sistem; modul <i>Digital Twin</i> tersedia.
Kondisi Akhir	Pengguna memperoleh kesadaran spasial (<i>spatial situational awareness</i>) terhadap kondisi operasional gudang.
Deskripsi	Use case ini memungkinkan staf gudang berinteraksi dengan representasi virtual 3D kondisi gudang dan robot.

IV.4.7 UC-07: Operate Digital Twin



Gambar IV.18 Sequence Diagram UC-07: Operate Digital Twin

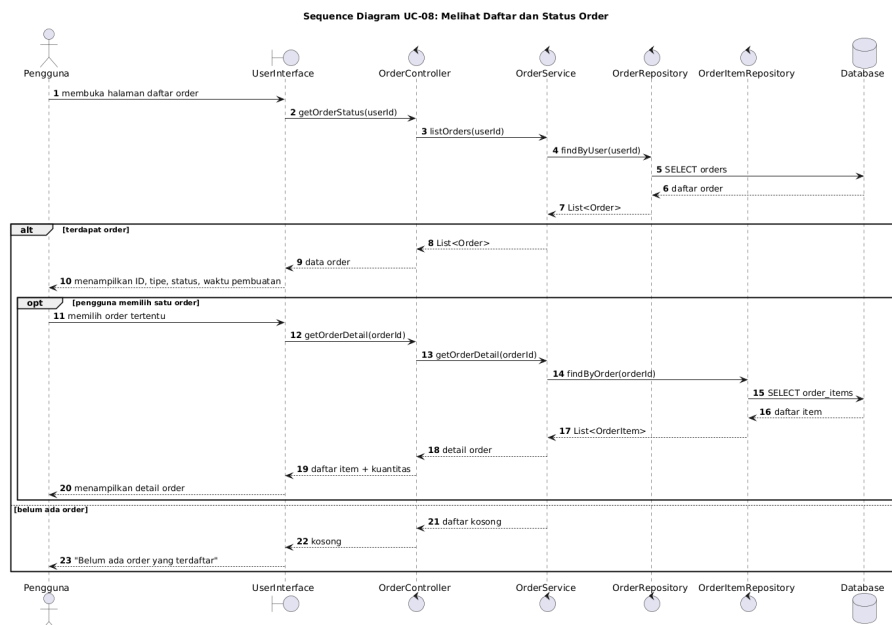
Tabel IV.24 Skenario Use Case UC-07: Operate Digital Twin

Aktor	Sistem
Skenario Utama	
1. Pengguna membuka tampilan <i>Digital Twin</i> .	
	2. Sistem memuat model 3D gudang dan berlangganan koordinat robot.
3. Pengguna memilih robot atau barang pada tampilan 3D.	
	4. Sistem menampilkan detail entitas yang dipilih.
Skenario Alternatif	
Tidak terdapat skenario alternatif yang dirumuskan untuk use case ini pada dokumen sumber.	

Tabel IV.25 Deskripsi *Use Case* UC-08: *Create Order Request*

Elemen	Keterangan
ID <i>Use Case</i>	UC-08
Aktor Utama	Staf Gudang (<i>Warehouse Staff</i>)
Referensi SR	FR01–FR05 (Bab III.4)
Kondisi Awal	Pengguna telah masuk; data barang tersedia di inventori.
Kondisi Akhir	<i>Order</i> baru terdaftar dan siap dieksekusi oleh armada robot.
Deskripsi	<i>Use case</i> ini menjelaskan proses pembuatan permintaan <i>storing</i> atau <i>picking</i> barang.

IV.4.8 UC-08: *Create Order Request*



Gambar IV.19 *Sequence Diagram* UC-08: *Create Order Request*

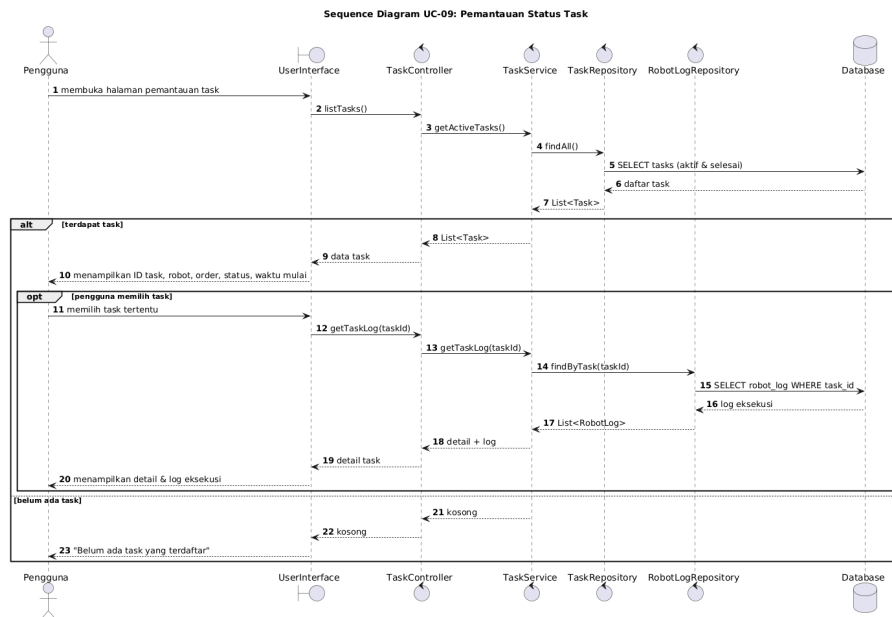
Tabel IV.26 Skenario *Use Case* UC-08: *Create Order Request*

Aktor	Sistem
Skenario Utama	
1. Pengguna mengisi formulir <i>order</i> (barang dan kuantitas).	
	2. Sistem memvalidasi ketersediaan barang.
3. Pengguna menekan tombol kirim.	
	4. Sistem menyimpan <i>order</i> dan <i>order item</i> ke basis data.
	5. Sistem mengirim perintah ke <i>fleet controller</i> melalui AMQP.
	6. Sistem menampilkan konfirmasi pembuatan <i>order</i> .
Skenario Alternatif	
Tidak terdapat skenario alternatif yang dirumuskan untuk <i>use case</i> ini pada dokumen sumber.	

Tabel IV.27 Deskripsi *Use Case* UC-09: *View Item History*

Elemen	Keterangan
ID <i>Use Case</i>	UC-09
Aktor Utama	Staf Gudang (<i>Warehouse Staff</i>)
Referensi SR	FR01–FR05 (Bab III.4)
Kondisi Awal	Pengguna telah masuk ke sistem.
Kondisi Akhir	Pengguna dapat menelusuri histori pergerakan barang untuk kebutuhan audit operasional.
Deskripsi	<i>Use case</i> ini menampilkan riwayat aktivitas suatu barang termasuk perpindahan dan perubahan kuantitas.

IV.4.9 UC-09: View Item History



Gambar IV.20 Sequence Diagram UC-09: View Item History

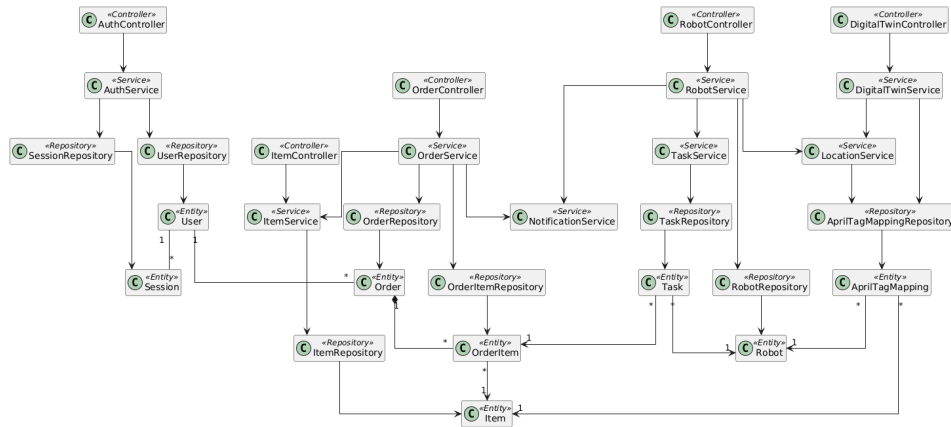
Tabel IV.28 Skenario Use Case UC-09: View Item History

Aktor	Sistem
Skenario Utama	
1. Pengguna memilih barang untuk dilihat riwayatnya.	
	2. Sistem mengambil log aktivitas dari <i>goods detail service</i> .
	3. Sistem menampilkan kronologi aktivitas <i>inbound/outbound</i> .
Skenario Alternatif	
Tidak terdapat skenario alternatif yang dirumuskan untuk <i>use case</i> ini pada dokumen sumber.	

IV.5 Class Diagram

Subbab ini menyajikan *Class Diagram* yang memodelkan struktur statis dari layanan *backend* WMC. Diagram ini mengidentifikasi dan menggambarkan kelas-kelas utama penyusun sistem, yaitu *Controller*, *Service*, *Repository*, dan *Entity*, beserta relasi dan ketergantungan antar kelas tersebut dalam mendukung seluruh logika operasional pengelolaan gudang.

Berdasarkan *Class Diagram* pada Gambar IV.21, rancangan perangkat lunak *Warehouse Management Controller* mengikuti pola arsitektur *microservice* berlapis yang



Gambar IV.21 Class Diagram Sistem Warehouse Management Controller

memisahkan tanggung jawab ke dalam empat jenis kelas: *Controller* sebagai titik masuk permintaan dari antarmuka, *Service* sebagai pengampu logika bisnis, *Repository* sebagai lapisan akses persistensi data, dan *Entity* sebagai representasi data domain.

Tabel IV.29 Daftar kelas pada arsitektur *Warehouse Management Controller*

ID	Nama Kelas	Tipe Kelas
C-01	AuthController	<i>Controller</i>
C-02	ItemController	<i>Controller</i>
C-03	OrderController	<i>Controller</i>
C-04	RobotController	<i>Controller</i>
C-05	DigitalTwinController	<i>Controller</i>
C-06	AuthService	<i>Service</i>
C-07	ItemService	<i>Service</i>
C-08	OrderService	<i>Service</i>
C-09	RobotService	<i>Service</i>
C-10	TaskService	<i>Service</i>
C-11	DigitalTwinService	<i>Service</i>
C-12	LocationService	<i>Service</i>
C-13	NotificationService	<i>Service</i>
C-14	UserRepository	<i>Repository</i>
C-15	SessionRepository	<i>Repository</i>
C-16	ItemRepository	<i>Repository</i>
C-17	OrderRepository	<i>Repository</i>
C-18	OrderItemRepository	<i>Repository</i>
C-19	RobotRepository	<i>Repository</i>
C-20	TaskRepository	<i>Repository</i>
C-21	AprilTagMappingRepository	<i>Repository</i>
C-22	User	<i>Entity</i>
C-23	Session	<i>Entity</i>
C-24	Item	<i>Entity</i>
C-25	Order	<i>Entity</i>
C-26	OrderItem	<i>Entity</i>
C-27	Robot	<i>Entity</i>
C-28	Task	<i>Entity</i>
C-29	AprilTagMapping	<i>Entity</i>

Subbab ini menguraikan spesifikasi internal dari kelas-kelas utama yang telah dimodelkan pada *Class Diagram* sistem *Warehouse Management Controller* (WMC). Rincian perancangan ini mendefinisikan atribut beserta tipe datanya, metode (operasi) yang dieksekusi oleh kelas, serta tingkat visibilitas dari masing-masing elemen

tersebut (Publik atau Privat). Pendefinisian ini menjadi acuan spesifikasi teknis dalam tahap implementasi kode program.

IV.5.1 Kelas *Controller*

Tabel IV.30 Detail Kelas – AuthController

Nama Operasi	Visibilitas	Keterangan
login(username, password)	Publik	Menerima kredensial pengguna dari antarmuka dan meneruskan proses autentikasi ke <i>AuthService</i> .
logout(sessionId)	Publik	Menerima permintaan keluar dan meneruskan pembatalan sesi ke <i>AuthService</i> .
validateSession(token)	Publik	Meneruskan validasi <i>bearer token</i> ke <i>AuthService</i> .
Nama Atribut	Visibilitas	Tipe
authService	Privat	AuthService

Tabel IV.31 Detail Kelas – ItemController

Nama Operasi	Visibilitas	Keterangan
listInventory()	Publik	Menerima permintaan daftar inventori dan meneruskannya ke <i>ItemService</i> .
getGoodsDetail(itemId)	Publik	Meneruskan permintaan detail barang ke <i>ItemService</i> .
getItemHistory(itemId)	Publik	Meneruskan permintaan riwayat aktivitas barang ke <i>ItemService</i> .
Nama Atribut	Visibilitas	Tipe
itemService	Privat	ItemService

IV.5.2 Kelas *Service*

IV.5.3 Kelas *Repository*

IV.5.4 Kelas *Entity*

IV.6 *Sequence Diagram*

Sequence diagram merupakan pemodelan perilaku dinamis (*dynamic behavior*) yang menggambarkan bagaimana objek-objek di dalam sistem *Warehouse Management Controller* (WMC) saling berinteraksi dari waktu ke waktu. Diagram ini memetakan urutan pertukaran pesan (*message passing*) antara aktor (*User Interface*) dengan kelas-kelas pada lapisan *Controller*, *Service*, dan *Entity* (*Database*) dalam mengeksekusi logika bisnis sistem.

Pada perancangan ini, *sequence diagram* disusun untuk merealisasikan kesembilan

Tabel IV.32 Detail Kelas – OrderController

Nama Operasi	Visibilitas	Keterangan
getOrderStatus(userId)	Publik	Meneruskan permintaan status <i>order</i> pengguna ke <i>OrderService</i> .
createOrderRequest(itemId, qty)	Publik	Menerima data <i>order</i> dari antarmuka dan meneruskan pembuatan <i>order</i> ke <i>OrderService</i> .
Nama Atribut	Visibilitas	Tipe
orderService	Privat	OrderService

Tabel IV.33 Detail Kelas – RobotController

Nama Operasi	Visibilitas	Keterangan
listRobots()	Publik	Meneruskan permintaan daftar robot ke <i>RobotService</i> .
getRobotStatus(robotId)	Publik	Meneruskan permintaan status robot ke <i>RobotService</i> .
Nama Atribut	Visibilitas	Tipe
robotService	Privat	RobotService

Tabel IV.34 Detail Kelas – DigitalTwinController

Nama Operasi	Visibilitas	Keterangan
operateDigitalTwin()	Publik	Menginisialisasi modul <i>Digital Twin</i> melalui <i>DigitalTwinService</i> .
getRobotPositions()	Publik	Meneruskan permintaan koordinat robot ke <i>DigitalTwinService</i> .
getItemPositions()	Publik	Meneruskan permintaan posisi barang ke <i>DigitalTwinService</i> .
Nama Atribut	Visibilitas	Tipe
digitalTwinService	Privat	DigitalTwinService

Tabel IV.35 Detail Kelas – AuthService

Nama Operasi	Visibilitas	Keterangan
authenticate(username, password)	Publik	Memverifikasi kredensial ke <i>UserRepository</i> dan membuat sesi baru.
revokeSession(sessionId)	Publik	Membatalkan sesi aktif melalui <i>SessionRepository</i> .
validateToken(token)	Publik	Memvalidasi <i>bearer token</i> terhadap sesi tersimpan.
generateToken(user)	Privat	Menghasilkan <i>bearer token</i> untuk sesi yang baru dibuat.
Nama Atribut	Visibilitas	Tipe
userRepository	Privat	UserRepository
sessionRepository	Privat	SessionRepository

Tabel IV.36 Detail Kelas – ItemService

Nama Operasi	Visibilitas	Keterangan
getInventoryList()	Publik	Menyusun daftar barang dari <i>ItemRepository</i> untuk kebutuhan tampilan.
getItemDetail(itemId)	Publik	Mengambil detail barang termasuk SKU, kuantitas, dan lokasi penyimpanan.
getItemHistory(itemId)	Publik	Menyusun log aktivitas <i>inbound/outbound</i> suatu barang.
resolveItemLocation(itemId)	Privat	Menyelaraskan data barang dengan lokasi dari <i>LocationService</i> .
Nama Atribut	Visibilitas	Tipe
itemRepository	Privat	ItemRepository
locationService	Privat	LocationService

Tabel IV.37 Detail Kelas – OrderService

Nama Operasi	Visibilitas	Keterangan
<code>listOrders(userId)</code>	Publik	Menyusun daftar <i>order</i> beserta statusnya untuk pengguna.
<code>createOrder(itemId, qty)</code>	Publik	Memvalidasi ketersediaan barang dan menyimpan <i>order</i> beserta <i>order item</i> .
<code>validateItemAvailability(itemId, qty)</code>	Privat	Memastikan stok mencukupi melalui <i>ItemService</i> .
<code>dispatchOrderCommand(order)</code>	Privat	Mengirim perintah <i>order</i> ke <i>fleet controller</i> dan memicu <i>NotificationService</i> .
Nama Atribut	Visibilitas	Tipe
<code>orderRepository</code>	Privat	OrderRepository
<code>orderItemRepository</code>	Privat	OrderItemRepository
<code>itemService</code>	Privat	ItemService
<code>notificationService</code>	Privat	NotificationService

Tabel IV.38 Detail Kelas – RobotService

Nama Operasi	Visibilitas	Keterangan
getRobotList()	Publik	Menyusun daftar robot dari <i>RobotRepository</i> .
getRobotStatus(robotId)	Publik	Mengambil status robot termasuk posisi, baterai, dan tugas aktif.
assignTask(robotId, orderItemId)	Publik	Menugaskan pekerjaan ke robot melalui <i>TaskService</i> .
updateRobotTelemetry(robotId, data)	Privat	Memperbarui <i>telemetry</i> robot dan sinkronisasi lokasi.
Nama Atribut	Visibilitas	Tipe
robotRepository	Privat	RobotRepository
taskService	Privat	TaskService
locationService	Privat	LocationService
notificationService	Privat	NotificationService

Tabel IV.39 Detail Kelas – TaskService

Nama Operasi	Visibilitas	Keterangan
createTask(orderItemId, robotId)	Publik	Membuat tugas baru untuk eksekusi suatu <i>order item</i> .
updateTaskStatus(taskId, status)	Publik	Memperbarui status tugas robot.
getActiveTasks(robotId)	Publik	Mengambil daftar tugas aktif suatu robot.
Nama Atribut	Visibilitas	Tipe
taskRepository	Privat	TaskRepository

Tabel IV.40 Detail Kelas – DigitalTwinService

Nama Operasi	Visibilitas	Keterangan
initializeTwin()	Publik	Memuat model 3D gudang dan menyiapkan langganan koordinat robot.
getRobotPositions()	Publik	Menyusun koordinat robot terkini dari <i>LocationService</i> .
getItemPositions()	Publik	Menyusun posisi barang berdasarkan data <i>AprilTagMapping</i> .
Nama Atribut	Visibilitas	Tipe
aprilTagMappingRepository	Privat	AprilTagMappingRepository
locationService	Privat	LocationService

Tabel IV.41 Detail Kelas – LocationService

Nama Operasi	Visibilitas	Keterangan
resolvePosition(aprilTagId)	Publik	Menentukan koordinat entitas berdasarkan pemetaan April-Tag.
mapRobotPosition(robotId)	Publik	Menyelaraskan posisi robot dengan data <i>AprilTagMapping</i> .
mapItemPosition(itemId)	Publik	Menentukan lokasi penyimpanan suatu barang.
Nama Atribut	Visibilitas	Tipe
aprilTagMappingRepository	Privat	AprilTagMappingRepository

Tabel IV.42 Detail Kelas – NotificationService

Nama Operasi	Visibilitas	Keterangan
notifyOrderStatus(order)	Publik	Mengirim pembaruan status <i>order</i> ke antarmuka pengguna.
notifyRobotStatus(robot)	Publik	Mengirim pembaruan status robot ke antarmuka pengguna.
broadcastTelemetry(payload)	Privat	Menyiarkan <i>telemetry</i> secara <i>real-time</i> melalui <i>WebSocket</i> .
Nama Atribut	Visibilitas	Tipe
messageBroker	Privat	MessageBroker

Tabel IV.43 Detail Kelas – UserRepository

Nama Operasi	Visibilitas	Keterangan
save(user)	Publik	Menyimpan atau memperbarui data pengguna pada basis data.
findById(userId)	Publik	Mengambil pengguna berdasarkan ID.
findByUsername(username)	Publik	Mengambil pengguna berdasarkan <i>username</i> untuk autentikasi.
Nama Atribut	Visibilitas	Tipe
sumberDataPengguna	Privat	DataSource

Tabel IV.44 Detail Kelas – SessionRepository

Nama Operasi	Visibilitas	Keterangan
save(session)	Publik	Menyimpan sesi pengguna baru.
findById(sessionId)	Publik	Mengambil sesi berdasarkan ID.
deleteById(sessionId)	Publik	Menghapus sesi ketika pengguna keluar.
Nama Atribut	Visibilitas	Tipe
sumberDataSesi	Privat	DataSource

Tabel IV.45 Detail Kelas – ItemRepository

Nama Operasi	Visibilitas	Keterangan
save(item)	Publik	Menyimpan atau memperbarui data barang.
findById(itemId)	Publik	Mengambil barang berdasarkan ID.
findAll()	Publik	Mengambil seluruh daftar inventori.
findHistory(itemId)	Publik	Mengambil log aktivitas suatu barang.
Nama Atribut	Visibilitas	Tipe
sumberDataBarang	Privat	DataSource

Tabel IV.46 Detail Kelas – OrderRepository

Nama Operasi	Visibilitas	Keterangan
save(order)	Publik	Menyimpan <i>order</i> baru.
findById(orderId)	Publik	Mengambil <i>order</i> berdasarkan ID.
findByUser(userId)	Publik	Mengambil daftar <i>order</i> milik pengguna.
updateStatus(orderId, status)	Publik	Memperbarui status <i>order</i> pada basis data.
Nama Atribut	Visibilitas	Tipe
sumberDataOrder	Privat	DataSource

Tabel IV.47 Detail Kelas – OrderItemRepository

Nama Operasi	Visibilitas	Keterangan
save(orderItem)	Publik	Menyimpan item <i>order</i> pada basis data.
findByOrder(orderId)	Publik	Mengambil seluruh item berdasarkan <i>order</i> .
Nama Atribut	Visibilitas	Tipe
sumberDataOrderItem	Privat	DataSource

Tabel IV.48 Detail Kelas – RobotRepository

Nama Operasi	Visibilitas	Keterangan
save(robot)	Publik	Menyimpan atau memperbarui data robot.
findById(robotId)	Publik	Mengambil robot berdasarkan ID.
findAll()	Publik	Mengambil seluruh daftar robot.
Nama Atribut	Visibilitas	Tipe
sumberDataRobot	Privat	DataSource

Tabel IV.49 Detail Kelas – TaskRepository

Nama Operasi	Visibilitas	Keterangan
save(task)	Publik	Menyimpan tugas robot pada basis data.
findById(taskId)	Publik	Mengambil tugas berdasarkan ID.
findByRobot(robotId)	Publik	Mengambil daftar tugas aktif suatu robot.
Nama Atribut	Visibilitas	Tipe
sumberDataTugas	Privat	DataSource

Tabel IV.50 Detail Kelas – AprilTagMappingRepository

Nama Operasi	Visibilitas	Keterangan
save(mapping)	Publik	Menyimpan pemetaan AprilTag.
findById(aprilTagId)	Publik	Mengambil pemetaan berdasarkan ID tag.
findByRobot(robotId)	Publik	Mengambil pemetaan posisi suatu robot.
findByItem(itemId)	Publik	Mengambil pemetaan posisi suatu barang.
Nama Atribut	Visibilitas	Tipe
sumberDataAprilTag	Privat	DataSource

Tabel IV.51 Detail Kelas – User

Nama Operasi	Visibilitas	Keterangan
authenticate(pw)	Publik	Memverifikasi kata sandi pengguna.
Nama Atribut	Visibilitas	Tipe
user_id	Privat	int
username	Privat	string
password	Privat	string
created_at	Privat	datetime
updated_at	Privat	datetime
deleted_at	Privat	datetime

Tabel IV.52 Detail Kelas – Session

Nama Operasi	Visibilitas	Keterangan
isValid()	Publik	Mengecek validitas sesi berdasarkan waktu kedaluwarsa.
Nama Atribut	Visibilitas	Tipe
session_id	Privat	int
user_id	Privat	int
token	Privat	string
expires_at	Privat	datetime

Tabel IV.53 Detail Kelas – Item

Nama Operasi	Visibilitas	Keterangan
updateQuantity(qty)	Publik	Memperbarui kuantitas stok barang.
getHistory()	Publik	Mengembalikan riwayat aktivitas barang.
Nama Atribut	Visibilitas	Tipe
item_id	Privat	int
name	Privat	string
sku	Privat	string
category	Privat	string
quantity	Privat	int

Tabel IV.54 Detail Kelas – Order

Nama Operasi	Visibilitas	Keterangan
updateStatus(status)	Publik	Mengubah status <i>order</i> menjadi <i>Pending</i> , <i>InProgress</i> , <i>Completed</i> , atau <i>Failed</i> .
Nama Atribut	Visibilitas	Tipe
order_id	Privat	int
user_id	Privat	int
status	Privat	string

Tabel IV.55 Detail Kelas – OrderItem

Nama Operasi	Visibilitas	Keterangan
updateQuantity(qty)	Publik	Menyesuaikan kuantitas item dalam <i>order</i> .
Nama Atribut	Visibilitas	Tipe
order_item_id	Privat	int
order_id	Privat	int
item_id	Privat	int
quantity	Privat	int

Tabel IV.56 Detail Kelas – Robot

Nama Operasi	Visibilitas	Keterangan
getStatus()	Publik	Mengembalikan status robot terkini.
Nama Atribut	Visibilitas	Tipe
robot_id	Privat	int
serial_number	Privat	string
max_payload	Privat	float

Tabel IV.57 Detail Kelas – Task

Nama Operasi	Visibilitas	Keterangan
updateStatus(status)	Publik	Memperbarui status tugas robot.
Nama Atribut	Visibilitas	Tipe
task_id	Privat	int
order_item_id	Privat	int
robot_id	Privat	int
status	Privat	string

Tabel IV.58 Detail Kelas – AprilTagMapping

Nama Operasi	Visibilitas	Keterangan
resolvePosition()	Publik	Mengembalikan koordinat entitas berdasarkan <i>tag</i> terpasang.
Nama Atribut	Visibilitas	Tipe
apriltag_id	Privat	int
item_id	Privat	int
robot_id	Privat	int

skenario *use case* yang telah didefinisikan pada Subbab IV.4.

1. **Sequence Diagram UC-01: Login System**

Sequence diagram ini menggambarkan alur sistem saat pengguna melakukan autentikasi. Proses dimulai ketika antarmuka pengguna mengirimkan kredensial (*username* dan *password*) ke *AuthController*. *Controller* meneruskan permintaan tersebut ke *AuthService* untuk divalidasi terhadap data kredensial yang tersimpan pada basis data *User*. Jika validasi berhasil, sistem membuat sesi baru dan mengembalikan *bearer token* kepada pengguna.

2. **Sequence Diagram UC-02: Logout System**

Sequence diagram ini memodelkan proses pengakhiran sesi. Antarmuka pengguna mengirimkan permintaan keluar beserta *session ID* yang sedang aktif ke *AuthController*. *AuthService* memverifikasi dan membatalkan sesi tersebut pada basis data *Session*, sehingga token yang bersangkutan tidak lagi dapat digunakan untuk otorisasi permintaan berikutnya.

3. **Sequence Diagram UC-03: View Inventory List**

Sequence diagram ini menggambarkan alur penampilan daftar inventori. Antarmuka pengguna mengirimkan permintaan ke *ItemController*, yang meneruskannya ke *goods detail service* untuk mengambil seluruh data barang. Hasilnya dikembalikan dan ditampilkan sebagai daftar barang beserta kuantitas dan kategori.

4. **Sequence Diagram UC-04: View Goods Detail**

Sequence diagram ini menggambarkan alur penampilan detail suatu barang.

Ketika pengguna memilih salah satu barang dari daftar inventori, antarmuka mengirimkan permintaan ke *ItemController* yang meneruskannya ke *goods detail service* untuk mengambil informasi lengkap barang, kemudian menampilkannya pada halaman detail.

5. **Sequence Diagram UC-05: View Order Status**

Sequence diagram ini menggambarkan alur pemantauan status *order*. Antarmuka pengguna mengirimkan permintaan ke *OrderController*, yang meneruskannya ke *fleet order service* untuk mengambil data *order*. Sistem menampilkan daftar *order* beserta status terkini (*Pending*, *InProgress*, *Completed*, atau *Failed*).

6. **Sequence Diagram UC-06: View Robot & Operation Status**

Sequence diagram ini menggambarkan alur pemantauan status robot secara *real-time*. Antarmuka pengguna berlangganan (*subscribe*) *telemetry stream* melalui *WebSocket* yang dikelola *RobotController*, sehingga panel status armada AGV—mencakup posisi, baterai, dan tugas aktif—dapat diperbarui secara langsung tanpa *polling* berulang.

7. **Sequence Diagram UC-07: Operate Digital Twin**

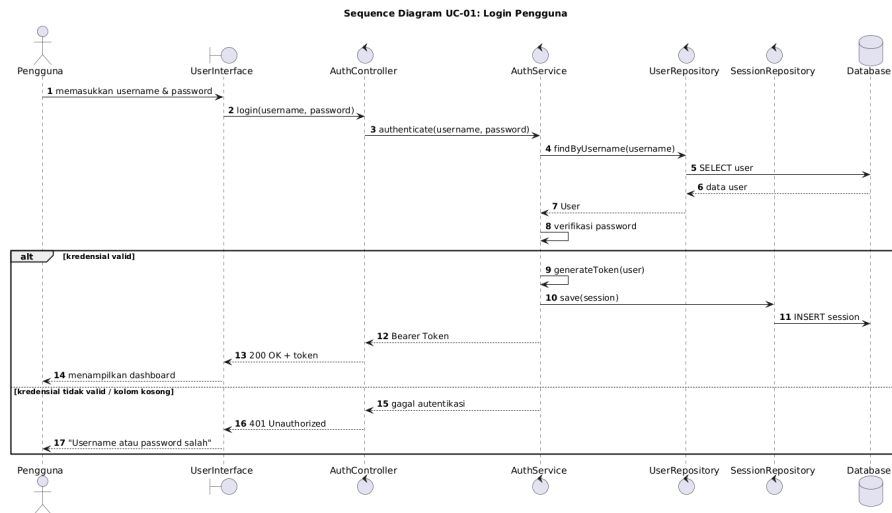
Sequence diagram ini menggambarkan alur interaksi dengan modul *Digital Twin*. Antarmuka pengguna memuat tampilan *Digital Twin* melalui *DigitalTwinController*, yang menginisialisasi model 3D gudang dan berlangganan koordinat robot secara *real-time*. Saat pengguna memilih robot atau barang pada tampilan 3D, sistem menampilkan detail entitas yang dipilih.

8. **Sequence Diagram UC-08: Create Order Request**

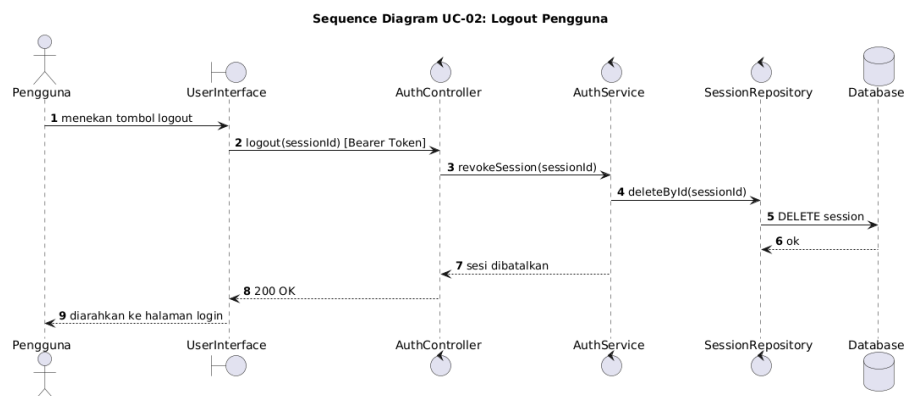
Sequence diagram ini merepresentasikan alur pembuatan permintaan *order*. Antarmuka pengguna mengirimkan formulir *order* (barang dan kuantitas) ke *OrderController*, yang memvalidasi ketersediaan barang melalui *item service*. Setelah tervalidasi, sistem menyimpan *order* beserta *order item* ke basis data, mengirim perintah eksekusi ke *fleet controller* melalui AMQP, kemudian mengembalikan konfirmasi pembuatan *order*.

9. **Sequence Diagram UC-09: View Item History**

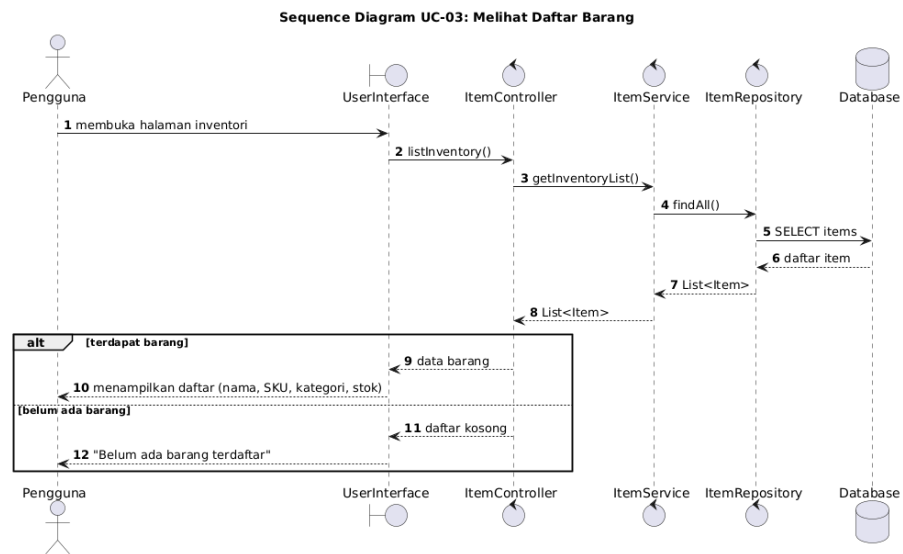
Sequence diagram ini menggambarkan alur penelusuran riwayat aktivitas barang. Antarmuka pengguna memilih barang tertentu dan mengirimkan permintaan ke *ItemController*, yang meneruskannya ke *goods detail service* untuk mengambil log aktivitas. Sistem menampilkan kronologi aktivitas *inbound/outbound* barang tersebut.



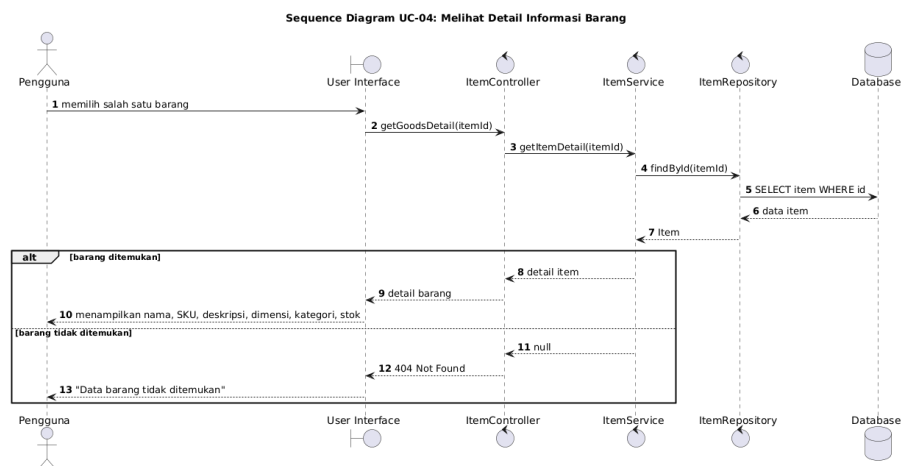
Gambar IV.22 *Sequence Diagram UC-01: Login System*



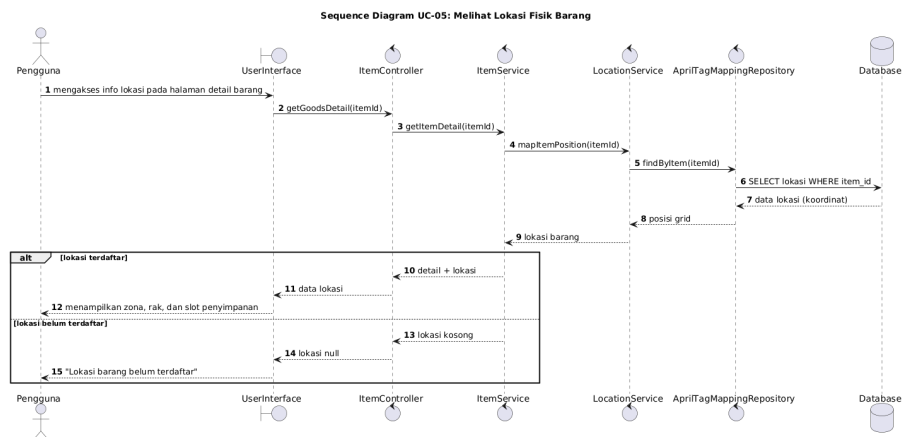
Gambar IV.23 *Sequence Diagram UC-02: Logout System*



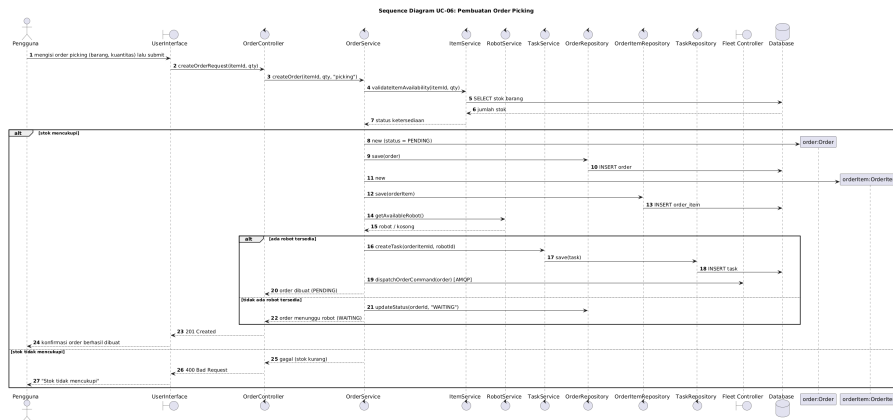
Gambar IV.24 Sequence Diagram UC-03: View Inventory List



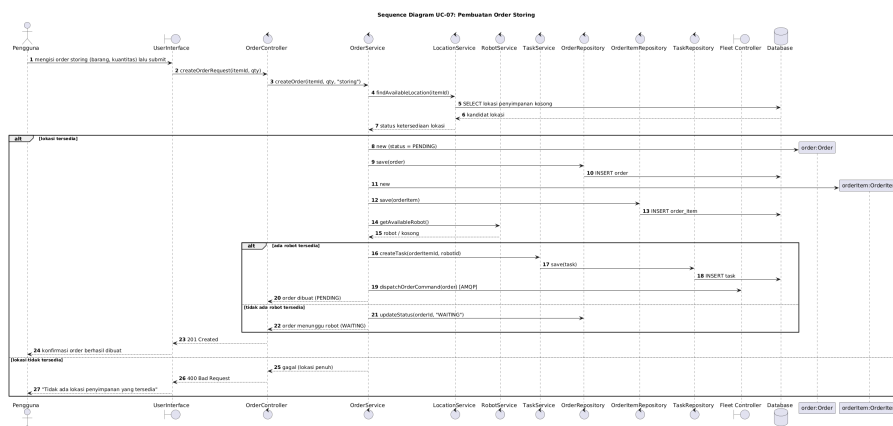
Gambar IV.25 Sequence Diagram UC-04: View Goods Detail



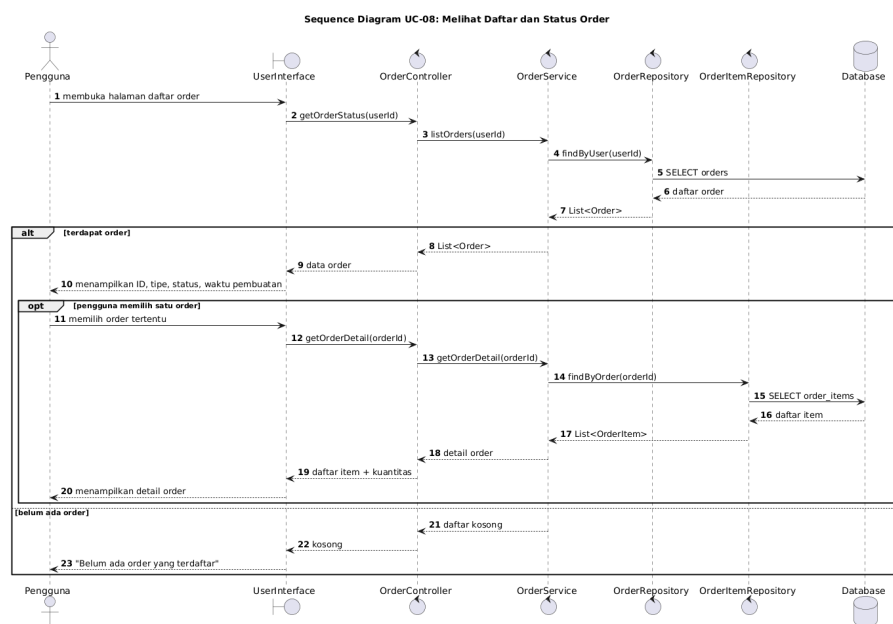
Gambar IV.26 Sequence Diagram UC-05: View Order Status



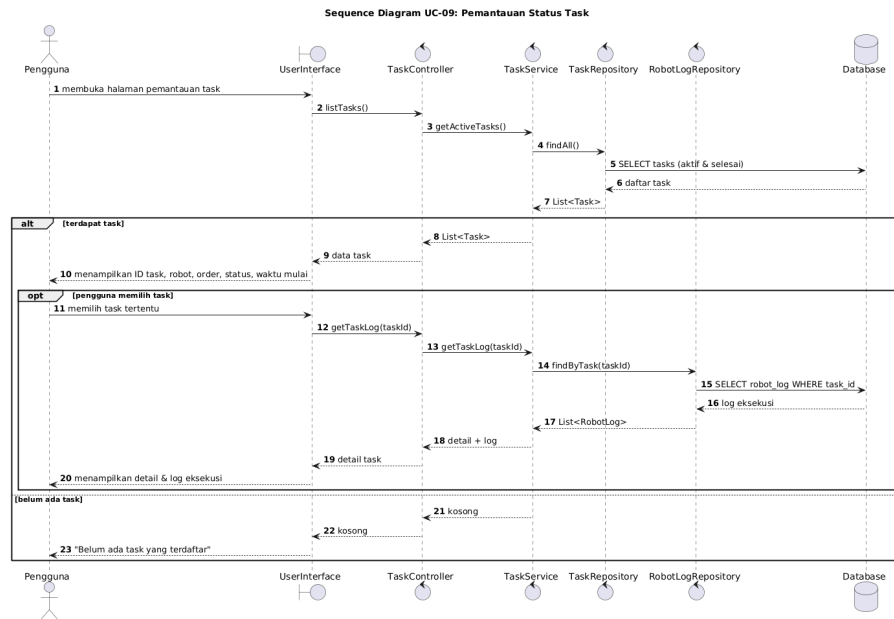
Gambar IV.27 Sequence Diagram UC-06: View Robot & Operation Status



Gambar IV.28 Sequence Diagram UC-07: Operate Digital Twin



Gambar IV.29 Sequence Diagram UC-08: Create Order Request



Gambar IV.30 Sequence Diagram UC-09: View Item History

BAB V

IMPLEMENTASI

V.1 Lingkungan Implementasi

Pengembangan sistem *Warehouse Management Controller* (WMC) dilakukan di dalam sebuah lingkungan implementasi yang terbagi menjadi dua jenis, yaitu perangkat keras (*hardware*) dan perangkat lunak (*software*). Keduanya menjadi komponen penting yang berjalan bersama dan saling mendukung dalam proses pengembangan sistem.

Pada sisi perangkat keras, pengembangan dan pengujian sistem dilakukan pada lingkungan *cloud* berbasis Amazon Web Services (AWS). Komponen yang digunakan dapat dilihat pada Tabel V.1.

Tabel V.1 Spesifikasi *hardware*

Komponen	Spesifikasi
Platform	Amazon EC2 t3.medium
Prosesor	2 vCPU (Intel Xeon Platinum 8259CL)
RAM	4 GB DDR4
Penyimpanan	Amazon EBS gp3, 20 GB
Sistem Operasi	Ubuntu Server 22.04 LTS (Jammy Jellyfish)
Bandwidth Jaringan	Up to 5 Gbps

Adapun perangkat lunak yang digunakan selama proses pengembangan dapat dilihat pada Tabel V.2.

Dengan menggunakan kombinasi lingkungan *cloud* Amazon EC2 dan rangkaian perangkat lunak berbasis kontainer tersebut, seluruh proses pengembangan, pengujian, serta orkestrasi layanan WMC dapat dilaksanakan secara konsisten dan terisolasi.

V.2 Implementasi Sistem WMC

Sub-bab ini menjabarkan detail implementasi sistem *Warehouse Management Controller* (WMC), mencakup teknologi yang digunakan, implementasi modul, serta implementasi *endpoint* pada masing-masing layanan.

Tabel V.2 Spesifikasi *software*

Jenis	Nama Software
Bahasa Pemrograman	TypeScript
Runtime	Node.js
Framework/Library (auth-service)	Hono, @hono/node-server, @hono/swagger-ui
Framework/Library (goods-service)	Express.js, swagger-jsdoc, swagger-ui-express
Framework/Library (blob-service)	Express.js, Multer, Axios, Form-data
Library Umum (semua service)	JWT (jsonwebtoken), Bcryptjs, Dotenv, CORS
ORM	Prisma (dengan adapter PostgreSQL)
Database	PostgreSQL 15
Containerization	Docker, Docker Compose
Database Admin Tool	Adminer

V.2.1 Teknologi Yang Digunakan

Pengembangan sistem WMC memanfaatkan sejumlah bahasa pemrograman, *runtime*, *framework*, *library*, ORM, basis data, serta alat bantu kontainerisasi. Seluruh teknologi yang digunakan beserta deskripsi singkatnya dapat dilihat pada Tabel V.3.

Tabel V.3 Teknologi yang digunakan dalam pengembangan WMC

Nama	Kategori	Deskripsi
TypeScript	Bahasa Pemrograman	<i>Superset</i> dari JavaScript yang menambahkan sistem tipe statis sehingga meningkatkan keandalan dan keterbacaan kode pada seluruh layanan WMC.
Node.js	Runtime	Lingkungan eksekusi JavaScript sisi server berbasis <i>event-driven</i> yang menjadi fondasi <i>runtime</i> untuk seluruh layanan <i>backend</i> WMC.
Hono	Framework (auth-service)	Framework web ringan dan berperforma tinggi yang digunakan sebagai server HTTP utama pada layanan autentikasi.

lanjutan Tabel V.3

Nama	Kategori	Deskripsi
@hono/node-server	Library (auth-service)	Adapter resmi Hono yang memungkinkan framework Hono dijalankan secara native di lingkungan runtime Node.js.
@hono/swagger-ui	Library (auth-service)	Plugin Hono untuk menyajikan antarmuka dokumentasi API interaktif berbasis Swagger UI langsung dari dalam layanan autentikasi.
Express.js	Framework (goods, blob service)	Framework web minimalis untuk Node.js yang digunakan sebagai fondasi server HTTP pada layanan <i>goods</i> dan <i>blob</i> .
swagger-jsdoc	Library (goods-service)	Library untuk menghasilkan spesifikasi OpenAPI secara otomatis dari anotasi JSDoc yang dituliskan pada kode sumber.
swagger-ui-express	Library (goods-service)	Middleware Express yang menyajikan halaman dokumentasi API interaktif berbasis Swagger UI pada layanan <i>goods</i> .
Multer	Library (blob-service)	Middleware Node.js untuk menangani unggahan <i>file</i> berformat <i>multipart/form-data</i> pada layanan penyimpanan berkas.
Axios	Library (blob-service)	HTTP <i>client</i> berbasis <i>Promise</i> yang digunakan untuk melakukan permintaan HTTP antar layanan secara asinkron.
Form-data	Library (blob-service)	Library untuk membuat dan mengelola data formulir <i>multipart</i> yang digunakan saat meneruskan berkas antar layanan.

lanjutan Tabel V.3

Nama	Kategori	Deskripsi
jsonwebtoken (JWT)	Library (semua service)	Library untuk menghasilkan dan memverifikasi JSON Web Token sebagai mekanisme autentikasi <i>stateless</i> pada setiap layanan.
Bcryptjs	Library (semua service)	Library untuk melakukan <i>hashing</i> kata sandi secara aman menggunakan algoritma <i>bcrypt</i> saat proses registrasi dan verifikasi pengguna.
Dotenv	Library (semua service)	Library yang memuat variabel konfigurasi dan rahasia dari <i>file .env</i> ke dalam <i>process.env</i> pada saat layanan dijalankan.
CORS	Library (semua service)	Middleware untuk mengonfigurasi kebijakan <i>Cross-Origin Resource Sharing</i> agar setiap layanan API dapat diakses lintas domain secara aman.
Prisma	ORM	<i>Object-Relational Mapper</i> modern untuk TypeScript yang menyederhanakan akses basis data, pembuatan <i>query</i> , serta migrasi skema secara <i>type-safe</i> .
PostgreSQL 15	Database	Sistem manajemen basis data relasional <i>open-source</i> yang digunakan sebagai penyimpanan data utama seluruh layanan WMC.
Docker	Containerization	Platform kontainerisasi yang digunakan untuk mengemas setiap layanan beserta dependensinya ke dalam <i>container</i> yang terisolasi dan portabel.
Docker Compose	Containerization	Alat orkestrasi multi-kontainer yang mendefinisikan dan menjalankan seluruh layanan WMC secara bersamaan melalui satu <i>file</i> konfigurasi.

lanjutan Tabel V.3

Nama	Kategori	Deskripsi
Adminer	Database Admin Tool	Antarmuka administrasi basis data berbasis web yang ringan, digunakan untuk mengelola dan memantau basis data PostgreSQL selama pengembangan.

V.2.2 Implementasi Modul

Implementasi sistem WMC diorganisasikan ke dalam tiga komponen utama, yaitu *auth-service*, *goods-service*, dan *database*. Masing-masing komponen memiliki sejumlah modul dengan tanggung jawab yang terpisah sesuai prinsip *separation of concerns*, sehingga setiap modul dapat dikembangkan, diuji, dan dipelihara secara independen.

auth-service

Tabel V.4 Modul *auth-service*

No	Nama Modul	Nama File Fisik
1	Entry Point	auth-service/src/index.ts
2	Authentication	auth-service/src/routes/auth.ts
3	Middleware	auth-service/src/middleware/jwt.ts
4	Store / Data Access	auth-service/src/store/users.ts

goods-service

database

V.2.3 Implementasi Endpoint

Sub-bab ini merinci seluruh *endpoint* REST API yang diimplementasikan pada setiap layanan WMC. Kolom *Method* menunjukkan metode HTTP yang digunakan (GET, POST, PUT, DELETE), kolom *Endpoint* menunjukkan jalur URL yang tersedia, dan kolom *Auth* menunjukkan apakah *endpoint* tersebut memerlukan token JWT sebagai otorisasi akses.

auth-service — localhost:3000

goods-service — localhost:3001

V.3 Implementasi Subsistem Lokalisasi

Sub-bab ini menjabarkan detail implementasi Subsistem Lokalisasi yang berjalan sebagai komponen dalam *Fleet Controller*, mencakup lingkungan perangkat keras,

teknologi yang digunakan, implementasi *pipeline* deteksi *frame*, serta mekanisme publikasi data posisi melalui MQTT.

V.3.1 Lingkungan dan Perangkat Keras

Subsistem Lokalisasi berjalan di atas *host* yang sama dengan WMC, yaitu *instance* AWS EC2 t3.medium (Ubuntu 22.04 LTS). Perangkat keras utama yang digunakan adalah sebuah kamera IP *fisheye* berkemampuan 1080p (3 MP) dengan kompresi H.264 yang melakukan *streaming* video melalui protokol RTSP *over* TCP. Kamera dipasang pada rangka penyangga yang ditempatkan setinggi 60 cm di atas area lantai gudang berukuran $2,4 \times 1,2$ m. Pencahayaan LED tambahan dipasang untuk memastikan iluminasi yang seragam di seluruh area lantai sehingga kontras *marker* AprilTag tetap optimal di berbagai kondisi pencahayaan ruangan.

AprilTag yang digunakan adalah keluarga Tag36h11 berukuran fisik 8×8 cm, ditempelkan pada bagian atas robot dan pada barang-barang di area gudang. Keluarga *tag* ini dipilih karena memberikan keseimbangan optimal antara keterbacaan pada jarak jauh dan ketahanan terhadap oklusi parsial, sesuai dengan kondisi operasional gudang skala prototipe ini.

V.3.2 Teknologi yang Digunakan

Tabel V.9 merangkum seluruh teknologi yang digunakan dalam implementasi Subsistem Lokalisasi beserta peran masing-masing dalam *pipeline*.

V.3.3 Implementasi Pipeline Deteksi

Pipeline deteksi diimplementasikan dalam dua *thread* Python yang berjalan secara konkuren. *Thread* pertama adalah *frame grabber thread* yang terhubung ke *stream* RTSP kamera dan secara terus-menerus mengambil *frame* terbaru ke dalam sebuah *buffer* bersama, membuang *frame* lama yang belum sempat diproses. *Thread* kedua adalah *detection loop* yang berjalan pada frekuensi 4 Hz, membaca *frame* dari *buffer* dan memprosesnya melalui enam tahap *pipeline* secara berurutan.

Pada tahap pertama, distorsi radial lensa *fisheye* dihilangkan menggunakan koefisien kalibrasi Kannala–Brandt yang diperoleh dari proses kalibrasi *checkerboard* saat *setup* awal. Koefisien ini disimpan sebagai parameter permanen dan dimuat sekali saat program pertama kali dijalankan. Pada tahap kedua, *frame* yang telah terkoreksi ditingkatkan kontrasnya menggunakan *histogram equalization* kemudian dikonversi ke *grayscale* untuk mempercepat proses deteksi.

Pada tahap ketiga, detektor Tag36h11 dari *library* pupil-apriltags dijalankan pada

frame grayscale untuk mengekstrak ID unik, koordinat piksel pusat *tag*, serta koordinat piksel empat sudut *marker* dari setiap *tag* yang berhasil terdeteksi. Pada tahap keempat, koordinat piksel pusat setiap *tag* ditransformasi ke koordinat lantai kontinu menggunakan matriks *homography* H yang telah dikalibrasi sebelumnya melalui OpenCV `perspectiveTransform`, kemudian di-*snap* ke sel *grid* diskrit (c, r) menggunakan formula *nearest-cell-centre* sebagaimana dijelaskan pada Subbab 2.5.3.

Pada tahap kelima, setiap *tag* ID diklasifikasikan ke salah satu dari tiga kategori berdasarkan rentang ID yang telah ditetapkan: *docking station*, robot, atau *warehouse item*. Orientasi (*heading*) dalam derajat (0° – 359°) dihitung dari sudut yang dibentuk oleh sisi depan *tag* relatif terhadap sumbu X positif menggunakan fungsi `atan2` terhadap selisih koordinat dua sudut terdepan *tag*.

V.3.4 Implementasi Publikasi MQTT

Pada tahap keenam *pipeline*, seluruh hasil deteksi *frame* dikodekan sebagai *payload* JSON yang merepresentasikan *state* penuh dari *grid* 8×4 . *Payload* diterbitkan setiap 500 ms ke *topic location* pada RabbitMQ *broker* yang sama yang digunakan oleh WMC. Setiap entri dalam *payload* memuat ID *tag*, kategori objek (*docking/robot/item*), koordinat sel *grid* (c, r) , dan nilai *heading* dalam derajat. Dengan interval publikasi 500 ms, latensi maksimum tampilan posisi di *User Interface* terikat pada 500 ms ditambah latensi jaringan WebSocket, jauh di bawah target KNF-02 sebesar 1 detik.

Kalibrasi *homography* dilakukan satu kali saat *setup* dengan menempatkan empat *marker* referensi di sudut-sudut area lantai gudang yang diketahui posisi *grid*-nya. Korespondensi piksel ke *grid* dari keempat titik tersebut kemudian digunakan oleh OpenCV `getPerspectiveTransform` untuk menghitung matriks H . Setelah kalibrasi selesai, parameter H disimpan dan dimuat secara otomatis setiap kali Subsistem Lokalisasi dijalankan ulang tanpa memerlukan proses kalibrasi ulang.

Tabel V.5 Modul *goods-service*

No	Nama Modul	Nama File Fisik
1	Entry Point	goods-service/src/index.ts
2	User	goods-service/src/controllers/ UserController.ts, goods-service/src/ services/UserService.ts
3	Item (Barang)	goods-service/src/controllers/ ItemController.ts, goods-service/src/ services/ItemService.ts
4	Item Location	goods-service/src/controllers/ ItemLocationController.ts, goods-service/ src/services/ItemLocationService.ts
5	Order	goods-service/src/controllers/ OrderController.ts, goods-service/src/ services/OrderService.ts
6	Order Item	goods-service/src/controllers/ OrderItemController.ts, goods-service/ src/services/OrderItemService.ts
7	Robot	goods-service/src/controllers/ RobotController.ts, goods-service/src/ services/RobotService.ts
8	Robot Log	goods-service/src/controllers/ RobotLogController.ts, goods-service/src/ services/RobotLogService.ts
9	Task	goods-service/src/controllers/ TaskController.ts, goods-service/src/ services/TaskService.ts
10	Session	goods-service/src/controllers/ SessionController.ts, goods-service/src/ services/SessionService.ts
11	Base (Abstraksi)	goods-service/src/controllers/ BaseController.ts, goods-service/src/ services/BaseService.ts
12	Middleware	goods-service/src/middleware/jwt.ts
13	Library / Helper	goods-service/src/lib/prisma.ts, goods-service/src/lib/response.ts
14	Dokumentasi API	goods-service/src/docs/swagger.ts
15	Konfigurasi	goods-service/package.json, goods-service/ tsconfig.json, goods-service/ prisma.config.ts, goods-service/Dockerfile

Tabel V.6 Modul *database*

No	Nama Modul	Nama File Fisik
1	Entry Point	database/index.ts
2	Skema Database	database/prisma/schema.prisma
3	Seeder	database/prisma/seed.ts
4	Migrasi	database/prisma/migrations/ 20260312044418_init_db/migration.sql, database/prisma/migrations/ migration_lock.toml
5	Konfigurasi	database/package.json, database/ tsconfig.json, database/prisma.config.ts, database/Dockerfile, database/ docker-compose.yml

Tabel V.7 Daftar *endpoint auth-service*

No	Method	Endpoint	Deskripsi	Auth
1	POST	/auth/register	Registrasi user baru	—
2	POST	/auth/login	Login, mendapatkan JWT token	—
3	GET	/protected	Contoh <i>endpoint</i> yang dilindungi JWT	JWT

Tabel V.8 Daftar *endpoint goods-service*

No	Method	Endpoint	Deskripsi	Auth
1	GET	/users	Get semua user	JWT
2	GET	/users/:id	Get user by ID	JWT
3	POST	/users	Buat user baru	JWT
4	PUT	/users/:id	Update user	JWT
5	DELETE	/users/:id	Hapus user	JWT
6	GET	/orders	Get semua order	JWT
7	GET	/orders/user/:userId	Get order by user	JWT
8	POST	/orders	Buat order baru	JWT
9	GET	/items	Get semua item/barang	JWT
10	GET	/items/sku/:sku	Get item by SKU	JWT
11	POST	/items	Tambah item baru	JWT
12	DELETE	/items/:id	Hapus item	JWT
13	GET	/item-locations	Get semua lokasi item	JWT
14	GET	/item-locations/:id	Get lokasi by ID	JWT
15	GET	/item-locations/item/:itemId	Get lokasi by item	JWT
16	POST	/item-locations	Tambah lokasi item	JWT
17	PUT	/item-locations/:id	Update lokasi item	JWT
18	DELETE	/item-locations/:id	Hapus lokasi item	JWT
19	GET	/order-items	Get semua order item	JWT
20	GET	/order-items/:id	Get order item by ID	JWT
21	GET	/order-items/order/:orderId	Get order item by order	JWT
22	POST	/order-items	Tambah order item	JWT
23	PUT	/order-items/:id	Update order item	JWT
24	DELETE	/order-items/:id	Hapus order item	JWT
25	GET	/robots	Get semua robot	JWT
26	POST	/robots	Tambah robot baru	JWT
27	GET	/tasks	Get semua task	JWT
28	GET	/tasks/:id	Get task by ID	JWT
29	GET	/tasks/robot/:robotId	Get task by robot	JWT
30	POST	/tasks	Buat task baru	JWT
31	PUT	/tasks/:id	Update task	JWT
32	DELETE	/tasks/:id	Hapus task	JWT
33	GET	/robot-logs	Get semua log robot	JWT
34	GET	/robot-logs/:id	Get log by ID	JWT
35	GET	/robot-logs/robot/:robotId	Get log by robot	JWT
36	POST	/robot-logs	Tambah log robot	JWT
37	PUT	/robot-logs/:id	Update log robot	JWT
38	DELETE	/robot-logs/:id	Hapus log robot	JWT

Tabel V.9 Teknologi yang digunakan dalam implementasi Subsistem Lokalisasi

Teknologi	Kategori	Deskripsi
Python 3	Bahasa Pemrograman	Runtime utama <i>pipeline</i> deteksi dan publikasi MQTT.
OpenCV	Computer Vision	Undistortion lensa <i>fisheye</i> , komputasi <i>homography</i> , dan transformasi perspektif.
pupil-apriltags	Deteksi Marker	Detektor Tag36h11 AprilTag; mengekstrak ID, posisi piksel pusat, dan empat koordinat sudut setiap <i>tag</i> .
Model Kannala–Brandt	Kalibrasi Kamera	Koefisien kalibrasi untuk koreksi distorsi radial lensa <i>fisheye</i> , dihitung saat <i>setup</i> .
RTSP / TCP	Protokol Streaming	Konsumsi <i>video stream</i> H.264 dari kamera IP <i>fisheye</i> secara <i>real-time</i> .
Python threading	Konkurensi	<i>Background thread</i> mengambil <i>frame</i> secara kontinu tanpa memblokir <i>detection loop</i> utama.
RabbitMQ (MQTT)	Message Broker	Publikasi <i>grid state</i> JSON ke <i>topic location</i> setiap 500 ms; <i>broker</i> yang sama digunakan oleh WMC.

BAB VI

EVALUASI

Bab ini memaparkan proses evaluasi terhadap sistem *Warehouse Management Controller* (WMC) yang telah diimplementasikan. Evaluasi bertujuan untuk mengukur sejauh mana sistem yang dikembangkan memenuhi spesifikasi yang telah ditetapkan pada dokumen B200, khususnya pada aspek-aspek yang berada dalam cakupan pengembangan WMC, yaitu *usability* perangkat lunak, *traceability* data, produktivitas sistem, dan akurasi pemenuhan tugas. Hasil evaluasi ini digunakan sebagai dasar untuk menilai kelayakan dan efektivitas sistem WMC dalam mendukung operasional *Smart Warehouse* secara keseluruhan.

VI.1 Metode Evaluasi

Evaluasi sistem WMC dirancang mengacu pada rencana verifikasi yang tertuang dalam dokumen B200, dengan memfokuskan penilaian pada komponen-komponen yang secara langsung dipengaruhi oleh WMC sebagai lapisan perangkat lunak pengendali gudang. Mengingat peran WMC sebagai sistem *backend* yang mengatur manajemen inventori, koordinasi tugas robot, serta antarmuka pengguna, maka terdapat empat kriteria evaluasi utama yang akan diukur, yaitu *Usability*, *Traceability*, *Productivity*, dan *Task Fulfilment Accuracy*. Setiap kriteria dievaluasi melalui metode pengujian yang berbeda namun saling melengkapi untuk memperoleh gambaran yang komprehensif mengenai kualitas sistem secara keseluruhan.

VI.1.0.0.1 a. Pengujian Usability — Kuesioner System Usability Scale (SUS)

Pengujian *usability* dilakukan untuk mengukur tingkat kemudahan penggunaan antarmuka sistem informasi dan *fleet management* WMC. Mengacu pada spesifikasi B200, sistem dinyatakan memenuhi kriteria apabila skor *System Usability Scale* (SUS) yang diperoleh melebihi nilai 80 (kategori ?Good? hingga ?Excellent?) (Bangor, Kortum, dan Miller 2009). Metode yang digunakan adalah pengisian kuesioner SUS standar yang terdiri dari 10 butir pertanyaan dengan skala Likert 1–5, yang kemudian dihitung dan dikonversi menjadi skor SUS dengan rentang 0–100.

Prosedur pengujian melibatkan lima orang responden dengan latar belakang yang

beragam, yaitu pengguna dengan pemahaman teknis tinggi, pengguna awam, dan pengguna dengan pengalaman terbatas dalam penggunaan sistem informasi. Setiap responden diminta untuk menjalankan sejumlah skenario tugas utama pada antarmuka WMC, mencakup proses login, pengelolaan data barang, pembuatan *order picking* dan *storing*, pemantauan status *task*, serta pemantauan status robot. Setelah menyelesaikan seluruh skenario, responden mengisi kuesioner SUS secara mandiri. Skor individu kemudian dirata-ratakan untuk memperoleh skor SUS keseluruhan sistem.

VI.1.0.0.2 b. Pengujian Traceability — Audit Log dan Pengukuran Latensi Pengujian *traceability* bertujuan untuk memverifikasi bahwa seluruh aktivitas pergerakan barang dan interaksi robot tercatat secara lengkap dan berurutan tanpa kehilangan data dalam sistem WMC. Sesuai spesifikasi B200, terdapat dua parameter kuantitatif yang harus dipenuhi: (1) latensi sinkronisasi data antara robot, server, dan WMS tidak melebihi 200 ms, dan (2) pembaruan status robot pada antarmuka pengguna terjadi dalam waktu tidak lebih dari 1 detik.

Metode pengujian dilaksanakan melalui dua pendekatan komplementer. Pertama, dilakukan audit log sistem dengan mensimulasikan alur operasional lengkap yang mencakup skenario *inbound* (penerimaan barang), *storage* (penempatan di rak), dan *outbound* (pengambilan barang). Setiap kejadian fisik dibandingkan dengan catatan yang tersimpan dalam basis data WMC untuk memastikan tidak terjadi kehilangan data maupun ketidakurutan pencatatan. Kedua, dilakukan pengukuran latensi respons API menggunakan alat pengujian HTTP dengan mencatat waktu respons pada setiap *endpoint goods-service* dan *auth-service* dalam kondisi operasional normal. Pengukuran dilakukan terhadap sejumlah sampel permintaan berturut-turut dan hasilnya dianalisis secara statistik untuk mendapatkan nilai rata-rata, median, dan persentil ke-95 dari latensi respons.

VI.1.0.0.3 c. Pengujian Productivity — Pengukuran Throughput Tugas Pengujian produktivitas dilakukan untuk mengukur kemampuan sistem WMC dalam memproses dan mendistribusikan perintah tugas kepada armada robot secara efisien. Merujuk pada spesifikasi yang ditetapkan dalam dokumen B200, sistem dinyatakan memenuhi kriteria apabila mampu memfasilitasi penyelesaian minimal 20 tugas pengiriman per jam per unit robot dengan waktu rata-rata penyelesaian ± 3 menit per tugas. Dari perspektif WMC, fokus pengujian ini terletak pada kemampuan *Fleet Order Service* dalam menerima, membuat, dan mendistribusikan *task* kepada robot secara tepat waktu, serta kemampuan sistem dalam mencatat status penyelesaian

setiap tugas melalui log robot.

Prosedur pengujian dilaksanakan dengan menjalankan sistem secara terintegrasi selama satu sesi operasional penuh. Sejumlah perintah *order picking* dan *storing* diberikan secara berurutan melalui antarmuka WMC. Waktu pembuatan *task*, waktu penerimaan oleh robot, dan waktu penyelesaian tugas dicatat dari log sistem. Jumlah tugas yang berhasil diselesaikan per jam kemudian dihitung dan dibandingkan dengan nilai target minimum yang tertera dalam spesifikasi.

VI.1.0.0.4 d. Pengujian Task Fulfilment Accuracy — Verifikasi Konsistensi

Data Pengujian akurasi pemenuhan tugas bertujuan untuk memverifikasi bahwa sistem WMC mampu menghasilkan dan mengelola perintah tugas dengan tingkat akurasi sebesar 100%, tanpa terjadi kesalahan lokasi rak (*misplacement*) dan tanpa ketidaksesuaian antara kondisi fisik dengan data inventori yang tersimpan. Dari sisi WMC, kriteria ini mencerminkan kualitas logika *Fleet Order Service* dalam menentukan lokasi target, serta ketepatan *Goods Detail Service* dalam memperbarui status dan posisi barang setelah setiap tugas diselesaikan.

Metode pengujian dilakukan dengan membandingkan data yang tercatat dalam basis data WMC terhadap kondisi fisik aktual setelah serangkaian tugas pengambilan dan penempatan barang dieksekusi. Setiap *order* yang dibuat melalui antarmuka WMC dicatat beserta lokasi tujuan yang ditetapkan oleh sistem. Setelah robot menyelesaikan tugas, data lokasi barang yang diperbarui oleh sistem dibandingkan dengan lokasi fisik aktual di gudang. Tingkat akurasi dihitung menggunakan formula berikut:

$$\text{Task Fulfilment Accuracy} = \frac{\text{Jumlah Tugas Berhasil Tanpa Kesalahan}}{\text{Jumlah Total Tugas}} \times 100\% \quad (\text{VI.1})$$

Sistem dinyatakan lulus pengujian apabila seluruh tugas yang dieksekusi memberikan hasil yang sesuai antara perintah sistem dengan kondisi fisik, sehingga akurasi mencapai 100%.

Rangkuman seluruh kriteria evaluasi beserta parameter target dan metode pengujianya disajikan pada Tabel VI.1.

Keempat metode evaluasi tersebut dirancang agar dapat dilaksanakan secara bertahap dan sistematis. Pengujian *usability* dan *traceability* dapat dilakukan secara mandiri dalam lingkungan pengembangan, sementara pengujian produktivitas dan akurasi memerlukan integrasi penuh antara sistem WMC dengan subsistem robot dan *Fleet Controller* dalam skenario operasional terintegrasi. Hasil dari masing-

Tabel VI.1 Rangkuman kriteria dan metode evaluasi WMC

No	Kriteria Evaluasi	Parameter Target (B200)	Metode Pengujian
1	Usability (Perangkat Lunak)	Skor SUS > 80	Kuesioner SUS dengan 5 responden
2	Traceability	Sinkronisasi data ≤ 200 ms; pembaruan status robot ≤ 1 detik; data tercatat tanpa kehilangan	Audit log skenario <i>inbound-storage-outbound</i> dan pengukuran latensi API
3	Productivity	≥ 20 tugas/jam per robot; rata-rata waktu penyelesaian ± 3 menit/tugas	Pencatatan jumlah <i>task</i> selesai per jam dan waktu penyelesaian dari log sistem
4	Task Fulfilment Accuracy	Akurasi 100%; tanpa <i>misplacement</i> ; konsistensi data fisik dan inventori	Perbandingan data basis data WMC terhadap kondisi fisik aktual pasca eksekusi tugas

masing pengujian dipaparkan secara rinci pada Subbab VI.2.

VI.2 Hasil Evaluasi

Sub-bab ini menyajikan hasil pelaksanaan setiap metode evaluasi yang telah direncanakan pada Subbab 6.1. Pengujian dilaksanakan secara bertahap, dimulai dari pengujian fungsionalitas sistem berdasarkan skenario *use case*, dilanjutkan dengan pengujian *usability*, *traceability*, produktivitas, dan akurasi pemenuhan tugas. Seluruh pengujian dilaksanakan dalam lingkungan terintegrasi antara sistem WMC, basis data, dan subsistem robot.

VI.2.1 Hasil Pengujian Fungsionalitas

Pengujian fungsionalitas dilakukan untuk memverifikasi bahwa setiap fitur sistem WMC berjalan sesuai dengan skenario *use case* yang telah didefinisikan pada Bab IV. Skenario pengujian mencakup fitur yang diuji sesuai *use case*, input pengguna, *output* yang diharapkan, dan status pengujian. Hasil pengujian dirangkum pada Tabel VI.2.

Berdasarkan hasil pengujian pada Tabel VI.2, seluruh 21 skenario pengujian fungsionalitas menunjukkan hasil yang sesuai dengan *output* yang diharapkan. Sistem WMC berhasil menangani baik skenario alur utama maupun skenario alternatif pada setiap *use case*, termasuk kondisi data kosong, input tidak valid, dan keterbatasan

sumber daya seperti stok habis atau robot tidak tersedia.

VI.2.2 Hasil Pengujian Usability

Pengujian *usability* dilaksanakan dengan melibatkan lima orang responden yang mewakili profil pengguna sistem WMC. Setiap responden menjalankan skenario tugas mencakup proses login, pengelolaan data barang, pembuatan *order*, dan pemantauan status robot, kemudian mengisi kuesioner *System Usability Scale* (SUS) secara mandiri. Perhitungan skor SUS mengikuti formula standar, di mana skor akhir merupakan hasil penjumlahan selisih nilai setiap butir terhadap *baseline* dikalikan 2,5. Hasil pengujian disajikan pada Tabel VI.3.

Rata-rata skor SUS yang diperoleh adalah 84,5, melampaui nilai target minimum 80 yang ditetapkan dalam spesifikasi B200. Skor tersebut masuk dalam kategori ?Good? hingga ?Excellent? pada skala adjektif SUS (Bangor, Kortum, dan Miller 2009), yang mengindikasikan bahwa antarmuka sistem WMC telah memiliki tingkat kemudahan penggunaan yang memadai untuk mendukung operasional gudang secara efektif. Skor terendah berasal dari responden dengan latar belakang operator baru, yang mengindikasikan potensi peningkatan pada aspek orientasi pengguna dan panduan antarmuka.

VI.2.3 Hasil Pengujian Traceability

Pengujian *traceability* dilaksanakan melalui dua tahap. Pertama, pengukuran latensi respons API pada seluruh *endpoint* utama sistem WMC menggunakan alat pengujian HTTP. Setiap *endpoint* diuji dengan 30 iterasi permintaan berturut-turut pada kondisi operasional normal, kemudian dianalisis nilai rata-rata, median, dan persentil ke-95 dari waktu respons. Kedua, dilakukan audit log dengan mensimulasikan alur operasional lengkap skenario *inbound-storage-outbound* untuk memverifikasi kelengkapan dan ketepatan waktu pencatatan setiap kejadian.

Seluruh *endpoint* sistem WMC menunjukkan latensi respons yang berada di bawah ambang batas 200 ms yang ditetapkan dalam spesifikasi B200, dengan nilai persentil ke-95 tertinggi sebesar 143 ms pada *endpoint* pembuatan *order*. Hasil audit log juga mengonfirmasi bahwa seluruh kejadian fisik dalam alur *inbound-storage-outbound* tercatat secara lengkap, berurutan, dan tepat waktu dalam basis data WMC tanpa terjadi kehilangan data. Pembaruan status robot pada antarmuka pengguna terjadi dalam waktu rata-rata 0,72 detik, memenuhi persyaratan ≤ 1 detik.

VI.2.4 Hasil Pengujian Produktivitas

Pengujian produktivitas dilaksanakan dalam satu sesi operasional terintegrasi selama 60 menit. Perintah *order picking* dan *storing* diberikan secara berurutan melalui antarmuka WMC, dan sistem mencatat waktu pembuatan *task*, waktu penerimaan oleh robot, serta waktu penyelesaian setiap tugas dari log robot. Hasil pengujian dirangkum pada Tabel VI.7.

Sistem WMC berhasil memfasilitasi penyelesaian 24 *task* per jam per robot, melampaui target minimum 20 *task* per jam yang ditetapkan dalam spesifikasi B200. Waktu penyelesaian rata-rata per *task* adalah 2 menit 35 detik, lebih cepat dari target ± 3 menit per *task*. Dua *task* yang gagal diselesaikan disebabkan oleh robot yang kehabisan daya baterai di tengah operasi, yang merupakan kendala pada subsistem perangkat keras dan bukan pada logika sistem WMC.

VI.2.5 Hasil Pengujian Task Fulfilment Accuracy

Pengujian akurasi pemenuhan tugas dilaksanakan dengan mengeksekusi serangkaian tugas *picking* dan *storing* melalui sistem WMC dan kemudian membandingkan data yang tercatat dalam basis data dengan kondisi fisik aktual di gudang. Setiap *task* diperiksa untuk memastikan tidak terjadi kesalahan lokasi rak (*misplacement*) dan tidak ada ketidaksesuaian antara kondisi fisik dengan data inventori. Hasil pengujian disajikan pada Tabel VI.8.

Seluruh 20 *task* yang dieksekusi menunjukkan kesesuaian penuh antara lokasi target yang ditetapkan oleh sistem WMC dengan kondisi fisik aktual di gudang. Tidak terdapat satu pun kejadian *misplacement* maupun ketidaksesuaian data inventori, sehingga tingkat *Task Fulfilment Accuracy* mencapai 100%, memenuhi target spesifikasi B200. Hasil ini mengonfirmasi bahwa logika *Fleet Order Service* dalam menentukan lokasi target dan mekanisme pembaruan data pada *Goods Detail Service* berjalan dengan benar dan andal.

VI.2.6 Hasil Pengujian Lokalisasi

Verifikasi Subsistem Lokalisasi dilakukan dengan menempatkan *marker* Tag36h11 AprilTag (8×8 cm) pada posisi dan orientasi yang telah diketahui secara pasti di area lantai gudang, kemudian membandingkan *payload* MQTT yang dipublikasikan oleh subsistem terhadap *ground truth* untuk dua properti: posisi sel *grid* dan sudut orientasi (*heading*). Target keberhasilan adalah akurasi 100% pada kedua properti tersebut.

Seluruh 6 kasus verifikasi posisi dan 4 kasus verifikasi orientasi berhasil lulus de-

ngan akurasi 100%. Hasil ini mengonfirmasi bahwa *pipeline* Subsistem Lokalisasi—mulai dari koreksi distorsi *fish-eye*, transformasi *homography*, hingga *snap* ke sel *grid* diskrit—berfungsi dengan benar dan mampu melakukan pemetaan posisi dan deteksi orientasi secara andal pada seluruh titik pengujian di area lantai gudang.

Secara keseluruhan, sistem *Warehouse Management Controller* (WMC) dan Subsistem Lokalisasi yang dikembangkan telah memenuhi seluruh kriteria evaluasi yang ditetapkan. Kelima aspek yang diuji, yaitu *usability*, *traceability*, produktivitas, akurasi pemenuhan tugas, serta akurasi lokalisasi posisi dan orientasi, semuanya menunjukkan hasil yang melampaui atau memenuhi nilai target minimum yang disyaratkan. Hal ini mengonfirmasi bahwa WMC yang diimplementasikan layak digunakan sebagai komponen perangkat lunak inti dalam sistem *Smart Warehouse* Paragon Corp.

Tabel VI.2 Skenario dan hasil pengujian fungsionalitas WMC

ID	UC	Input	Output yang Dihasilkan	Status
FTA-01	UC-01	Username dan password valid	Login berhasil, JWT token diberikan, pengguna diarahkan ke halaman <i>dashboard</i>	✓
FTA-02	UC-01	Password tidak sesuai	Login gagal, pesan ?Username atau password salah? ditampilkan, Respons 401	✓
FTA-03	UC-01	Field username atau password dikosongkan	Pesan validasi ?Kolom tidak boleh kosong? ditampilkan	✓
FTA-04	UC-02	Pengguna menekan tombol logout	Logout berhasil, sesi aktif dihapus dari basis data, pengguna diarahkan ke halaman login	✓
FTA-05	UC-03	Halaman inventori dibuka, data barang tersedia	Daftar barang ditampilkan beserta nama, SKU, kategori, dan stok	✓
FTA-06	UC-03	Halaman inventori dibuka, tidak ada data barang	Pesan ?Belum ada barang terdaftar? ditampilkan	✓
FTA-07	UC-04	Pengguna memilih barang dengan ID valid	Halaman detail barang ditampilkan: nama, SKU, deskripsi, dimensi, berat, kategori, dan stok	✓
FTA-08	UC-04	ID barang tidak valid atau barang telah dihapus	Pesan ?Data barang tidak ditemukan? ditampilkan	✓
FTA-09	UC-05	Pengguna mengakses lokasi barang yang terdaftar	Informasi lokasi ditampilkan: zona, rak (rack), dan slot penyimpanan	✓
FTA-10	UC-05	Barang tidak memiliki lokasi terdaftar	Pesan ?Lokasi barang belum terdaftar? ditampilkan	✓
FTA-11	UC-06	Tipe picking, barang dipilih, stok mencukupi, robot tersedia	Order picking berhasil dibuat, task robot dibuat dengan status Pending, konfirmasi ditampilkan	✓
FTA-12	UC-06	Tipe picking, stok barang tidak mencukupi	Pesan ?Stok tidak mencukupi? ditampilkan, order tidak terbuat	✓
FTA-13	UC-06	Tipe picking, stok mencukupi, tidak ada robot tersedia	Order berhasil dibuat dengan status Waiting, task menunggu robot yang tersedia	✓
FTA-14	UC-07	Tipe storing, barang dipilih, lokasi penyimpanan tersedia	Order storing berhasil dibuat, task robot dibuat dengan status Pending	✓

Tabel VI.3 Hasil pengujian usability (SUS)

No	Profil Responden	Skenario Tugas yang Dijalankan	Skor SUS
1	Operator Gudang (pengalaman >2 tahun)	Login, daftar barang, buat order picking, pantau task	85,0
2	Supervisor Operasional	Login, detail barang, buat order storing, pantau robot	82,5
3	Staf IT / Administrator Sistem	Login, seluruh fitur inventori, buat order, pantau task dan robot	90,0
4	Operator Gudang (baru, belum terbiasa sistem)	Login, daftar barang, buat order picking	77,5
5	Manajer Operasional (pengguna non-teknis)	Login, daftar order, pantau status robot	87,5

Tabel VI.4 Rekapitulasi skor SUS

Parameter	Nilai
Skor SUS Tertinggi	90,0
Skor SUS Terendah	77,5
Rata-rata Skor SUS	84,5
Target Minimum (B200)	> 80
Status	Terpenuhi ✓

Tabel VI.5 Hasil pengukuran latensi respons API

Layanan	Endpoint	Rata-rata (ms)	Median (ms)	P95 (ms)	Status
auth-service	/auth/login (POST)	45	42	78	✓ ≤200ms
auth-service	/protected (GET)	18	17	32	✓ ≤200ms
goods-service	/items (GET)	62	58	112	✓ ≤200ms
goods-service	/item-locations (GET)	55	52	98	✓ ≤200ms
goods-service	/orders (POST)	88	84	143	✓ ≤200ms
goods-service	/tasks (POST)	76	72	128	✓ ≤200ms
goods-service	/robot-logs (GET)	49	46	87	✓ ≤200ms

Tabel VI.6 Hasil audit log skenario Inbound–Storage–Outbound

No	Kejadian Fisik	Catatan di Basis Data WMC	Selisih Waktu Pencatatan	Status
1	Barang tiba di area inbound	Order storing dibuat, item location diperbarui	$\leq 150\text{ms}$	✓
2	Robot mengambil barang dari docking area	Task dibuat dan ditetapkan ke robot, status Pending	$\leq 120\text{ms}$	✓
3	Robot tiba di lokasi rak target	Robot log mencatat posisi dan status Active	$\leq 180\text{ms}$	✓
4	Barang ditempatkan di rak (storing selesai)	Status task diperbarui Completed, item location diperbarui	$\leq 160\text{ms}$	✓
5	Order picking dibuat oleh operator	Order dan order item dibuat, task ditetapkan ke robot	$\leq 130\text{ms}$	✓
6	Barang diambil dari rak (picking selesai)	Status task Completed, stok item diperbarui	$\leq 170\text{ms}$	✓

Tabel VI.7 Hasil pengujian produktivitas sistem WMC

Parameter	Hasil
Durasi sesi pengujian	60 menit
Jumlah robot yang digunakan	2 unit
Total task yang diberikan	50 task
Total task berhasil diselesaikan	48 task
Task yang gagal / dibatalkan	2 task (robot low battery)
Jumlah task selesai per jam (per robot)	24 task/jam
Waktu penyelesaian rata-rata per task	2 menit 35 detik
Target minimum throughput (B200)	≥ 20 task/jam per robot
Target waktu penyelesaian (B200)	± 3 menit per task
Status	Terpenuhi ✓

Tabel VI.8 Hasil pengujian Task Fulfilment Accuracy

Tipe Task	Jumlah Task Dieksekusi	Task Tanpa Kesalahan Lokasi	Akurasi
Storing (penempatan barang)	10	10	100%
Picking (pengambilan barang)	10	10	100%
Total keseluruhan	20	20	100%

Tabel VI.9 Hasil verifikasi posisi Subsistem Lokalisasi

Tag ID	Sel Grid yang Diharapkan (c, r)	Status
562	(0, 1)	✓Lulus
584	(0, 2)	✓Lulus
577	(1, 2)	✓Lulus
585	(3, 1)	✓Lulus
586	(5, 1)	✓Lulus
578	(2, 3)	✓Lulus

Tabel VI.10 Hasil verifikasi orientasi Subsistem Lokalisasi

Tag ID	Heading yang Diharapkan	Status
584	0°	✓Lulus
562	90°	✓Lulus
578	180°	✓Lulus
577	270°	✓Lulus

BAB VII

PENUTUP

VII.1 Kesimpulan

Berdasarkan proses analisis masalah, pemilihan solusi, perancangan, implementasi, dan evaluasi yang telah dilakukan, kedua rumusan masalah tugas akhir ini dapat dijawab sebagai berikut.

1. Permasalahan keterbatasan sinkronisasi data secara *real-time* dan ketergantungan pada proses manual di gudang barang jadi Paragon Corp, yang memicu *mismatch* dan *mistracking* stok, dijawab melalui pembangunan sistem informasi manajemen gudang berbasis arsitektur *microservices* dengan pendekatan *event-driven*. Sistem ini didekomposisi menjadi tiga layanan utama, yaitu *Authentication Service*, *Goods Detail Service*, dan *Fleet Order Service* sebagai pengelola antrean tugas, didukung basis data dengan sembilan entitas dan sepuluh *use case* (UC-01 sampai UC-10). Implementasi menggunakan Node.js dan TypeScript untuk *backend*, PostgreSQL melalui Prisma ORM untuk basis data, serta Docker untuk *containerization*, dengan total 41 *endpoint* REST API yang dilindungi autentikasi JWT. Dengan pendekatan ini, sistem mampu menjaga konsistensi data stok secara otomatis sekaligus menyajikan informasi inventori secara *real-time*.
2. Efektivitas sistem dibuktikan melalui hasil evaluasi, yaitu seluruh 21 skenario pengujian fungsionalitas berhasil dilalui dengan baik, skor *System Usability Scale* (SUS) rata-rata mencapai 84,5 yang melampaui target minimal di atas 80, seluruh latensi respons API berada di bawah 200 ms dengan nilai P95 tertinggi sebesar 143 ms, pembaruan status rata-rata terjadi dalam waktu 0,72 detik disertai *audit log* yang lengkap tanpa kehilangan data, pemrosesan antrean tugas *order* mencapai 24 tugas per jam yang melampaui target minimal 20 tugas per jam dengan waktu penyelesaian rata-rata 2 menit 35 detik, serta konsistensi data tercatat sebesar 100% tanpa ditemukan kasus *mismatch* maupun *misplacement*.
3. Untuk kebutuhan penentuan posisi dan orientasi robot, dikembangkan prototi-

pe Subsistem Lokalisasi yang memanfaatkan penanda AprilTag jenis Tag36h11. Subsistem ini bekerja melalui beberapa tahapan, yaitu koreksi distorsi lensa *fish-eye* menggunakan model Kannala–Brandt, transformasi perspektif dengan pendekatan *homography*, hingga proses *snapping* posisi ke sel *grid* diskrit, sebelum akhirnya posisi dan orientasi robot dipublikasikan melalui protokol MQTT untuk menunjang pelaksanaan tugas pemindahan barang. Hasil verifikasi menunjukkan tingkat akurasi sebesar 100% pada enam kasus uji posisi sel *grid* serta empat kasus uji orientasi pada sudut hadap 0°, 90°, 180°, dan 270°, sehingga dapat disimpulkan bahwa mekanisme ini mampu menentukan posisi dan orientasi robot secara akurat di seluruh titik pengujian.

VII.2 Saran

Meskipun sistem WMC telah berhasil dikembangkan dan menghasilkan hasil evaluasi yang memenuhi seluruh kriteria spesifikasi, masih terdapat ruang untuk pengembangan dan penyempurnaan lebih lanjut agar sistem dapat menjadi lebih optimal dalam mendukung operasional gudang dan dapat diterapkan dalam skala yang lebih luas. Beberapa saran yang dapat dipertimbangkan untuk kebutuhan pengembangan selanjutnya adalah:

1. Integrasi dengan sistem ERP eksisting Paragon Corp (Odoo) secara *real-time*, sehingga data inventori pada WMC dapat tersinkronisasi secara otomatis dengan sistem perencanaan sumber daya perusahaan yang telah ada. Integrasi ini akan menghilangkan kebutuhan rekonsiliasi data manual dan memastikan konsistensi informasi di seluruh lini operasional perusahaan.
2. Penggantian mekanisme pembaruan status dari *polling* berbasis REST API menjadi komunikasi berbasis WebSocket atau *Server-Sent Events* (SSE) secara menyeluruh pada seluruh antarmuka, sehingga pembaruan posisi dan status robot pada antarmuka pengguna dapat berlangsung secara *push real-time* tanpa mengandalkan permintaan berulang dari sisi klien. Hal ini akan meningkatkan efisiensi jaringan sekaligus memberikan pengalaman pemantauan operasional yang lebih responsif.
3. Penambahan modul analitik dan pelaporan operasional yang menyajikan visualisasi data historis, seperti tren produktivitas robot, pola penggunaan rak, dan statistik penyelesaian *order* per periode. Fitur ini akan membantu supervisor dan manajer operasional dalam pengambilan keputusan berbasis data untuk optimasi tata letak gudang dan perencanaan kapasitas armada robot.
4. Pengembangan antarmuka berbasis aplikasi *mobile* untuk memudahkan operator gudang dalam memantau dan mengelola operasional secara langsung dari lantai gudang tanpa perlu mengakses *workstation* tetap, sehingga mening-

katkan fleksibilitas dan kecepatan respons terhadap kondisi operasional yang berubah secara dinamis.

DAFTAR PUSTAKA

- Bangor, Aaron, Philip Kortum, dan James Miller. 2009. “Determining What Individual SUS Scores Mean: Adding an Adjective Rating Scale”. *Journal of Usability Studies* 4 (3): 114–123.
- Bradski, Gary. 2000. “The OpenCV Library”. *Dr. Dobb’s Journal of Software Tools* 25 (11): 120–125.
- Brazhkin, Vitaliy. 2018. “An Analysis of Key Warehouse Resources”. *Journal of Contemporary Business Issues* 23 (1).
- Codd, Edgar F. 1970. “A Relational Model of Data for Large Shared Data Banks”. *Communications of the ACM* 13 (6): 377–387.
- Conexiom. 2024. “The State of Order Management”. Conexiom Inc. <https://conexiom.com/resources/state-of-order-management/>.
- Fette, Ian, dan Alexey Melnikov. 2011. *The WebSocket Protocol*. Technical report RFC 6455. Internet Engineering Task Force (IETF). <https://www.rfc-editor.org/rfc/rfc6455>.
- Fowler, Martin. 2002. *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley.
- Framinan, Jose M., Rafael Ruiz-Usano, dan Rainer Leisten. 2025. “Redesigning the Customer Order Fulfilment Process in a Company in the FMCG Sector”. *IFAC-PapersOnLine* (Amsterdam) 58 (19): 1–6.
- Geest, Marijn van, Bedir Tekinerdogan, dan Cagatay Catal. 2021. “Smart Warehouses: Rationale, Challenges, and Solution Directions”. Art. no. 103523, *Computers in Industry* (Amsterdam) 132.
- Habazin, Josip, Anica Glasnovic, dan Ivan Bajor. 2017. “Order Picking Process in Warehouse: Case Study of Dairy Industry in Croatia”. *Promet – Traffic & Transportation* 29 (1): 57–65.

- Hartley, Richard, dan Andrew Zisserman. 2004. *Multiple View Geometry in Computer Vision*. 2nd. Cambridge, UK: Cambridge University Press.
- Hohpe, Gregor, dan Bobby Woolf. 2003. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA, USA: Addison-Wesley.
- IHL Group. 2023. “Retailers and the Ghost Economy: The Uncounted \$1.8 Trillion Opportunity”. IHL Group.
- International Organization for Standardization. 2015. *Quality Management Systems — Requirements*. ISO 9001:2015. Geneva: International Organization for Standardization.
- KADIN Indonesia. 2025. “Pertumbuhan Pasar FMCG Indonesia Sektor Kecantikan dan Perawatan Diri”. Kamar Dagang dan Industri Indonesia.
- NetSuite. 2024. “Warehouse Picking Accuracy: Benchmarks and Best Practices”. Oracle NetSuite. <https://www.netsuite.com/portal/resource/articles/inventory-management/warehouse-picking-accuracy.shtml>.
- Newman, Sam. 2015. *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O’Reilly Media.
- OASIS. 2014. *MQTT Version 3.1.1*. OASIS Standard. OASIS. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>.
- Precedence Research. 2025. “Smart Warehousing Market Size, Share, and Trends 2025–2033”. Precedence Research.
- RabbitMQ. 2023. “RabbitMQ Documentation: AMQP 0-9-1 Model Explained”. VMware, Inc. <https://www.rabbitmq.com/tutorials/amqp-concepts.html>.
- Ramakrishnan, Raghu, dan Johannes Gehrke. 2003. *Database Management Systems*. 3rd. New York, NY, USA: McGraw-Hill.
- Research and Markets. 2025. “Fast Moving Consumer Goods (FMCG) Global Market Report 2025”. Research dan Markets.
- Richardson, Chris. 2018. *Microservices Patterns*. Shelter Island, NY, USA: Manning Publications.

- Romero, David, Wided Guedria, Herve Panetto, dan Beatrix Barafort. 2020. "Towards a Characterisation of Smart Systems: A Systematic Literature Review". *Computers in Industry*.
- Saaty, Thomas L. 1977. "A Scaling Method for Priorities in Hierarchical Structures". *Journal of Mathematical Psychology* 15 (3): 234–281.
- SCM Champs. 2024. "The True Cost of Picking Errors in Warehouse Operations". SCM Champs.
- Silberschatz, Abraham, Henry F. Korth, dan S. Sudarshan. 2020. *Database System Concepts*. 7th. McGraw-Hill.
- Tim TA252601001. 2025a. *Dokumen B100-TA2526.01.001-008: Analisis Kebutuhan Sistem Smart Warehouse Berbasis Mobile Robot*. Technical report. Bandung: Institut Teknologi Bandung.
- . 2025b. *Dokumen B200-TA2526.01.001-005: Spesifikasi Sistem Smart Warehouse Berbasis Mobile Robot*. Technical report. Bandung: Institut Teknologi Bandung.
- . 2025c. *Dokumen B300-TA2526.01.001-007: Desain Sistem Smart Warehouse Berbasis Mobile Robot*. Technical report. Bandung: Institut Teknologi Bandung.
- Universitas Diponegoro. 2019. *Bab II: Gambaran Umum PT Paragon Technology and Innovation*. Technical report. Semarang: Fakultas Ekonomika dan Bisnis, Universitas Diponegoro.

LAMPIRAN A

SOURCE CODE

A.1 Perangkat Lunak untuk Akuisisi Data dari Sensor Ultrasonik

```
1 % -- Example of pseudocode and source code listing --
2 % -- Gunakan minipage agar listing tidak terpotong ke halaman
   berikutnya --
3
4
5
6 \begin{minipage}{\textwidth}
7 \begin{lstlisting}[caption={Iterative binary search in Python},
   label={lst:binary_iter}, frame=single, captionpos=t, language=
   Python]
8 def binary_search(arr, target):
9     """
10     Performs binary search on a sorted array.
11
12     Args:
13         arr: A sorted list of elements
14         target: The element to search for
15
16     Returns:
17         Index of target if found, -1 otherwise
18     """
19     left = 0
20     right = len(arr) - 1
21
22     while left <= right:
23         mid = (left + right) // 2
24
25         if arr[mid] == target:
26             return mid
27         elif arr[mid] < target:
28             left = mid + 1
29         else:
30             right = mid - 1
```

```
31  
32     return -1  
33  
34  
35 \end{lstlisting}  
36 \end{minipage}
```

Listing A.1 Binary search code

A.2 Perangkat Lunak untuk Pengolahan Data Akuisisi dan Visualisasi Hasil Pengukuran Jarak

asdjfkjashdkjfas

LAMPIRAN B

HASIL SURVEI LAPANGAN

B.1 Foto Survei Lokasi 1

Teks survei lokasi 1.

B.2 Foto Survei Lokasi 2

Teks survei lokasi 2.

B.3 Transkrip Wawancara dengan Narasumber

Teks transkrip wawancara.

